

Introduction to Quantum Algorithms and Quantum Programming

B. Valiron

Course Notes v.2026.02.01

Contents

1	Introduction	3
2	Mathematical Background	4
2.1	Notations	4
2.2	Sums and series	4
2.3	Complex Numbers	5
2.4	Vector Space	7
2.5	Scalar Product	8
2.6	Kets and Bras	9
2.7	Kronecker product	11
2.8	Linear Maps	13
2.9	Hermitian and Unitary Maps	15
2.10	Exercices	18
3	Qubit-based Computation	21
3.1	The Quantum Co-Processor Model	21
3.2	One Quantum Bit	22
3.3	Several Quantum Bits	25
3.4	The Quantum Circuit Model	27
3.5	Quantum Gates on 1 Qubit	31
3.6	Quantum Gates on Several Qubits	34
3.7	Creating New Quantum Registers	37
3.8	Reading Quantum Registers	37
3.9	Discarding Quantum Registers	42
3.10	Cloning, Copy, Teleportation	46
3.11	Exercices	48
4	Hardware Constraints and Circuit Synthesis	52
4.1	A bit of Complexity Theory	52
4.2	Low-level gate-sets	53
4.3	Universality of CNOT and 1-qubit rotations	55

4.4	Tradeoffs: a Case-Study	58
4.5	Quantum Computation with Magic States	60
4.6	Measurement-Based Quantum Computation	62
4.7	Classical Computation in the Co-Processor	67
4.8	A Word on Hardware	68
4.9	Focus: Linear Optical Quantum Computation	71
4.10	Exercises	82
5	Structure of Quantum Algorithms	85
5.1	High-Level View	85
5.2	Oracles	86
5.3	Encoding Natural Numbers	91
5.4	Amplitude Amplification	95
5.5	Quantum Fourier Transform	99
5.6	Phase Estimation	102
5.7	Trotterization	105
5.8	Exercises	110
6	Algorithms for LSQ era	112
6.1	Simple Oracle-Based Algorithms	112
6.2	Shor	115
6.3	HHL	119
6.4	Exercises	125
7	Algorithms for NISQ era	126
7.1	Variational Algorithms	126
7.2	VQE	126
7.3	QAOA	130
7.4	Example: MAXCUT and QUBO problems	134
7.5	Exercises	136
A	Cosine-Sine Decomposition	137
A.1	Statement	137
A.2	Proof	137
	Some Standard Elementary Quantum Gates	142
	References	143
	Index	145

1 Introduction

This set of notes encapsulates most of the material I am teaching on the topic of quantum algorithm, quantum programming and quantum compiling. The target is a student at Master level in Computer Science, with some acquaintance with mathematics. Section 2 attempts at covering the mathematical knowledge necessary to follow.

As additional standard, published material, I can recommend:

- [KLM07] Kaye, Laflamme and Mosca, *An Introduction to Quantum Computing*, Oxford University Press, 2007.
- [Mer07] Mermin, *Quantum Computer Science - An Introduction*, Cambridge University Press, 2007.
- [NC02] Nielsen and Chuang, *Quantum Computation and Quantum Information*, Cambridge University press, whose last edition is 2010.

Regarding quantum programming, a framework that is now ubiquitous is Qiskit, a python library developed by IBM. Their online documentation and tutorials are very exhaustive and worth a look ¹.

Finally, MBQC and linear optics are maybe not standard yet. I can however recommend the following readings:

- Kok *et al.*'s review on quantum computation with linear optics [KMN⁺07].
- Danos, Kashefi, Panangaden and Perdrix's chapter on MBQC [DKPP09a]. Section 4.6 in this set of notes heavily relies on this document.

¹<https://docs.quantum.ibm.com/guides>

2 Mathematical Background

2.1 Notations

2.1.1 The set of natural numbers is $\mathbb{N}$. Addition is written “+” and multiplication “·”. When clear, we simply use concatenation for the multiplication. For instance, $2n$ is $2 \cdot n$.

2.1.2 The factorial $n!$ is $1 \cdot 2 \cdot 3 \cdot 4 \cdot \dots \cdot n$. By convention $0! = 1$.

2.1.3 We assume known the notion of *real number*. The set of reals is denoted with $\mathbb{R}$. For the record, we recall that real numbers form a *field*: the set $\mathbb{R}$ is equipped with

- An addition “+” and a null element “0”, also called *zero*.
- A multiplication “·” and a neutral element “1”
- An inverse operation written $(-)^{-1}$, or $\frac{1}{-}$, defined on all non-zero real number.

2.1.4 Real intervals are written $[a, b]$ when both a and b are part of the interval, and $[a, b)$, $(a, b]$ or (a, b) when the right, or the left, or both ends of the interval are not included. Following the standard notation, we use the placeholders $-\infty$ and ∞ to represent unbounded intervals on the left or on the right.

2.1.5 We use the representation $\sqrt{x}$ for the square root of x , and x^a for x to the power a . The exponent a can be negative: in this case, we mean the power of the inverse of x . For instance, $x^{-2} = \frac{1}{x^2}$.

2.1.6 The 2-elements Boolean algebra is written $\mathbb{B}$. Its objects are 0 and 1, standing respectively for “False” and “True”. The “and” operation (the conjunction) is denoted with $\wedge$, or simply as a product “·”, or by juxtaposition: $x \wedge y = x \cdot y = xy$. The “or” (the disjunction) is written $\vee$. The “not” (the negation) is written $\neg$. The exclusive or (also known as XOR) is written with $\oplus$.

2.2 Sums and series

Along this document we shall be using sums over finite and infinite sets of indices. In this section, we summarize what should be known.

2.2.1 Sum symbol. Given a function $f : \mathbb{N} \rightarrow \mathbb{R}$, and given $i \leq j \in \mathbb{N}$, we define

$$\sum_{n=i}^j f(n) \triangleq f(i) + f(i+1) + \dots + f(j).$$

Provided that the limit is well-defined, we define

$$\sum_{n=i}^{\infty} f(n) \triangleq f(i) + f(i+1) + \dots \triangleq \lim_{j \rightarrow \infty} \sum_{n=i}^j f(n)$$

$\sum_{n=0}^{\infty} \frac{1}{n!} x^n = e^x \quad (1)$	$\sum_{n=0}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6} \quad (4)$
$\sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n} = \cos(x) \quad (2)$	$\sum_{i=0}^n a^i = \frac{1 - a^{n+1}}{1 - a} \quad (5)$
$\sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = \sin(x) \quad (3)$	$\sum_{i=0}^{\infty} a^i = \frac{1}{1 - a} \quad (6)$

Table 1: Values for a few known series. Eq. (4) is called the *Basel problem*. In Eqs. (1), (3) and (2), x is any real number. In Eq. (5), $a \neq 1$, while Eq. (6) holds whenever $0 \leq a < 1$.

Such a limit is called a *series*. If the limit is defined, we say that the series is *converging*. The series is *absolutely converging* if

$$\sum_{n=i}^{\infty} |f(n)|$$

is converging.

2.2.2 Lists of known series. Table 1 summarizes the values of few known series. Series of the form of Eq. (6) are called *geometric series*. Maybe the most important one for this course is Eq. (1) that we recall here:

$$\sum_{n=0}^{\infty} \frac{1}{n!} x^n = e^x.$$

2.3 Complex Numbers

We write $\mathbb{C}$ for the field of *complex numbers*. A complex number is of the form $\alpha = a + b \cdot i$ with a and b reals and i the *imaginary number*: a number such that $i^2 = -1$.

2.3.1 Conjugate. The *conjugate* of a complex number $\alpha = a + b \cdot i$ is defined as $\bar{\alpha} = a - b \cdot i$. The *absolute value*, or the *norm* of α is the non-negative, real number

$$|\alpha| = \sqrt{a^2 + b^2}.$$

The conjugate and the absolute value are related through the property $|\alpha|^2 = \alpha \cdot \bar{\alpha}$. Indeed

$$\begin{aligned} \alpha \cdot \bar{\alpha} &= (a + b \cdot i)(a - b \cdot i) \\ &= a^2 + ab \cdot i - ab \cdot i + (b \cdot i)(-b \cdot i) \\ &= a^2 - i^2 \cdot b^2 \\ &= a^2 + b^2 \end{aligned}$$

2.3.2 Radial Representation. Consider the complex number $\alpha = a + b \cdot i$, and assume $\alpha \neq 0$. One can rewrite it as

$$\begin{aligned}\alpha &= \frac{|\alpha|}{|\alpha|} (a + b \cdot i) \\ &= |\alpha| \cdot \left(\frac{a}{|\alpha|} + \frac{b}{|\alpha|} \cdot i \right)\end{aligned}$$

We have

$$\begin{aligned}\left(\frac{a}{|\alpha|} \right)^2 + \left(\frac{b}{|\alpha|} \right)^2 &= \frac{a^2}{|\alpha|^2} + \frac{b^2}{|\alpha|^2} \\ &= \frac{a^2 + b^2}{|\alpha|^2} \\ &= 1\end{aligned}$$

since $|\alpha|^2 = a^2 + b^2$. So there exists an angle $\theta \in [0, 2\pi)$ such that $\cos(\theta) = \frac{a}{|\alpha|}$ and $\sin(\theta) = \frac{b}{|\alpha|}$, meaning that

$$\alpha = |\alpha| \cdot (\cos(\theta) + \sin(\theta) \cdot i)$$

If $\alpha \neq 0$, there is a unique such θ .

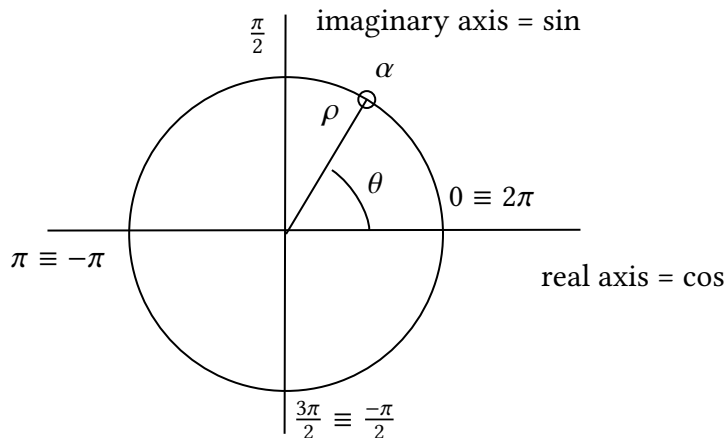
Our complex number can then be written in a canonical form

$$\alpha = \rho (\cos(\theta) + \sin(\theta) \cdot i),$$

with ρ non-negative real number:

- ρ is the *amplitude* of α ,
- θ is the *phase* of α .

Complex numbers can then be represented in the *complex plane* using a *radial representation* as follows.



$e^{i\frac{\pi}{2}} = i$	(7)	$e^{a+b} = e^a e^b$	(11)
$e^{i\pi} = -1$	(8)	$\overline{e^a} = e^{\bar{a}}$	(12)
$e^{2i\pi} = 1$	(9)	$\overline{e^{i\theta}} = e^{-i\theta}$	(13)
$e^{i(\theta+2\pi)} = e^{i\theta}$	(10)	$e^{ab} = (e^a)^b$	(14)

Table 2: Equalities regarding exponentiation. θ is real, a and b are complex values.

2.3.3 Exponentiation. The number $\cos(\theta) + i \cdot \sin(\theta)$ can also be written as $e^{i\theta}$. Indeed, remember Table 1: Replacing x with $i\theta$ in Eq. (1), the definition of e^x , we get

$$\begin{aligned}
e^{i\theta} &= \sum_{n=0}^{\infty} \frac{(i\theta)^n}{n!} \\
&= \sum_{n=0}^{\infty} \frac{(i\theta)^{2n}}{(2n)!} + \sum_{n=0}^{\infty} \frac{(i\theta)^{2n+1}}{(2n+1)!} && \text{(splitting odd and even indices)} \\
&= \sum_{n=0}^{\infty} \frac{i^{2n} \theta^{2n}}{(2n)!} + \sum_{n=0}^{\infty} \frac{i^{2n+1} \theta^{2n+1}}{(2n+1)!} && \text{(developing)} \\
&= \sum_{n=0}^{\infty} \frac{(i^2)^n \theta^{2n}}{(2n)!} + \sum_{n=0}^{\infty} \frac{i(i^2)^n \theta^{2n+1}}{(2n+1)!} && \text{(developing)} \\
&= \sum_{n=0}^{\infty} \frac{(-1)^n \theta^{2n}}{(2n)!} + \sum_{n=0}^{\infty} \frac{i(-1)^n \theta^{2n+1}}{(2n+1)!} && \text{(since } i^2 = -1) \\
&= \sum_{n=0}^{\infty} \frac{(-1)^n \theta^{2n}}{(2n)!} + i \cdot \sum_{n=0}^{\infty} \frac{(-1)^n \theta^{2n+1}}{(2n+1)!} && \text{(factoring by } i) \\
&= \cos(x) + i \cdot \sin(x) && \text{(using Eqs. (2) and (3)).}
\end{aligned}$$

2.4 Vector Space

2.4.1 In this course, we only consider vector spaces in finite dimension over complex numbers.

2.4.2 Let X be a finite set $\{e_0 \dots e_{n-1}\}$. Each e_i is called a *canonical basis element*. A (complex) *vector space* $\mathcal{E}$ with basis X consists of *vectors*. The *dimension* of the vector space is the number of canonical basis elements in X . A vector v can be regarded as a mapping from the canonical basis elements in X to complex values. From a computer science point of view, one can regard this vector as an association table, or as a “Python dictionary”:

$$v = \{e_0 : \alpha_0, \dots, e_{n-1} : \alpha_{n-1}\}.$$

We say that α_i is the *coefficient* of v at coordinate e_i , and we write $v_{e_i} = \alpha_i$.

2.4.3 A vector can be represented as an *array*. This however requires an *ordering* of the canonical basis vectors. Array-like notation are discussed later, in Sec. 2.6.8.

2.4.4 The vector space $\mathcal{E}$ is equipped with a sum and a multiplication by a scalar:

$$(+): \mathcal{E} \times \mathcal{E} \longrightarrow \mathcal{E}$$

$$(\cdot): \mathbb{C} \times \mathcal{E} \longrightarrow \mathcal{E}$$

Their action is pointwise: if b is a canonical basis element of $\mathcal{E}$, $(v + w)_b = v_b + w_b$ and $(\alpha \cdot v)_b = \alpha \cdot (v_b)$.

2.4.5 Ket-notation. A canonical basis element e_i yields a particular vector written $|e_i\rangle$ and called “ket- e_i ”: it is the vector of coefficient 1 at e_i and 0 everywhere else:

$$|e_i\rangle \triangleq e_j \mapsto \begin{cases} 1 & \text{when } j = i \\ 0 & \text{else} \end{cases}.$$

Using this convention, a vector $v \in \mathcal{E}$ can be written as the linear combination

$$\sum_{i=0}^{n-1} v_{e_i} \cdot |e_i\rangle,$$

or, equivalently:

$$\sum_{x \in X} v_x \cdot |x\rangle.$$

Such combinations behave well with sum and scalar multiplication:

$$\begin{aligned} \left(\sum_{x \in X} v_x \cdot |x\rangle \right) + \left(\sum_{x \in X} w_x \cdot |x\rangle \right) &= \sum_{x \in X} (v_x + w_x) \cdot |x\rangle, \\ \alpha \cdot \left(\sum_{x \in X} v_x \cdot |x\rangle \right) &= \sum_{x \in X} (\alpha \cdot v_x) \cdot |x\rangle. \end{aligned}$$

2.5 Scalar Product

2.5.1 Let $\mathcal{E}$ be a complex vector space as above. We define an operation acting on two vectors

$$\langle - | - \rangle : \mathcal{E} \times \mathcal{E} \mapsto \mathbb{C}$$

called a *scalar product*, defined as

$$\langle u | v \rangle \triangleq \sum_x \overline{u_x} v_x.$$

The operation is linear on the right:

$$\langle u | \alpha \cdot v + w \rangle = \alpha \cdot \langle u | v \rangle + \langle u | w \rangle$$

but anti-linear on the left:

$$\langle \alpha \cdot u + v | w \rangle = \overline{\alpha} \cdot \langle u | w \rangle + \langle v | w \rangle.$$

In particular, it is anti-symmetric as follows:

$$\langle u | v \rangle = \overline{\langle v | u \rangle}.$$

2.5.2 A (complex) *Hilbert space* is then a complex vector space equipped with such a scalar product. From this scalar product one can define two notions: a notion of norm, and a notion of orthogonality.

2.5.3 Norm. Given a Hilbert space $\mathcal{E}$ with basis X and a vector $v \in \mathcal{E}$, we define $\|v\|$ as $\sqrt{\langle v|v \rangle}$. Since this is $\sqrt{\sum_x \overline{v_x} v_x} = \sqrt{\sum_x |v_x|^2}$, this is always a non-negative real number. The following properties hold.

$$\begin{aligned}\|\alpha \cdot v\| &= |\alpha| \cdot \|v\|, \\ \|v + w\| &\leq \|v\| + \|w\|, \\ \|x\| &= 1 \text{ when } x \in X.\end{aligned}$$

2.5.4 Orthogonality. Given a Hilbert space $\mathcal{E}$ with basis X and two vectors $v, w \in \mathcal{E}$, we say that v is *orthogonal* to w , written $v \perp w$, if $\langle v|w \rangle = 0$. By definition of the scalar product, distinct canonical basis elements are pairwise orthogonal.

2.5.5 Orthonormal basis. In general, a *basis* for a vector space is a set of vectors spanning the whole space. The elements of a basis are called *basis elements*. A basis is an *orthonormal basis* if basis elements are each of norm 1, and pairwise orthogonal.

2.5.6 For the Hilbert space $\mathcal{H}$, an example of non-canonical, orthonormal basis is the set $\{|+\rangle, |-\rangle\}$, where

$$\begin{aligned}|+\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \\ |-\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).\end{aligned}$$

2.6 Kets and Bras

2.6.1 In quantum computation, the canonical basis elements are representing classical values: a linear combination can then be seen as a “superposition” of classical values. As we discussed in Sec. 2.4.5, a standard notation for making canonical basis elements out of arbitrary classical values is the *ket-notation*

$$|\rangle.$$

2.6.2 The two-dimensional Hilbert space $\mathcal{H}$ that we shall be considering in these notes is built from the canonical basis set $\{0, 1\}$. A typical vector of $\mathcal{H}$ is then

$$\alpha \cdot |0\rangle + \beta \cdot |1\rangle.$$

The chosen “0” and “1” stands for two classical, distinguished values, such as “False” and “True” respectively. Their name is only a convention: one could have chosen any other pair of names such as $|T\rangle$ and $|F\rangle$, $|\text{foo}\rangle$ and $|\text{bar}\rangle$... They should in particular not be confused with the complex scalars 0 and 1. Similarly, $|0\rangle$ is NOT the null element of the vector space, so

$$|0\rangle \neq 0.$$

2.6.3 You might have noticed the clash of notation between

$$\langle u | v \rangle$$

and for instance

$$\alpha \cdot |0\rangle + \beta \cdot |1\rangle.$$

This is on purpose, and yields a pun: if the notation $|-\rangle$ is called a “ket”, we call $\langle -|$ a “bra”, which yield the standard name “braket” for $\langle -|-\rangle$ (yes, this notation is called “braket” in English), so that

$$\langle x | y \rangle = \langle x | \cdot | y \rangle,$$

the “multiplication”, or the *application* of the function $\langle x |$ to the argument $|y\rangle$. If $x \in X$ is a basis element for $\mathcal{E}$, the notation $\langle x |$ stands for the linear operation

$$\langle x | : y \mapsto \langle x | y \rangle = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{else} \end{cases}.$$

By linearity, we then have $\langle x | u \rangle = \langle x | \cdot |u\rangle$ for a general vector u in $\mathcal{E}$.

2.6.4 Duality bra-ket. for the vector space $\mathcal{E}$ of basis elements $x \in X$, the bras $\langle x |$ are *functionals*: functions $\mathcal{E} \rightarrow \mathbb{C}$. As such, they can be equipped with a sum and a (complex) scalar multiplication. The functional $\alpha \langle x | + \langle y |$ is the map

$$\alpha \langle x | + \langle y | : u \mapsto \alpha \langle x | u \rangle + \langle y | u \rangle,$$

There is a strong duality: if $u = \sum_x \alpha_x |x\rangle$ and $v = \sum_x \beta_x |x\rangle$, then one can check that

$$\langle u | v \rangle = \left(\sum_x \overline{\alpha_x} \cdot \langle x | \right) \left(\sum_x \beta_x \cdot |x\rangle \right) = u^* \cdot v$$

where $u^* = \sum_x \overline{\alpha_x} \cdot \langle x |$ is the *dual*, or *conjugate transpose* of u .

2.6.5 Notation. Vectors of the form $\sum_{x \in X} \alpha_x \cdot |x\rangle$ will be written with greek letters inside kets, such as $|\phi\rangle, |\psi\rangle$, etc. When using lower-case latin letters $|x\rangle, |y\rangle, |b\rangle, |c\rangle$, we shall be referring to canonical basis elements. Functionals of the form $\sum_{x \in X} \alpha_x \cdot \langle x |$ will dually be written as $\langle \phi |, \langle \psi |$, etc. We then have the property that

$$|\phi\rangle^* = \langle \phi | \quad \text{and} \quad \langle \phi |^* = |\phi\rangle.$$

The application of a functional (i.e. a bra) to a vector (i.e. a ket) is always written as a multiplication.

2.6.6 Remark Bras and kets can contain more than letters and numbers: we use for instance

$$|+\rangle \triangleq \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle \triangleq \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

in 2.5.6 and

$$|+i\rangle \triangleq \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle), \quad |-i\rangle \triangleq \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$$

in 3.2.10.

2.6.7 Beware! Do not forget to conjugate the complex coefficient when moving from kets to bras and bras to kets.

2.6.8 Matrix-style representation. Bras, kets, and linear operations in general can be represented in a matrix-style notation. For this to make sense, we need to *order* the canonical basis elements of the vector spaces. For the Hilbert space $\mathcal{H}$, the basis $|0\rangle, |1\rangle$ is ordered in the *lexicographic order*. A ket is in this convention a column-vector as follow

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

so that

$$\alpha \cdot |0\rangle + \beta \cdot |1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}.$$

A bra becomes a row-vector :

$$\langle 0| = (1 \ 0), \quad \langle 1| = (0 \ 1).$$

If $|\phi\rangle = \alpha \cdot |0\rangle + \beta \cdot |1\rangle$ and $|\psi\rangle = \gamma \cdot |0\rangle + \delta \cdot |1\rangle$, we can check that

$$\langle \phi | \psi \rangle = (\bar{\alpha} \ \bar{\beta}) \cdot \begin{pmatrix} \gamma \\ \delta \end{pmatrix} = \bar{\alpha}\gamma + \bar{\beta}\delta.$$

The conjugate transpose of a column vector is a row vector and vis versa, as follows.

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix}^* = (\bar{\alpha} \ \bar{\beta}), \quad (\alpha \ \beta)^* = \begin{pmatrix} \bar{\alpha} \\ \bar{\beta} \end{pmatrix}.$$

2.6.9 With the matrix-style representation of 2.6.8, kets and bras can be combined in arbitrary ways, as long as dimensions match. For instance,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} = a|0\rangle\langle 0| + c|1\rangle\langle 0| + b|0\rangle\langle 1| + d|1\rangle\langle 1|.$$

Multiplication by a ket-vector is then transparent. For instance:

$$\begin{aligned} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} (\alpha|0\rangle + \beta|1\rangle) &= (|0\rangle\langle 1| + |1\rangle\langle 0|)(\alpha|0\rangle + \beta|1\rangle) \\ &= \alpha|0\rangle\langle 1|0\rangle + \alpha|1\rangle\langle 0|0\rangle + \beta|0\rangle\langle 1|1\rangle + \beta|1\rangle\langle 0|1\rangle \\ &= \alpha|0\rangle\langle 1|0\rangle + \alpha|1\rangle\langle 0|0\rangle + \beta|0\rangle\langle 1|1\rangle + \beta|1\rangle\langle 0|1\rangle \\ &= \alpha\langle 1|0\rangle|0\rangle + \alpha\langle 0|0\rangle|1\rangle + \beta\langle 1|1\rangle|0\rangle + \beta\langle 0|1\rangle|1\rangle \\ &= \alpha \cdot 0 \cdot |0\rangle + \alpha \cdot 1 \cdot |1\rangle + \beta \cdot 1 \cdot |0\rangle + \beta \cdot 0 \cdot |1\rangle \\ &= \alpha|1\rangle + \beta|0\rangle \end{aligned}$$

2.7 Kronecker product

2.7.1 Consider two vector spaces $\mathcal{E}$ of canonical basis $B = \{e_i\}_i$ and $\mathcal{F}$ of canonical basis $C = \{f_j\}_j$. We define a new vector space $\mathcal{E} \otimes \mathcal{F}$ using the canonical basis set $B \times C$, the cartesian product of B and C . The space $\mathcal{E} \otimes \mathcal{F}$ is called the *Kronecker product* of $\mathcal{E}$ and $\mathcal{F}$, or *tensor* of $\mathcal{E}$ and $\mathcal{F}$. We could write the pair of e_i and f_j in various manner such as (e_i, f_j) , e_i, f_j , $e_i f_j$ etc. We shall be usig simple concatenation when clear. A typical vector of $\mathcal{E} \otimes \mathcal{F}$ is then of the form

$$\sum_{i,j} \alpha_{i,j} |e_i f_j\rangle.$$

2.7.2 The tensor notation is overloaded to vectors: it is used to represent the *bilinear map* $\mathcal{E} \times \mathcal{F} \rightarrow \mathcal{E} \otimes \mathcal{F}$ defined as

$$\left(\sum_i \alpha_i \cdot |e_i\rangle \right) \otimes \left(\sum_j \beta_j \cdot |f_j\rangle \right) = \sum_{i,j} \alpha_i \beta_j \cdot |e_i f_j\rangle.$$

It is bilinear in the sense that the following properties hold.

$$(\alpha \cdot |\phi\rangle + |\psi\rangle) \otimes w = \alpha \cdot (|\phi\rangle \otimes w) + |\psi\rangle \otimes w, \quad (15)$$

$$w \otimes (\alpha \cdot |\phi\rangle + |\psi\rangle) = \alpha \cdot (w \otimes |\phi\rangle) + w \otimes |\psi\rangle, \quad (16)$$

$$(\alpha \cdot |\phi\rangle) \otimes |\psi\rangle = \alpha \cdot (|\phi\rangle \otimes |\psi\rangle), \quad (17)$$

$$|\phi\rangle \otimes (\alpha \cdot |\psi\rangle) = \alpha \cdot (|\phi\rangle \otimes |\psi\rangle), \quad (18)$$

$$0 \otimes |\phi\rangle = 0. \quad (19)$$

$$|\phi\rangle \otimes 0 = 0. \quad (20)$$

2.7.3 As the vector space $\mathcal{E} \otimes \mathcal{F}$ is “just” a vector space, the definitions of scalar product and norm presented in Sec. 2.5 still hold. In particular, if

$$|\phi\rangle = \sum_{i,j} \alpha_{i,j} \cdot |e_i f_j\rangle, \quad |\psi\rangle = \sum_{i,j} \beta_{i,j} \cdot |e_i f_j\rangle,$$

then

$$\langle \phi | \psi \rangle = \left(\sum_{i,j} \overline{\alpha_{i,j}} \cdot |e_i f_j\rangle \right) \left(\sum_{i,j} \beta_{i,j} \cdot |e_i f_j\rangle \right) = \sum_{i,j} \overline{\alpha_{i,j}} \beta_{i,j},$$

and

$$\| \psi \| = \sqrt{\sum_{i,j} |\alpha_{i,j}|^2}.$$

If instead $|\phi\rangle = \sum_i \alpha_i \cdot |e_i\rangle$ and $|\psi\rangle = \sum_j \beta_j \cdot |f_j\rangle$, one can then derive that

$$\| |\phi\rangle \otimes |\psi\rangle \| = \| |\phi\rangle \| \cdot \| |\psi\rangle \|,$$

and if moreover $|\phi'\rangle = \sum_i \alpha'_i \cdot |e_i\rangle$ and $|\psi'\rangle = \sum_j \beta'_j \cdot |f_j\rangle$,

$$(\langle \phi | \otimes \langle \psi |)(|\phi'\rangle \otimes |\psi'\rangle) = \langle \phi | \phi' \rangle \langle \psi | \psi' \rangle.$$

2.7.4 Notation The notation of Sec. 2.7.1 is overloaded for arbitrary kets. For instance, remember that $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: we write $|+0\rangle$ for the vector

$$\begin{aligned} | + 0 \rangle &= | + \rangle \otimes | 0 \rangle \\ &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle \\ &= \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle). \end{aligned}$$

In general, $|\phi\psi\rangle$ is $|\phi\rangle \otimes |\psi\rangle$.

2.8 Linear Maps

2.8.1 Given two vector spaces $\mathcal{E}$ and $\mathcal{F}$ with respective bases $\{e_i\}_i$ and $\{f_j\}_j$, a *linear map* (or *linear operator*) f from $\mathcal{E}$ to $\mathcal{F}$ is a set-function from $\mathcal{E}$ to $\mathcal{F}$ such that

$$f(u + v) = f(u) + f(v), \quad f(\alpha \cdot u) = \alpha \cdot f(u).$$

2.8.2 A linear function from $\mathcal{E}$ to $\mathcal{F}$ is uniquely characterized by its action on the basis elements. Using the ket-notation :

$$f\left(\sum_x \alpha_x \cdot |x\rangle\right) = \sum_x \alpha_x \cdot f(|x\rangle).$$

2.8.3 Provided that the canonical basis is ordered, one can use matrices to represent linear maps. In dimension 2, a matrix has the generic form

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}.$$

In the *coefficient* a_{ij} , the index i stands for the line number while j stands for the column number. Provided that we use the lexicographic ordering on canonical basis states as in Sec. 2.6.8, this matrix represents the linear map $\mathcal{H} \rightarrow \mathcal{H}$ defined by

$$\begin{aligned} |0\rangle &\mapsto a_{11} \cdot |0\rangle + a_{21} \cdot |1\rangle, \\ |1\rangle &\mapsto a_{12} \cdot |0\rangle + a_{22} \cdot |1\rangle. \end{aligned}$$

2.8.4 If A and B are two $n \times n$ matrices and if a_{ij} and b_{ij} are their respective coefficients on line i and column j , then the coefficient at position (i, j) of the matrix $A \cdot B$ is $\sum_k a_{ik} b_{kj}$. For instance, in dimension 2:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & \dots \\ \dots & \dots \end{pmatrix}.$$

2.8.5 The action of a linear operator on a vector becomes the matrix multiplication with the corresponding vector. Indeed, a column vector is "just" a matrix with only one column, so

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} a_{11}v_1 + a_{12}v_2 \\ a_{21}v_1 + a_{22}v_2 \end{pmatrix}$$

If A is the matrix of a linear operation f and B the matrix of g , then

$$f(g(v)) = A \cdot (B \cdot v) = (A \cdot B) \cdot v,$$

so $f \circ g$ corresponds to $A \cdot B$.

2.8.6 For instance, the linear map $\text{HAD} : \mathcal{H} \rightarrow \mathcal{H}$ is defined using the kets introduced in Sec. 2.5.6 as follows:

$$\text{HAD} : \begin{cases} |0\rangle \mapsto |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \\ |1\rangle \mapsto |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle). \end{cases}$$

It can be represented with the matrix

$$\frac{1}{\sqrt{2}} \begin{pmatrix} \boxed{1} & \boxed{1} \\ \boxed{1} & \boxed{-1} \end{pmatrix} \begin{matrix} |+\rangle \\ |-\rangle \\ |0\rangle \\ |1\rangle \end{matrix} \quad (21)$$

The first column is $|+\rangle$, the output of the function at $|0\rangle$, and the second column is $|-\rangle$, the output of the function at $|1\rangle$.

2.8.7 The operation HAD is called *Hadamard*. It can be written in a more compact way as

$$|x\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + (-1)^x |1\rangle),$$

with the convention of Sec. 2.6.5 that x is one of the basis-set element “0” or “1”. the exponent $(-1)^x$ then assimilates x to its “integer” value, and $(-1)^0 = 1$ while $(-1)^1 = -1$.

2.8.8 The application of the function to a vector corresponds to matrix-vector multiplication. for instance, $\text{HAD} |0\rangle$ is

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

2.8.9 Similarly, the composition of functions corresponds to matrix multiplication. One can for instance check that $\text{HAD} \circ \text{HAD}$ is the identity. Using the definition from Sec. 2.8.7:

$$\begin{aligned} \text{HAD}(\text{HAD} |x\rangle) &= \text{HAD}\left(\frac{1}{\sqrt{2}}(|0\rangle + (-1)^x |1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(\text{HAD} |0\rangle + (-1)^x \text{HAD} |1\rangle) \\ &= \frac{1}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + (-1)^x \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) \\ &= \frac{1}{2}(|0\rangle + |1\rangle + (-1)^x |0\rangle - (-1)^x |1\rangle) \\ &= \frac{1}{2}((1 + (-1)^x) |0\rangle + (1 - (-1)^x) |1\rangle) \\ &= \begin{cases} \frac{1}{2}(2 \cdot |0\rangle + 0 \cdot |1\rangle) & \text{if } x = 0, \\ \frac{1}{2}(0 \cdot |0\rangle + 2 \cdot |1\rangle) & \text{if } x = 1 \end{cases} \end{aligned}$$

$$= |x\rangle.$$

One can then check that

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

2.8.10 As for column and row vectors, we can define a notion of conjugate transpose for matrices:

$$\begin{pmatrix} \alpha_{1,1} & \cdots & \alpha_{1,n} \\ \vdots & \ddots & \vdots \\ \alpha_{m,1} & \cdots & \alpha_{m,n} \end{pmatrix}^* \triangleq \begin{pmatrix} \overline{\alpha_{1,1}} & \cdots & \overline{\alpha_{m,1}} \\ \vdots & \ddots & \vdots \\ \overline{\alpha_{1,n}} & \cdots & \overline{\alpha_{m,n}} \end{pmatrix}$$

2.8.11 We shall be using I and Id for the identity map on vector spaces. This is obviously a linear map.

2.8.12 The Kronecker product is also overloaded to functions, as follows. Given two linear maps $U: \mathcal{E} \rightarrow \mathcal{E}$ and $V: \mathcal{F} \rightarrow \mathcal{F}$, we define the linear map

$$U \otimes V: \mathcal{E} \otimes \mathcal{F} \rightarrow \mathcal{E} \otimes \mathcal{F}$$

by

$$(U \otimes V) |xy\rangle \triangleq (U |x\rangle) \otimes (V |y\rangle).$$

Because of the linearity of the operation, function application and composition permutes with the tensor as follows. Let $|\phi\rangle \in \mathcal{E}$, $|\psi\rangle \in \mathcal{F}$, $U: \mathcal{E} \rightarrow \mathcal{E}$ and $V: \mathcal{F} \rightarrow \mathcal{F}$, then

$$(U \otimes V)(|\phi\rangle \otimes |\psi\rangle) = (U |\phi\rangle) \otimes (V |\psi\rangle), \quad (22)$$

$$(U \otimes V) \circ (U' \otimes V') = (U \circ U') \otimes (V \circ V'). \quad (23)$$

2.8.13 The tensor symbol can be regarded as forming an impermeable, spacial separation between the left side and the right side.

“What’s on the left stays on the left;
what’s on the right stays on the right.”

2.8.14 Note how the symbol “ $\otimes$ ” has been overloaded 3 times: for vector spaces (as in “ $\mathcal{E} \otimes \mathcal{F}$ ”), for vectors (as in “ $|\phi\rangle \otimes |\psi\rangle$ ”), and for linear operations (as in “ $U \otimes V$ ”).

2.9 Hermitian and Unitary Maps

2.9.1 A linear map $U: \mathcal{E} \rightarrow \mathcal{E}$ is called *unitary* if

- it is invertible : there exists a linear map $\mathcal{E} \rightarrow \mathcal{E}$ written U^{-1} such that $U^{-1} \circ U$ and $U \circ U^{-1}$ are the identity,
- the inverse of U is its conjugate transpose: $U^{-1} = U^*$.

Unitary maps preserves norm and orthogonality: they can be regarded as rotations. For instance, the map Hadamard from Sec. 2.8.6 is unitary. It sends the canonical basis $\{|0\rangle, |1\rangle\}$ to the basis $\{|+\rangle, |-\rangle\}$.

2.9.2 If U is a unitary map, then

$$\langle \phi | \psi \rangle = \langle U\phi | U\psi \rangle.$$

2.9.3 If $U|\psi\rangle = \lambda|\psi\rangle$, we say that $|\psi\rangle$ is an *eigenvector* of U and λ an *eigenvalue*. In the case where U is unitary, λ is of the form $e^{2i\pi\omega}$ with $0 \leq \omega < 1$ a real number, because U preserves the norm, and thus $|\lambda| = 1$. By abuse of language, we say that ω is the *phase* of the eigenvalue.

2.9.4 A linear map $H: \mathcal{E} \rightarrow \mathcal{E}$ is called *Hermitian* when $H = H^*$. Equivalently:

- If $(h_{i,j})_{i,j}$ is a matrix representation of H in an orthonormal basis, then $h_{i,j} = \overline{h_{j,i}}$.
- H admits a set of eigenvectors $\{|u_j\rangle\}_j$ forming an orthonormal basis, and all of its eigenvalues are real numbers.

With the convention in 2.6.9, the Hermitian operator can be written as

$$H = \sum_j \lambda_j |u_j\rangle \langle u_j|, \quad (24)$$

when λ_j is the eigenvalue corresponding to $|u_j\rangle$. One can check that

$$\begin{aligned} H|u_{j_0}\rangle &= \sum_j \lambda_j |u_j\rangle \langle u_j|u_{j_0}\rangle \\ &= \lambda_{j_0} |u_{j_0}\rangle \langle u_{j_0}|u_{j_0}\rangle + \sum_{j \neq j_0} \lambda_j |u_j\rangle \langle u_j|u_{j_0}\rangle \\ &= \lambda_{j_0} \langle u_{j_0}|u_{j_0}\rangle |u_{j_0}\rangle + \sum_{j \neq j_0} \lambda_j \langle u_j|u_{j_0}\rangle |u_j\rangle \\ &= \lambda_{j_0} \cdot 1 \cdot |u_{j_0}\rangle + \sum_{j \neq j_0} \lambda_j \cdot 0 \cdot |u_j\rangle \\ &= \lambda_{j_0} |u_{j_0}\rangle \end{aligned}$$

since the $|u_j\rangle$ s were picked pairwise distinct.

2.9.5 The eigenvalues of a Hermitian matrix are all real numbers, and there is a finite number of them. One of them is then *minimal*, the other *maximal*. In general, we talk about *extremal* eigenvalues, and by abuse of notation we speak of minimal / maximal / extremal eigenvectors to refer to the corresponding eigenvectors. Note that (say) minimal eigenvector are not unique. For instance, the identity has many of them.

2.9.6 In the context of quantum computation, a Hermitian map is often called a *Hamiltonian*. Hamiltonian operators are used to model physical properties of a system such as its energy. For the purpose of this course, we can safely stay at this (low) level of understanding.

2.9.7 The set of Hermitian maps forms a real vector field: it is closed under addition and (real) scalar multiplication.

2.9.8 Unitary and Hermitian operators are related through the exponential formula in Eq (1): If H is Hermitian, then

$$e^{iH} = \sum_{k=0}^{\infty} \frac{1}{k!} H^k$$

is unitary. Conversely, every unitary U can be written as e^{iH} for a Hermitian operator H .

2.9.9 Eigenvectors are preserved through exponentiation. Suppose that $|\phi\rangle$ is an eigenvector of the Hermitian H with eigenvalue λ , so $H|\phi\rangle = \lambda|\phi\rangle$. Then

$$\begin{aligned} (e^{iH})|\phi\rangle &= \left(\sum_{k=0}^{\infty} \frac{1}{k!} H^k \right) |\phi\rangle \\ &= \sum_{k=0}^{\infty} \frac{1}{k!} H^k |\phi\rangle \\ &= \sum_{k=0}^{\infty} \frac{1}{k!} \lambda^k |\phi\rangle \\ &= \left(\sum_{k=0}^{\infty} \frac{1}{k!} \lambda^k \right) |\phi\rangle \\ &= e^{i\lambda} |\phi\rangle. \end{aligned}$$

The eigenvalue of e^{iH} corresponding to $|\phi\rangle$ is $e^{i\lambda}$.

2.9.10 If A and B are Hermitian operators (or, in general, square matrices) such that $AB = BA$, then

$$e^{A+B} = e^A e^B.$$

2.9.11 If A is an Hermitian operators (or, in general, any square matrix) and if U is a unitary, then

$$e^{U \cdot A \cdot U^*} = U \cdot (e^A) \cdot U^*.$$

2.9.12 Consider the linear maps (called *Pauli maps*)

$$X \triangleq \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z \triangleq \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y \triangleq \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

in the ordered basis $\{|0\rangle, |1\rangle\}$. These maps are both Hermitian and unitary. Squaring them yields the identity.

2.9.13 A known result is that any Hermitian matrix of size $2^n \times 2^n$ can be written as a linear combination of tensors of Pauli matrices of 2.9.12 (and identity) with real coefficients. More precisely, any Hermitian matrix of dimension $2^n \times 2^n$ can be written as

$$\sum_{G_1, \dots, G_n \in \{X, Y, Z, Id\}} h_{G_1, \dots, G_n} \cdot G_1 \otimes \dots \otimes G_n$$

with $h_{G_1, \dots, G_n} \in \mathbb{R}$.

2.10 Exercises

2.10.1 Consider the vector $|\phi\rangle = \frac{i}{3}|0\rangle + \frac{2\sqrt{2}}{3}|1\rangle$ in $\mathcal{H}$.

1. Show that $|\phi\rangle$ is of norm 1.
2. Compute $\langle\phi|$.
3. Compute $\langle\phi|+\rangle$ and $\langle-\phi|$.
4. The vector $|\phi\rangle$ has been given in the basis $\{|0\rangle, |1\rangle\}$. Write it in the basis $\{|+\rangle, |-\rangle\}$.
5. Give an vector of norm 1 orthogonal to $|\phi\rangle$

2.10.2 Assume that x is either 0 or 1. Show that

$$\frac{1}{\sqrt{2}}(|x\rangle - |1 \oplus x\rangle) = (-1)^x \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

The symbol “ $\oplus$ ” is the XOR bit operation.

2.10.3 Consider the set $\mathcal{B} \triangleq \{e_0, e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$ and a bijection σ on the set $\mathcal{B}$. Let $\mathcal{E}$ be the Hilbert space generated by the set $\mathcal{B}$, and U the linear map $\mathcal{E} \rightarrow \mathcal{E}$ defined as

$$U : |e_k\rangle \mapsto |\sigma(e_k)\rangle \quad \text{for } k = 0, \dots, 7$$

Show that this map is unitary.

2.10.4 Consider the setting of Exo 2.10.3. Let σ be the map sending e_k to $e_{3 \cdot k \bmod 8}$, that is, the index k is mapped as followed:

$$\begin{array}{cccc} 0 \mapsto 0 & 2 \mapsto 6 & 4 \mapsto 4 & 6 \mapsto 2 \\ 1 \mapsto 3 & 3 \mapsto 1 & 5 \mapsto 7 & 7 \mapsto 5 \end{array}$$

Write the matrix U corresponding to the unitary based on σ with the basis ordering

$$e_0, e_1, e_2, e_3, e_4, e_5, e_6, e_7.$$

What does it means when $U_{m,n} = 1$?

2.10.5 Consider the ket vectors of $\mathcal{H} \otimes \mathcal{H}$

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

1. Show that these 4 vectors form an orthonormal basis
2. Consider the ket vector

$$|\phi\rangle = \frac{2}{\sqrt{5}} |01\rangle + \frac{i}{\sqrt{5}} |10\rangle$$

- (a) Show that this is a vector of norm 1.
- (b) Write it as a linear combination of $|\Phi^+\rangle$, $|\Phi^-\rangle$, $|\Psi^+\rangle$ and $|\Psi^-\rangle$.
- (c) Compute $\langle \phi | \Psi^+ \rangle$.

2.10.6 Show that for every vector $|\phi\rangle$ in $\mathcal{H}$, we have

$$|\phi\rangle = \langle 0|\phi\rangle \cdot |0\rangle + \langle 1|\phi\rangle \cdot |1\rangle$$

2.10.7 Consider the vectors $|\phi\rangle = \frac{i}{3} |0\rangle + \frac{2\sqrt{2}}{3} |1\rangle$ and $|\psi\rangle = \frac{1}{\sqrt{5}} |0\rangle + \frac{2}{\sqrt{5}} |1\rangle$ in $\mathcal{H}$.

1. Compute $|\phi\rangle \otimes |\psi\rangle$ in the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$.
2. Compute $|\phi\rangle \otimes |\psi\rangle$ in the basis $\{|++\rangle, |+-\rangle, |-+\rangle, |--\rangle\}$.
3. Give an orthonormal basis of $\mathcal{H} \otimes \mathcal{H}$ for which $|\phi\rangle \otimes |\psi\rangle$ is one of the elements.

2.10.8 Consider the operation $\odot$ acting on two bitstrings:

$$(x_1 \dots x_n) \odot (y_1 \dots y_n) = (x_1 \wedge y_1) \oplus \dots \oplus (x_n \wedge y_n).$$

Show that $\text{HAD}^{\otimes n}$ performs the action

$$|x_1 \dots x_n\rangle \mapsto \frac{1}{\sqrt{2}^n} \sum_{y_1 \dots y_n} (-1)^{(x_1 \dots x_n) \odot (y_1 \dots y_n)} |y_1 \dots y_n\rangle.$$

2.10.9 For each of the Pauli matrix $G = X, Y, Z$ of 2.9.12:

1. Show that G is both unitary and hermitian
2. Show that $G^2 = Id$

2.10.10 Consider the decomposition of Hermitian matrices presented in 2.9.13. This exercise focuses on the 4×4 case.

1. Compute the 16 following 4×4 matrices

$$\begin{array}{cccccccc} X \otimes X, & X \otimes Y, & X \otimes Z, & X \otimes I, & Y \otimes X, & Y \otimes Y, & Y \otimes Z, & Y \otimes I, \\ Z \otimes X, & Z \otimes Y, & Z \otimes Z, & Z \otimes I, & I \otimes X, & I \otimes Y, & I \otimes Z, & I \otimes I. \end{array}$$

2. Consider the following Hermitian matrix:

$$\begin{pmatrix} 2 & 0 & 4-i & 0 \\ 0 & 5 & 3 & 0 \\ 4+i & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Write it as a linear decomposition of the 16 matrices in the first question, with real coefficients.

2.10.11 Consider the following matrix.

$$M = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Questions:

1. Write M as a linear combination of Pauli matrices (and identity)
2. Find the eigenvectors and eigenvalues of M
3. Write M in the form of Eq.(24) in 2.9.4.

2.10.12 Consider the matrices X, Y and Z of 2.9.12. Fill in the multiplication table: each cell should be of the form $\pm iG$ or $\pm G$ (with $G \in \{X, Y, Z, Id\}$).

$\times$	I	X	Y	Z
I				
X				
Y				
Z				

2.10.13 Compute the following:

$$\text{HAD} \cdot X \cdot \text{HAD}, \quad \text{HAD} \cdot Z \cdot \text{HAD}, \quad \text{HAD} \cdot Y \cdot \text{HAD}.$$

2.10.14 Consider the following two linear maps:

$$\text{SWAP} : |00\rangle \mapsto |00\rangle \quad |01\rangle \mapsto |10\rangle \quad |11\rangle \mapsto |11\rangle \quad |10\rangle \mapsto |01\rangle$$

$$\text{CNOT} : |00\rangle \mapsto |00\rangle \quad |10\rangle \mapsto |11\rangle \quad |01\rangle \mapsto |01\rangle \quad |11\rangle \mapsto |10\rangle$$

For each of the following linear maps acting on $\mathcal{H} \otimes \mathcal{H}$, give all its possible representative matrices depending on the orderings of the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ (and for each matrix, give the corresponding ordering(s)).

3 Qubit-based Computation

3.1 The Quantum Co-Processor Model

3.1.1 In the standard interpretation, a quantum computer is a standard, conventional computer together with a particular kind of coprocessor. Other, conventional coprocessor includes: GPUs, FGPAs, TLS/SSL accelerators, *etc.* Inside the computer, the processor is the part that actually *runs* a program; a co-processor is governed by the processor for specific tasks (for instance, a GPU is used for fast matrix operations). A co-processor typically has its own memory, and the operations available to the co-processor will be performed on this local memory.

3.1.2 For our purpose, a typical use of a co-processor is then:

1. Allocate and initialize part of the memory local to the co-processor;
2. Perform some action;
3. Read the memory and free (de-allocate) some part of the memory.

These 3 actions are the primary requirements for a co-processor: we need to be able to set up the memory, read it, and do something on it (ideally non-trivial).

3.1.3 The part of the memory that is being manipulated is modeled in term of *register*: elementary units of data stored on the physical medium and that can be individually addressed. A classical register can for instance hold a single bit, or a byte (i.e. an 8-bit sequence), or 8 bytes (i.e. 64 bits), *etc.* By extension, an array of registers can itself be seen as a register. For instance, an array of 64-bit integers can be described as a register. In the storage of a bit, there are therefore three distinct levels of abstraction:

- Mathematical level: The Boolean algebra $\mathbb{B} = \{ 0, 1 \}$;
- Computer science level: A register holding a bit of information, with a unique identifier to programmatically address it (a *pointer* or a *reference*);
- Physics level: A bunch of transistors wired together, realizing a hardware implementation of the register.

If the word *register* can be used to refer to any of these levels, it is important to keep the distinction in mind, in particular the fact that “register” refer to both a *mathematical value* and a *location*.

3.1.4 Our *quantum* co-processor is designed with the philosophy presented in [3.1.1](#) and [3.1.2](#): it holds a *quantum memory* that can be initialized and read, while allowing one to perform a particular set of (usually) *local* operations. The memory consists of *quantum registers*, and for the purpose of this set of notes, these holds the so-called *quantum bit*, described in Section [3.2](#).

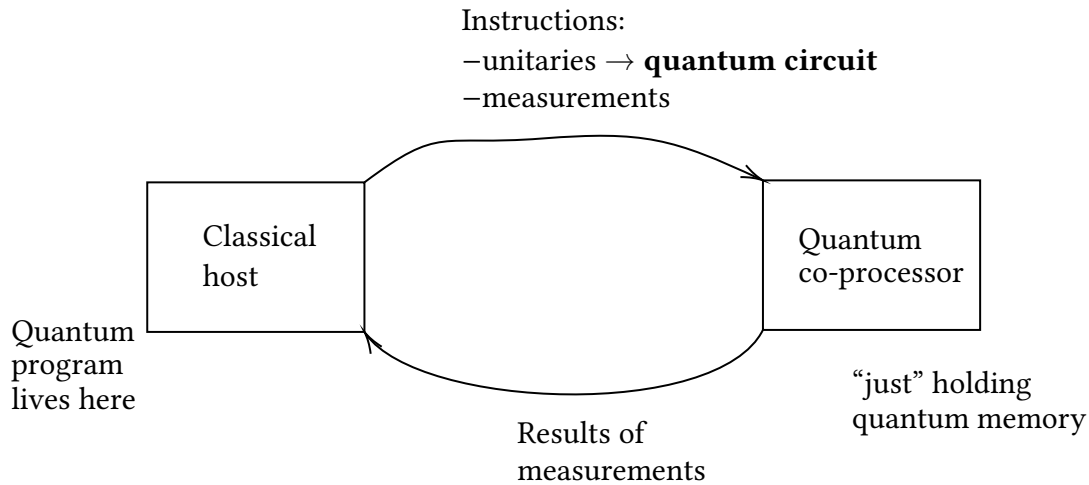


Figure 1: The coprocessor model.

3.2 One Quantum Bit

3.2.1 A *quantum bit*, or *qubit*, corresponds to a piece of quantum memory containing an elementary unit of quantum information.

3.2.2 The easiest way to understand what is going on is to refer to what happens in the classical setting. In the latter, to encode a bit of information, one chooses a medium, and two states. These states have to be *unequivocally distinguished*, and they should be easy to act upon (to encode computation). One such state then represents “True”, the other “False”. Examples include:

- Coin, head/tail;
- Coin, steel or copper;
- Magnet, north or south facing up;
- Piece of paper, black or white color.

The choice is completely arbitrary and is purely conventional: The knowledge of which states were chosen is mandatory to recover the information.

3.2.3 One follows the same strategy for quantum information. A quantum bit, the smallest piece of quantum information, is encoded on an object governed by the law of quantum mechanics, together with a choice of two states for this object. As for the classical case, one should be able to

- distinguish the two states
- act upon the states: initialize, read, and otherwise modify the state.

Examples include:

- Photon, vertical and horizontal polarization;

- Photon, position (wire A or wire B);
- Electron, spin up or spin down;
- Electron, orbital position;
- Atom, energy level.

3.2.4 Hilbert spaces form the framework for the mathematical formalism to represent the state of a quantum system. For our purpose, such a state is a *normalized vector* in a Hilbert space (up to global phase, see 3.2.5). A quantum bit is represented within the two-dimensional Hilbert space $\mathcal{H}$: the chosen two states are represented with $|0\rangle$ and $|1\rangle$ (called *basis state*). In this framework, distinguishability corresponds to *orthogonality*, while the actions are modeled with unitary maps.

3.2.5 Global phase. Let us formalize the mathematical representation of a quantum bit. Within the state $\mathcal{H}$, define the set of kets

$$S_1 = \{ \alpha |0\rangle + \beta |1\rangle \in \mathcal{H} \mid |\alpha|^2 + |\beta|^2 = 1 \}.$$

We say that two elements $|\phi\rangle, |\psi\rangle \in S_1$ are *related by a global phase* if there exists an angle θ such that $|\psi\rangle = e^{i\theta} |\phi\rangle$. In this case, we write $|\psi\rangle \equiv_{\text{phase}} |\phi\rangle$.

States of quantum bits are *equivalence classes under $\equiv_{\text{phase}}$* . In other words, the state of a quantum bit (qubit) Q is a subset of S_1 such that

- Elements of Q are pairwise related by a global phase: $|\phi\rangle, |\psi\rangle \in Q$ implies that $\exists \theta$ such that $|\psi\rangle = e^{i\theta} |\phi\rangle$;
- Q is maximal: $|\phi\rangle \in Q$ implies that for every angle θ , we have $e^{i\theta} |\phi\rangle \in Q$.

3.2.6 An element $|\psi\rangle \in Q$ is called a *representative element* of Q . By abuse of notation, when talking about a qubit, the equivalence class Q is identified with its representative elements: we say that $|\psi\rangle \in Q$ is the state of a qubit instead of referring to Q .

3.2.7 Quantum register Coming back to the discussion of 3.1.3, we can spell out the 3 abstraction levels for a quantum register holding a qubit:

- At the mathematical level: a normalized vector of $\mathcal{H}$, modulo a global phase. At this level, we only have a mathematical object, there is no information.
- At the computer science level: a way to programmatically refer to it (by some pointer for instance), and the fact that $|0\rangle$ and $|1\rangle$ corresponds to the basis of information.
- At the physics level: the choice of an object and a pair of orthogonal states: photon and polarization, electron and spin, *etc.*

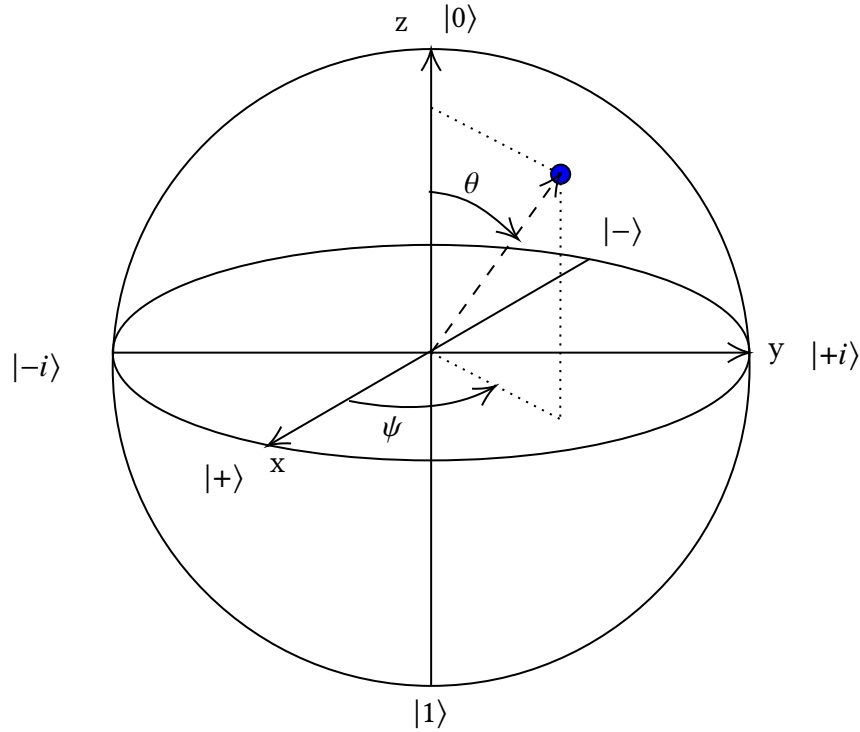


Figure 2: Bloch sphere

3.2.8 Canonical representative. Consider a qubit $\alpha |0\rangle + \beta |1\rangle$. The two complex numbers α and β can be written as $\rho_a e^{i\phi_a}$ and $\rho_b e^{i\phi_b}$ with ρ_a and ρ_b non-negative and such that $\rho_a^2 + \rho_b^2 = 1$. So there exists an angle $\theta \in [0, \pi]$ such that $\rho_a = \cos(\frac{\theta}{2})$ and $\rho_b = \sin(\frac{\theta}{2})$. Another representative element of the same qubit is $e^{-i\phi_a} (\alpha |0\rangle + \beta |1\rangle)$ which is then $\cos(\frac{\theta}{2}) |0\rangle + \sin(\frac{\theta}{2}) e^{i(\phi_b - \phi_a)} |1\rangle$. We call this representative element the *canonical representative element*.

3.2.9 Bloch sphere. Without loss of generality, a qubit can therefore be parameterized by two angles $\theta \in [0, \pi]$ and $\psi \in [0, 2\pi)$ as follows:

$$\cos\left(\frac{\theta}{2}\right) \cdot |0\rangle + \sin\left(\frac{\theta}{2}\right) e^{i\psi} \cdot |1\rangle.$$

The angle ψ is called the *phase* of the qubit. This gives a 3-D representation of a qubit on the so-called *Bloch sphere*, shown in Figure 2. Apart from $|0\rangle$ and $|1\rangle$, the canonical representative element is uniquely described by such a pair of angles. (θ, ψ) .

3.2.10 On the Bloch sphere, two antipodal points correspond to two orthogonal kets, i.e. to an orthonormal basis. The three axes x , y and z respectively corresponds to the bases $\{|-\rangle, |+\rangle\}$, $\{|-i\rangle, |+i\rangle\}$ and $\{|1\rangle, |0\rangle\}$:

$$\begin{aligned} |+\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), & |-\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \\ |+i\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle + i \cdot |1\rangle), & |-i\rangle &\triangleq \frac{1}{\sqrt{2}}(|0\rangle - i \cdot |1\rangle). \end{aligned}$$

We find once again one of the orthonormal basis introduced in 2.5.6.

3.3 Several Quantum Bits

3.3.1 A quantum memory usually contains more than one quantum bit: each quantum bit is encoded as a physical object, while its mathematical description relies on $\mathcal{H}$. The quantum memory therefore consists of several such objects. The mathematical model of the *joint quantum system* consisting of all of these objects has a state described by the *Kronecker product* of the individual state spaces: If A and B are two quantum systems whose state spaces are respectively $\vec{E}$ and $\vec{F}$, the state space of the *joint system* AB is $\mathcal{E} \otimes \mathcal{F}$, defined as in 2.7.1.

3.3.2 Let us recall what happens in the classical case. When considering 2 registers, each holding a bit of information, the *state* of the joint system consisting of the 2 registers is the *product* of the individual state spaces: $\mathbb{B} \times \mathbb{B}$. If the state of the system is (0, 1), we can for instance say that the second register is in state 1. The joint system is always *separable*.

3.3.3 Consider now a 2-qubit registers. This system has a state belonging to the tensor space $\mathcal{H} \otimes \mathcal{H}$. As discussed in 2.7.1, it is defined with the canonical basis elements

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle.$$

The notation is scalable: the canonical basis for $\mathcal{H} \otimes \mathcal{H} \otimes \mathcal{H}$ is

$$|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle.$$

3.3.4 This notation is versatile. For instance, the following set of ket-vectors defines a basis for $\mathcal{H} \otimes \mathcal{H} \otimes \mathcal{H}$, called the *SHIFT basis*²:

$$|000\rangle, |111\rangle, | +01\rangle, | -01\rangle, |1+0\rangle, |1-0\rangle, |01+\rangle, |01-\rangle,$$

where for example $|01+\rangle$ stands for $|0\rangle \otimes |1\rangle \otimes |+\rangle$ as discussed in 2.7.4. The kets $|+\rangle$ and $|-\rangle$ were defined in 2.5.6.

3.3.5 The canonical basis of $\mathcal{H}^{\otimes n}$ (i.e. the tensoring of $\mathcal{H}$ n times) is the set

$$\{ |b_1 \dots b_n\rangle \mid b_i \in \{0, 1\} \},$$

the set of all bitstrings of size n . Note how the space $\mathcal{H}^{\otimes n}$ is of dimension 2^n . These bitstrings can be seen as the binary representation of numbers between 0 and $2^n - 1$: when the dimension of the space is clear we will use

$$\sum_{i=0}^{2^n-1} \alpha_i \cdot |i\rangle$$

with $|i\rangle$ understood as the binary representation of i on the correct bitstring size. By convention, in this course the least significant bit is stored on the right: $|011\rangle$ corresponds to $|3\rangle$. Note however that this is completely arbitrary: one could have done the opposite (and QISKIT indeed chooses the other—see 5.3.2 for a longer discussion).

²<https://arxiv.org/abs/2202.00440>

3.3.6 Entanglement. In general, a vector of $\mathcal{H} \otimes \mathcal{H}$ is of the form

$$\alpha \cdot |00\rangle + \beta \cdot |01\rangle + \gamma \cdot |10\rangle + \delta \cdot |11\rangle.$$

One can wonder whether all such vector can be written as $|\phi\rangle \otimes |\psi\rangle$, for $|\phi\rangle, |\psi\rangle \in \mathcal{H}$, as in the classical case discussed in 3.3.2: the answer is no.

- If it is possible, the vector is called *separable*;
- If it is not, the vector is called *entangled*.

An example of entangled state is

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

It is called a *Bell state*, or an *EPR pair*³.

3.3.7 Bell basis. This state can be extended to a basis of entangled elements: the *Bell basis*. It is defined as

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

3.3.8 Following the convention of 2.6.8 we keep the lexicographic order for writing the basis, and this allows us to write kets as column vectors. Given two kets in $\mathcal{H}$ defined as $|\phi_1\rangle = \begin{pmatrix} \alpha_1 \\ \beta_1 \end{pmatrix}$ and $|\phi_2\rangle = \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix}$, the tensor of $|\phi_1\rangle$ with $|\phi_2\rangle$ is the vector

$$\begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix} \quad \begin{array}{l} \leftarrow |00\rangle \\ \leftarrow |01\rangle \\ \leftarrow |10\rangle \\ \leftarrow |11\rangle \end{array}$$

It can be computed in a block-matrix manner as follows:

$$|\phi_1\rangle \otimes |\phi_2\rangle = \begin{pmatrix} \alpha_1 |\phi_2\rangle \\ \beta_1 |\phi_2\rangle \end{pmatrix} = \begin{pmatrix} \alpha_1 \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix} \\ \beta_1 \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix}$$

³Named after Einstein, Podolsky and Rosen.

3.4 The Quantum Circuit Model

3.4.1 In 3.1.3 and 3.2.7, we discussed how registers should be individually addressable, and how one should be able to act on them. In the case of the quantum co-processor, all of this is constrained by what can be realized at the physical level. In general, the physics of the encoding objects allows one to perform two kinds of operations: *unitaries* on the state space: the *internal operations* discussed in 3.1.2, and *measurements*—the reading of the memory. We first focus on the internal operations: the unitaries.

3.4.2 By unitary operation, we literally mean the linear operations that have been described in 2.9.1: Internal actions of the co-processor on the quantum memory are restricted to unitary operations on the state space. Assume that the memory holds n qubits and that U is a unitary on $\mathcal{H}^{\otimes n}$. If the state of the memory is $|\phi\rangle$, applying the “action” of U on the memory has the net effect of deterministically changing the state of the memory to $U|\phi\rangle$. There are no side-effects in the sense that no classical information is leaked to the classical computer (beside the information that the action has been performed), and in the sense that the state of the memory after the action is uniquely determined by U and by its state before the action.

3.4.3 Of course, in general the co-processor does not have access to arbitrary operations on all of the memory at once. As for a classical machine where only a finite set of instructions is available, the quantum co-processor is limited to a finite set of unitary operations, typically acting on one or two qubits at a time. Such operations are referred to as *quantum gates*: elementary unitary gates that are considered not too costly in general. The notion is flexible, and each algorithm might consider a slightly different set of gates. One of the problem is to conciliate what it can do with what we want to do. 3.5 and 3.6 present standard gate-sets; we focus here on the technique to combine gates together.

3.4.4 Following the discussion of 3.4.2, the sequential action of the unitaries U followed by V then corresponds to the action $V \circ U$, the *composition* of V and U . In this scheme, a timed sequence of actions has the mathematical correspondance of successive composition of operators. With the caveat that “ U then V then W ” corresponds to $W \circ V \circ U$ (note the reversal in the order).

3.4.5 The action consisting of not doing anything (such as the instruction no-op in assembly) is still an action: it corresponds to the *identity* map: if the state of the system is $|\phi\rangle$, it is still $|\phi\rangle$ after doing nothing. This representation plays well with the composition of action presented in 3.4.4: doing nothing followed with U is literally the same thing as doing U , and this is acknowledged with the fact that $U \circ Id = U$.

3.4.6 Consider two quantum systems A and B respectively described by the state spaces $\mathcal{E}$ and $\mathcal{F}$. We discussed in 3.3.1 that the joint system is described by $\mathcal{E} \otimes \mathcal{F}$. Suppose now that we perform a unitary U on A and V on B. The action on the $\mathcal{E} \otimes \mathcal{F}$ is the unitary map $U \otimes V$ defined in 2.8.12. If we only perform U on A (while not touching B), overall action on the global system AB is $A \otimes Id$: the sub-system B is acted upon with a trivial action, whose semantics is described in 3.4.5. Thanks to the properties of the

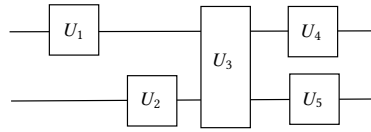


Figure 3: An example of quantum circuit

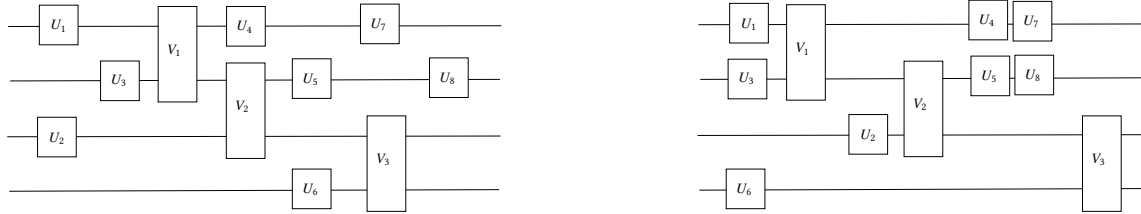


Figure 4: Two equivalent circuits

tensor product, performing U on A followed by V on B is equivalent to performing first V on B then U on A . Indeed, the former is the operation $(Id \otimes B) \circ (A \otimes Id)$ while the latter is $(A \otimes Id) \circ (Id \otimes B)$. Thanks to 2.8.13, we know that both actions are equal to $A \otimes B$.

3.4.7 A quantum memory consists of separately addressable quantum bits: each quantum bit is a distinct subsystem, of state-space $\mathcal{H}$. The global state-space of the memory is the tensor product of all of these state-spaces. We then have two notion of sequentiality:

- timed sequentiality: as discussed in 3.4.4, this is represented with *composition* of operations.
- spacial separation: as discussed in 3.4.6, this is represented with *tensor* of operations.

Following what can be done for Boolean circuits, we can design a notion of *quantum circuit*.

- Wires are horizontal lines, representing the life-span of one qubit;
- Vertical juxtaposition of wires corresponds to the spacial separation of qubits. By convention, the top wire is the first qubit in the list of qubits, the last wire is the last qubit;
- Boxes on wires corresponds to actions. They can span over several wires to represent action on several qubits;
- Horizontal sequences of boxes correspond to the successive action of boxes.

3.4.8 An example of quantum circuit is shown in Fig. 3. There are two wires. The circuit corresponds to the following sequence of actions: apply U_1 on qubit 1; apply U_2 on qubit 2; apply U_3 on qubits 1 and 2; apply simultaneously U_4 and U_5 on qubit 1 and 2 respectively.

3.4.9 In the circuit of Fig. 3, we decided to place U_1 before U_2 , and to place U_4 and U_5 simultaneously. Thanks 3.4.6, we know that this is completely arbitrary: boxes behave as beads on a string, and one can freely “move” boxes along wires. The only limit being that in general, one cannot make them go over one another. In particular, the two circuits presented in Fig. 4 then corresponds to the same action on the quantum memory.

3.4.10 Suppose that A and B are two linear operations acting on $\mathcal{H}$. Using the lexicographic ordering of the basis, they can be represented with the matrices

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

Then $A \otimes B$ is defined as a block-matrix, in a same way as tensors of vectors in 3.3.8 as follows

$$\begin{aligned} A \otimes B &= \begin{pmatrix} a_{11}B & a_{12}B \\ a_{21}B & a_{22}B \end{pmatrix} \\ &= \begin{pmatrix} a_{11} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} & a_{12} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \\ a_{21} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} & a_{22} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}b_{11} & a_{11}b_{12} & a_{12}b_{11} & a_{12}b_{12} \\ a_{11}b_{21} & a_{11}b_{22} & a_{12}b_{21} & a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} & a_{22}b_{11} & a_{22}b_{12} \\ a_{21}b_{21} & a_{21}b_{22} & a_{22}b_{21} & a_{22}b_{22} \end{pmatrix} \end{aligned}$$

The basis ordering is the lexicographic order used in 3.3.3: canonical basis elements are listed as $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

3.4.11 The order in the tensor product is important: performing $Id \otimes \text{HAD}$ or $\text{HAD} \otimes Id$ on a 2-qubit system is not the same thing. This is reflected by corresponding matrices, which are respectively

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}, \quad \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}.$$

3.4.12 Not all 4×4 unitary matrix corresponds to a tensor of two 2×2 unitary matrices. For instance, the operation

$$\text{SWAP} \triangleq \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{matrix} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{matrix}$$

is unitary, but it cannot be written as $U \otimes V$ with U and V 1-qubit operations. Written as an operation on the canonical basis, corresponds to the function

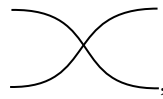
$$|x\rangle \otimes |y\rangle \mapsto |y\rangle \otimes |x\rangle.$$

We call it the *swap* operation.

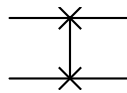
3.4.13 The swap operation has the same effect as literally swapping the position of two qubits. And indeed, one can show that

$$\text{SWAP} \circ (U \otimes \text{Id}) \circ \text{SWAP} = \text{Id} \otimes U.$$

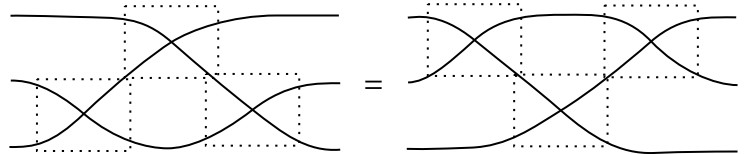
3.4.14 Thanks to 3.4.13, the graphical representation of quantum circuits can be extended with a special representation for the swap operation: the crossing of wires, as follows:



or, more compactly,



The first graphical representation is compatible with a series of equational properties satisfied by quantum circuits equipped with swaps, such as



3.4.15 The equational theory derived from the rules such as the one shown in 3.4.14 gives rise to what is called a PROP⁴. The bottom line is that

Wires can be shuffled as much as we want, the only constraint is that they need to always stay *from left to right*.

Said otherwise, the spacial ordering of wires is irrelevant.

3.4.16 The fact that one cannot exchange two sequential actions on the same wire means that there is a notion of *causality* in the sequence of actions. Since the spacial ordering of wires is irrelevant, this causality is somehow the only important information to keep track of. The nice, planar circuit representation contains too much information: we can instead use directed acyclic graphs (DAG) to represent circuit. The circuits of Fig. 4 can then be represented in a unique manner by the DAG shown in Fig 5. If one has to programmatically construct a circuit, a DAG might be more suitable than a sequence of gates. This is for instance one of the possible circuit representation within the PYTHON library QISKIT⁵.

⁴The canonical reference is [a paper from S. Lack](#), but it is a bit off topic

⁵<https://qiskit.org/documentation/stubs/qiskit.dagcircuit.DAGCircuit.html>

3.5.3 Hadamard gate. The *Hadamard* presented in 2.8.6 is also amongst the usual quantum gates. It is written H , or HAD. It is its own inverse: $H^2 = Id$.

3.5.4 T-gate. By composing Pauli gates, S -gates and Hadamard gates one can only reach a finite number of unitary matrices. To get an infinite set, we need to add a refined phase-gate: the typical choice is the square root of S , the T -gate:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

3.5.5 Theorem (Approximate Universality). Consider a unitary U acting on $\mathcal{H}$, and an error $\varepsilon > 0$. There exists a sequence C of gates H and T such that C implements U up to error ε :

$$\forall |\psi\rangle, \quad \|(U - C)|\psi\rangle\| \leq \varepsilon \cdot \|\psi\rangle\|.$$

Moreover, this sequence of gates is “short” in the sense that there is a constant c such that the size of C is in $O(\log^c(1/\varepsilon))$.

3.5.6 Th. 3.5.5 is a result known since 1995 with the now standard *Solovay-Kitaev algorithm*, with $c = 3.97$, followed in 2002 by an improvement with $c = 3 + \delta$, for δ arbitrarily small. Of course, as usual in this situation, the smallest δ is, the biggest the overhead. Since the Solovay-Kitaev algorithm based on a geometric interpretation, the circuit synthesis problem for 1-qubit gates has been greatly improved with algorithms relying on the resolution of Diophantine equations. In the 2010’s, several competing groups developed more and more refined versions; the last paper of this academic jousting [RS16] yields an optimal strategy with $c = 1$. From a practical standpoint, it provides a very fast and efficient procedure to approximate Z -rotations, i.e. unitaries of the form

$$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

For instance, for $\theta = \pi/128$ and $\varepsilon = 10^{-10}$, the procedure yields in a tenth of a second the approximation

HTSHTSHTSHTHTHTSHTHTSHTSHTSHTHTHTSHTSHTHTHTSHTHTSHTHTH
THTHTHTSHTSHTSHTHTSHTHTSHTHTHTHTSHTHTHTSHTHTSHTHTHTHTS
HTSHTSHTHTHTSHTSHTSHTHTSHTSHTSHTSHTHTSHTHTSHTSHTHTHTH
THTSHTHTHTHTSHTSHTHTSHTHTHTSHTHTHTHTHTSHTSHTHTHTHT
HTSHTHTHTHTSHTHTHTHTHTH.

If these optimized algorithms do not usually bring anything in the relative asymptotic complexity of quantum and classical algorithms, they are crucial when discussing practical implementations.

3.5.7 Rotations If we want to perform an exact *circuit synthesis* of 1-qubit unitaries, one strategy is to include the so-called *rotations gates* $R_X(\theta)$, $R_Y(\theta)$ and $R_Z(\theta)$, parameterized by an angle θ . These rotations respectively corresponds to rotations around the X , Y and Z axis of the Bloch sphere. Choosing G from $\{X, Y, Z\}$, they are defined as

↳ 3.2.9

follows:

$$R_G(\theta) \triangleq e^{-i\frac{\theta}{2} \cdot G} = \cos\left(\frac{\theta}{2}\right) \cdot Id - i \sin\left(\frac{\theta}{2}\right) \cdot G.$$

Matrix-wise, they can be written as

$$\begin{aligned} R_X(\theta) &= \begin{pmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \\ R_Y(\theta) &= \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \\ R_Z(\theta) &= \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \end{aligned}$$

3.5.8 The equality $e^{-i\frac{\theta}{2} \cdot G} = \cos\left(\frac{\theta}{2}\right) \cdot Id - i \sin\left(\frac{\theta}{2}\right) \cdot G$ in 3.5.7 is derived as follows.

$$\begin{aligned} e^{-i\frac{\theta}{2} \cdot G} &= \sum_{k=0}^{\infty} \frac{1}{k!} \left(-i\frac{\theta}{2} \cdot G \right)^k \\ &= \sum_{k=0}^{\infty} \frac{1}{(2k)!} \left(-i\frac{\theta}{2} \cdot G \right)^{2k} + \sum_{k=0}^{\infty} \frac{1}{(2k+1)!} \left(-i\frac{\theta}{2} \cdot G \right)^{2k+1} \\ &= \sum_{k=0}^{\infty} \frac{(-1)^{2k} i^{2k} \left(\frac{\theta}{2}\right)^{2k}}{(2k)!} (G)^{2k} + \sum_{k=0}^{\infty} \frac{(-1)^{2k+1} i^{2k+1} \left(\frac{\theta}{2}\right)^{2k+1}}{(2k+1)!} (G)^{2k+1} \\ &= \sum_{k=0}^{\infty} \frac{(i^2)^k \left(\frac{\theta}{2}\right)^{2k}}{(2k)!} (G^2)^k + \sum_{k=0}^{\infty} \frac{-(-1)^{2k} \cdot i \cdot (i^2)^k \left(\frac{\theta}{2}\right)^{2k+1}}{(2k+1)!} G \cdot (G^2)^k \\ &= \sum_{k=0}^{\infty} \frac{(-1)^k \left(\frac{\theta}{2}\right)^{2k}}{(2k)!} Id + \sum_{k=0}^{\infty} \frac{-i \cdot (-1)^k \left(\frac{\theta}{2}\right)^{2k+1}}{(2k+1)!} G \\ &= \left(\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} \left(\frac{\theta}{2}\right)^{2k} \right) \cdot Id - i \left(\sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} \left(\frac{\theta}{2}\right)^{2k+1} \right) \cdot G \\ &= \cos\left(\frac{\theta}{2}\right) \cdot Id - i \sin\left(\frac{\theta}{2}\right) \cdot G, \end{aligned}$$

remembering that $G^2 = Id$ and the trigonometric decompositions of Table 1 from Page 5.

3.5.9 The gate $R_Z(\theta)$ is defined with a global phase: it is customary to define a gate equivalent modulo global phase as follows:

$$P_H(\theta) \triangleq R_\theta \triangleq \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}.$$

We then have $T = P_H(\pi/4)$, $S = P_H(\pi/2)$ and $Z = P_H(\pi)$. Note how $P_H(\theta) = e^{i\frac{\theta}{2}} R_Z(\theta)$.

3.5.10 Theorem (Parametrization of 1-qubit gates). The canonical form of a unitary map U on 1 qubit is parametrized by 3 angles *up to a global phase* as follows:

$$U = \begin{pmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ e^{i\phi} \sin(\theta/2) & e^{i(\lambda+\phi)} \cos(\theta/2) \end{pmatrix}. \quad (25)$$

Such a map U can be written as a product of rotation gates and a global phase as follows:

$$\begin{pmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ e^{i\phi} \sin(\theta/2) & e^{i(\lambda+\phi)} \cos(\theta/2) \end{pmatrix} = e^{i(\phi+\lambda)/2} R_Z(\phi) R_Y(\theta) R_Z(\lambda). \quad (26)$$

3.5.11 Proof. To prove the decomposition of U into the form of Eq. (25), one can first note that the first column is the parametrization of a qubit state given in 3.2.9. The second column is obtained by specifying the fact that it is supposed to be of norm 1, and orthogonal to the first column.

For the proof of Eq. (26), we can simply unfold the definitions of the rotations, and execute the matrix multiplications as follows.

$$\begin{aligned} & e^{i(\phi+\lambda)/2} R_Z(\phi) R_Y(\theta) R_Z(\lambda) \\ &= e^{i(\phi+\lambda)/2} \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix} \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \begin{pmatrix} e^{-i\lambda/2} & 0 \\ 0 & e^{i\lambda/2} \end{pmatrix} \\ &= e^{i(\phi+\lambda)/2} \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix} \begin{pmatrix} e^{-i\lambda/2} \cos(\theta/2) & -e^{i\lambda/2} \sin(\theta/2) \\ e^{-i\lambda/2} \sin(\theta/2) & e^{i\lambda/2} \cos(\theta/2) \end{pmatrix} \\ &= e^{i(\phi+\lambda)/2} \begin{pmatrix} e^{-i(\lambda+\phi)/2} \cos(\theta/2) & -e^{i(\lambda-\phi)/2} \sin(\theta/2) \\ e^{i(\phi-\lambda)/2} \sin(\theta/2) & e^{i(\lambda+\phi)/2} \cos(\theta/2) \end{pmatrix} \\ &= \begin{pmatrix} \cos(\theta/2) & -e^{i\lambda} \sin(\theta/2) \\ e^{i\phi} \sin(\theta/2) & e^{i(\lambda+\phi)} \cos(\theta/2) \end{pmatrix}, \end{aligned}$$

which is the definition of U . □

3.5.12 The global phase in Th. 3.5.10 is there to make sure that “things fall in the right place”. From a computational point of view, it is invisible since quantum states are considered modulo global phases. Note however that global phases in unitaries are important to keep in mind when considering controlled gates (see 3.6.11 for a discussion).

3.6 Quantum Gates on Several Qubits

3.6.1 One-qubit operations are limited in the sense that from a basis ket they do not make it possible to realize entangled states such as the EPR state. Although the SWAP operation is a 2-qubit operations, it preserves separability: to get all possible states from a basis state we need more. A strategy to get there—and to get more expressivity in general—is to use *controlled operations*. § 3.3.6
§ 3.4.13

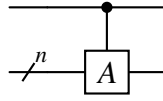
3.6.2 Notation To represent a bundle of n wires, we shall use the notation

$$\text{---}^n$$

3.6.3 If A is a unitary on n qubits, one can build a new unitary map called *controlled operation* as follows. Pick $A : \mathcal{H}^{\otimes n} \rightarrow \mathcal{H}^{\otimes n}$, and build the unitary $C-A : \mathcal{H} \otimes \mathcal{H}^{\otimes n} \rightarrow \mathcal{H} \otimes \mathcal{H}^{\otimes n}$ by adding a *control qubit*:

$$C-A: \begin{cases} |0\rangle \otimes |x\rangle \mapsto |0\rangle \otimes |x\rangle \\ |1\rangle \otimes |x\rangle \mapsto |1\rangle \otimes (A \cdot |x\rangle). \end{cases}$$

The map $C-A$ admits a graphical notation as follows:



The bullet is placed on the control qubit. The vertical line is not a wire: it is there to indicate which gate is being controlled by which wire.

3.6.4 The matrix corresponding to $C-A$ is defined blockwise as follows:

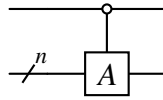
$$\begin{pmatrix} Id & 0 \\ 0 & A \end{pmatrix} \begin{matrix} |0x\rangle \\ |1x\rangle \end{matrix} .$$

$|0x\rangle \quad |1x\rangle$

It preserves the two orthogonal subspaces $|0\rangle \otimes \mathcal{H}^{\otimes n}$ and $|1\rangle \otimes \mathcal{H}^{\otimes n}$: it does nothing on the former while applying A on the latter.

3.6.5 The control operation is compositional: $C-(A \circ B)$ is $(C-A) \circ (C-B)$, and $C-I = I$.

3.6.6 It is useful to be able to perform *negative controls*: controls where the action is triggered in $|0\rangle$ and not on $|1\rangle$. The graphical notation is



The operation is

$$\begin{aligned} |0\rangle \otimes |x\rangle &\mapsto |0\rangle \otimes (A \cdot |x\rangle) \\ |1\rangle \otimes |x\rangle &\mapsto |1\rangle \otimes |x\rangle . \end{aligned}$$

3.6.7 The gate CNOT, or $C-X$ acts on 2 qubits as follows:

$$\text{CNOT: } \begin{cases} |0\rangle \otimes |x\rangle \mapsto |0\rangle \otimes |x\rangle \\ |1\rangle \otimes |x\rangle \mapsto |1\rangle \otimes |\neg x\rangle \end{cases} .$$

This can be written in a more compact way as

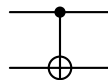
$$\text{CNOT: } |x\rangle \otimes |y\rangle \mapsto |x\rangle \otimes |y \oplus x\rangle .$$

The matrix of the CNOT gate is

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{matrix} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{matrix}$$

$|00\rangle \quad |01\rangle \quad |10\rangle \quad |11\rangle$

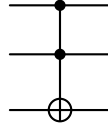
Its graphical representation in a circuit is



3.6.8 One can also build the gate $C-C-X$, also known as the *Toffoli* gate:

$$|xy\rangle \otimes |z\rangle \mapsto |xy\rangle \otimes |z \oplus xy\rangle.$$

As a circuit, it is



3.6.9 The control of a gate R_θ has an internal symmetry: $C-R_\theta$ sends every canonical basis vector to itself, with a phase $e^{i\theta}$ in the case $|11\rangle$. The matrix is

↳ 3.5.9

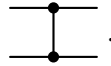
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{pmatrix},$$

and



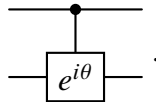
It sends $|x\rangle \otimes |y\rangle$ to $(e^{i\theta})^{xy} \cdot |x\rangle \otimes |y\rangle$.

3.6.10 A special case is the gate $C-Z$, a controlled rotation of angle π . It has the special diagrammatic notation



It sends $|x\rangle \otimes |y\rangle$ to $(-1)^{xy} \cdot |x\rangle \otimes |y\rangle$.

3.6.11 Control and global phases In 3.5.12, we mentioned that global phases are irrelevant for unitaries when applied on the whole state space. When the unitary corresponds to a subcircuit that might be controlled, the global phase is important. Indeed, consider the circuit



where the controlled gate applies a global phase to a qubit: $|x\rangle \mapsto e^{i\theta} |x\rangle$. The circuit performs the operation:

$$\begin{aligned} |0\rangle \otimes |x\rangle &\mapsto |0\rangle \otimes |x\rangle, \\ |1\rangle \otimes |x\rangle &\mapsto e^{i\theta} |1\rangle \otimes |x\rangle. \end{aligned}$$

That is, the global phase is only applied when the top qubit is in state $|1\rangle$. The action of this circuit is not equivalent to the identity: it is equivalent to a phase-gate on the top qubit.

We shall come back to this point when controlling arbitrary unitaries in Th. 4.3.2 and 4.3.4.

3.7 Creating New Quantum Registers

3.7.1 In 3.1.2, we discussed how the co-processor implements three classes of actions. So far, we discussed the internal actions, mathematically represented with unitary operators. In this section, we shall discuss the two other classes: allocation and initialization, and deallocation and reading.

3.7.2 Initialization is the easiest to handle: if the memory has say 1 qubit in state $|\phi\rangle$, adding a new qubit in state $|0\rangle$ corresponds to changing the memory state to $|\phi\rangle \otimes |0\rangle$. Before the action, the memory state space was $\mathcal{H}$, and after the action it is $\mathcal{H} \otimes \mathcal{H}$. Note that we arbitrarily decided to add the new qubit at the end, but we could have done otherwise.

3.7.3 Initialization as in 3.7.2 can be seen as a linear operation:

$$\begin{aligned}\mathcal{H} &\rightarrow \mathcal{H} \otimes \mathcal{H} \\ |x\rangle &\mapsto |x\rangle \otimes |0\rangle.\end{aligned}$$

This map is clearly not unitary since it is not total: vectors of the form $|\phi\rangle \otimes |1\rangle$ are not in the image. However, it still preserves norm and orthogonality, and if we consider it as a map from $\mathcal{H}$ to the *subspace* $\mathcal{H} \otimes |0\rangle$, it is unitary.

3.7.4 We say that we initialize *auxiliary qubits*, or *ancillas*. The circuit representation for ancillas is as follows

$$\begin{array}{c} \text{-----} \\ |0\rangle\text{-----} \end{array}$$

This represents the operation of 3.7.3.

3.8 Reading Quantum Registers

3.8.1 Measurement. The last class of action is concerned with “poking the existing memory state”: either by de-allocating part of it, or simply reading it. Any such operation leaves the realm of unitary maps and need the notion of *measurement*. It is moreover the *only* way to get back classical data out of quantum data.

3.8.2 The measure of a qubit state $\alpha \cdot |0\rangle + \beta \cdot |1\rangle$, we obtain

- with prob. $|\alpha|^2$: the Boolean value “0”, and the qubit is now in state $|0\rangle$
- with prob. $|\beta|^2$: the Boolean value “1”, and the qubit is now in state $|1\rangle$

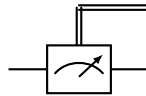
The qubit state has been probabilistically *projected* on one of the canonical basis vector. As vectors are normalized, the sum of probabilities is indeed equal to 1: we are sure to get a result.

3.8.3 Note that measuring $|0\rangle$ returns “0” with probability 1. Therefore, measuring twice the same qubit returns twice the same result.

3.8.4 Since the state of a quantum bit is collapsed by the measurement, we can consider that the qubit “disappeared” during the process: a so-called *destructive measure*. In this case, we can consider that the qubit is turned into a bit. The circuit notation is extended with a special gate for the measurement, and a special wire type for bit, represented with a double line. Destructive measurements are represented with



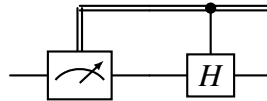
while non-destructive are



Here we consider that the gate “spits out” a bit on top. After the measurement gate, the qubit is either $|0\rangle$ or $|1\rangle$ depending on the measured Boolean value. We can control on bit wire: it means that the controlled gate is applied provided that the bit is in state 1 for black bullet and in state 0 for white bullets.

3.8.5 Beware A circuit with a measurement is in general *not* invertible, and the procedure presented in 3.4.17 makes in general no sense with measurements.

3.8.6 For instance,



implements the operation sending $\alpha|0\rangle + \beta|1\rangle$ to $|0\rangle$ with probability $|\alpha|^2$ and $|-\rangle$ with probability $|\beta|^2$. We know which state we are left with by reading the bit wire.

3.8.7 A qubit is never alone: it is part of a larger memory. The behavior in this general situation is very similar to what happens for 1 qubit: the state of the system is *projected*. More precisely, suppose that the memory consists of $n + 1$ qubits and that we measure the first qubit of the memory. The state space of the memory is $\mathcal{H} \otimes \mathcal{H}^{\otimes n}$. The state of the memory can then be written as

$$|\phi\rangle = \sum_{i=0}^{2^n-1} (\alpha_{0,i} \cdot |0\rangle \otimes |i\rangle + \beta_{1,i} \cdot |1\rangle \otimes |i\rangle). \quad (27)$$

Measuring the first qubit projects the state $|\phi\rangle$ to one of the two orthogonal subspaces $|0\rangle \otimes \mathcal{H}$ and $|1\rangle \otimes \mathcal{H}$. These subspaces respectively contains all of the vectors of the forms

$$\begin{aligned} |0\rangle \otimes \mathcal{H} &\triangleq \left\{ \sum_{i=0}^{2^n-1} \beta_i \cdot |0\rangle \otimes |i\rangle \right\}, \\ |1\rangle \otimes \mathcal{H} &\triangleq \left\{ \sum_{i=0}^{2^n-1} \gamma_i \cdot |1\rangle \otimes |i\rangle \right\}. \end{aligned}$$

Said otherwise, the vectors of $|0\rangle \otimes \mathcal{H}$ only have canonical basis kets starting with 0, and the vectors of $|1\rangle \otimes \mathcal{H}$ only have canonical basis kets starting with 1.

[3.8.2](#)

3.8.8 Measuring the first qubit of the quantum memory, the vector $|\phi\rangle$ in Eq. (27) is then collapsed into either

$$|\phi_0\rangle = \sum_{i=0}^{2^n-1} \frac{\alpha_{0,i}}{\rho_0} \cdot |0\rangle \otimes |i\rangle = |0\rangle \otimes \left(\sum_{i=0}^{2^n-1} \frac{\alpha_{0,i}}{\rho_0} \cdot |i\rangle \right)$$

or

$$|\phi_1\rangle = \sum_{i=0}^{2^n-1} \frac{\beta_{1,i}}{\rho_1} \cdot |1\rangle \otimes |i\rangle = |1\rangle \otimes \left(\sum_{i=0}^{2^n-1} \frac{\beta_{1,i}}{\rho_1} \cdot |i\rangle \right)$$

with

$$\rho_0 = \sqrt{\sum_{i=0}^{2^n-1} |\alpha_{0,i}|^2}, \quad \rho_1 = \sqrt{\sum_{i=0}^{2^n-1} |\alpha_{1,i}|^2}.$$

The collapse happens *modulo renormalization*: the adjunction of ρ_0 and ρ_1 ensures that $|\phi_0\rangle$ and $|\phi_1\rangle$ are unit vectors. We can separate the state of the first qubit, and if we define

$$|\psi_0\rangle = \sum_{i=0}^{2^n-1} \frac{\alpha_{0,i}}{\rho_0} \cdot |i\rangle, \quad |\psi_1\rangle = \sum_{i=0}^{2^n-1} \frac{\beta_{1,i}}{\rho_1} \cdot |i\rangle,$$

they are also normalized vectors, and

$$|\phi\rangle = \rho_0 \cdot |0\rangle \otimes |\psi_0\rangle + \rho_1 \cdot |1\rangle \otimes |\psi_1\rangle. \quad (28)$$

3.8.9 Having set up the framework in 3.8.8, we can now use the decomposition of Eq. 28 to describe the process of the measure of the first qubit:

- With probability ρ_0^2 , the measurement returns the bit 0 and the memory state collapses to $|0\rangle \otimes |\psi_0\rangle$;
- With probability ρ_1^2 , the measurement returns the bit 1 and the memory state collapses to $|1\rangle \otimes |\psi_1\rangle$.

In particular, if we perform a second measure of the same qubit, as for the case of 3.8.2 we get the same output bit with probability 1.

3.8.10 Let us consider the 2-qubit case, and measure the first qubit in

$$|\phi\rangle = \alpha \cdot |00\rangle + \beta \cdot |01\rangle + \gamma \cdot |10\rangle + \delta \cdot |11\rangle.$$

The projection on the subspace $|1\rangle \otimes \mathcal{H}$ is

$$\alpha \cdot |00\rangle + \beta \cdot |01\rangle,$$

and the one on subspace $|1\rangle \otimes \mathcal{H}$ is

$$\gamma \cdot |10\rangle + \delta \cdot |11\rangle.$$

If we write $\rho_0 = \sqrt{|\alpha|^2 + |\beta|^2}$ and $\rho_1 = \sqrt{|\gamma|^2 + |\delta|^2}$, the decomposition of Eq. (28) gives

$$|\phi\rangle = \rho_0 \cdot |0\rangle \otimes \left(\frac{\alpha}{\rho_0} \cdot |0\rangle + \frac{\beta}{\rho_0} \cdot |1\rangle \right) + \rho_1 \cdot |1\rangle \otimes \left(\frac{\gamma}{\rho_1} \cdot |0\rangle + \frac{\delta}{\rho_1} \cdot |1\rangle \right).$$

We can check that we are in the situation of 3.8.8: $|\rho_0|^2 + |\rho_1|^2 = 1$, and each component is normalized. According to 3.8.9, by measuring the first qubit we then obtain

- the bit 0 with probability $|\rho_0|^2 = |\alpha|^2 + |\beta|^2$, and the state of the system is now

$$|0\rangle \otimes \left(\frac{\alpha}{\rho_0} \cdot |0\rangle + \frac{\beta}{\rho_0} \cdot |1\rangle \right)$$

- the bit 1 with probability $|\rho_1|^2 = |\gamma|^2 + |\delta|^2$, and the state of the system is now

$$|1\rangle \otimes \left(\frac{\gamma}{\rho_1} \cdot |0\rangle + \frac{\delta}{\rho_1} \cdot |1\rangle \right).$$

3.8.11 We can then proceed and measure the second qubit. If we had measured 0 for the first qubit, the state is

$$|0\rangle \otimes \left(\frac{\alpha}{\rho_0} \cdot |0\rangle + \frac{\beta}{\rho_0} \cdot |1\rangle \right),$$

and measuring the second qubit yields

- the bit 0 with probability $\left| \frac{\alpha}{\rho_0} \right|^2$, and the state of the system is now $|00\rangle$;
- the bit 1 with probability $\left| \frac{\beta}{\rho_0} \right|^2$, and the state of the system is now $|01\rangle$.

If we had instead measured 1 for the first qubit, the state is

$$|1\rangle \otimes \left(\frac{\gamma}{\rho_1} \cdot |0\rangle + \frac{\delta}{\rho_1} \cdot |1\rangle \right).$$

and measuring the second qubit yields

- the bit 0 with probability $\left| \frac{\gamma}{\rho_1} \right|^2$, and the state of the system is now $|10\rangle$;
- the bit 1 with probability $\left| \frac{\delta}{\rho_1} \right|^2$, and the state of the system is now $|11\rangle$.

3.8.12 Combining the sequence of measurements, when a 2-qubit system is in state

$$|\phi\rangle = \alpha \cdot |00\rangle + \beta \cdot |01\rangle + \gamma \cdot |10\rangle + \delta \cdot |11\rangle,$$

measuring the first then the second qubit yields the values

- "00" with the state now at $|00\rangle$ with probability $|\alpha|^2$;
- "01" with the state now at $|01\rangle$ with probability $|\beta|^2$;
- "10" with the state now at $|10\rangle$ with probability $|\gamma|^2$;
- "11" with the state now at $|11\rangle$ with probability $|\delta|^2$.

3.8.13 Note one can measure qubits in an arbitrary order, this does not change the final result.

3.8.14 In general, if we measure the whole state of a memory of n qubits:

$$\sum_{i=0}^{2^n-1} \alpha_i \cdot |i\rangle,$$

we retrieve the bitstring corresponding to i (as discussed in 3.3.5) with probability $|\alpha_i|^2$. In this case, the memory state is collapsed to the canonical basis ket $|i\rangle$.

3.8.15 Deferred measurements. The *deferred measurement* principle states that one can always “push” the measurements at the end of a circuit without changing its overall behavior. The trick is to replace classically controlled gates with quantum controlled gates. For instance:



has the same effect as



3.8.16 Example. Applied to the state $|00\rangle$, the circuit (29) has the following effect:

$$\begin{aligned}
 |00\rangle &\mapsto \frac{1}{\sqrt{2}} |0\rangle \otimes (|0\rangle + |1\rangle) && \text{applying HAD} \\
 &= \frac{1}{\sqrt{2}} (|00\rangle + |01\rangle) \\
 &\mapsto \begin{cases} (|0\rangle, \text{bit } 0) & \text{with prob. } 1/2 \\ (|0\rangle, \text{bit } 1) & \text{with prob. } 1/2 \end{cases} && \text{measuring} \\
 &\mapsto \begin{cases} (|0\rangle, \text{bit } 0) & \text{with prob. } 1/2 \\ (|1\rangle, \text{bit } 1) & \text{with prob. } 1/2 \end{cases} && \text{applying CNOT}
 \end{aligned}$$

Thus, overall, the qubit is in state $|0\rangle$ or $|1\rangle$ with probability $\frac{1}{2}$. The bit wire stores the value of the qubit wire.

The circuit (30) has instead the effect:

$$\begin{aligned}
 |00\rangle &\mapsto \frac{1}{\sqrt{2}} |0\rangle \otimes (|0\rangle + |1\rangle) && \text{applying HAD} \\
 &= \frac{1}{\sqrt{2}} (|00\rangle + |01\rangle) \\
 &\mapsto \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) && \text{applying CNOT} \\
 &\mapsto \begin{cases} (|0\rangle, \text{bit } 0) & \text{with prob. } 1/2 \\ (|1\rangle, \text{bit } 1) & \text{with prob. } 1/2 \end{cases} && \text{measuring.}
 \end{aligned}$$

We therefore get the same effect as the previous circuit.

3.8.17 Measuring in other bases The measurement presented in 3.8.2 “tests” a qubit against the basis states $|0\rangle$ and $|1\rangle$. It can be spelled out with the scalar product: the probability of getting $|0\rangle$ while measuring $|\phi\rangle$ is

$$|\langle 0 | \phi \rangle|^2.$$

This turns out to be arbitrary, and it can in fact be done in any basis.⁶ For instance, we can measure a qubit against the basis $\{|+\rangle, |-\rangle\}$: we get $|+\rangle$ with probability

$$|\langle + | \phi \rangle|^2. \quad (31)$$

Of course, the bit returned by the measurement is not defined *a priori* and this has to be specified in advanced.

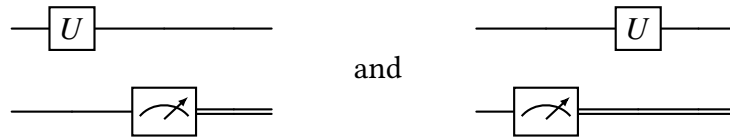
3.8.18 One can always reduce the problem of the general measurement presented in 3.8.17 to the measure in the canonical basis. Consider the probability of Eq. (31): using 2.9.2 we can reduce it as follows:

$$|\langle + | \phi \rangle|^2 = |\langle \text{HAD}+ | \text{HAD}\phi \rangle|^2 = |\langle 0 | \text{HAD}\phi \rangle|^2.$$

Measuring in the basis $|+\rangle, |-\rangle$ can therefore be obtained by applying a Hadamard gate followed with a measurement in the standard basis.

3.8.19 By abuse of language we say that we measure in the basis HAD, referring to the fact that the unitary can be regarded as an encoding of the basis (as in Eq (21) in 2.8.6).

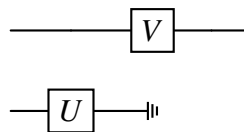
3.8.20 Commutation of measure The permutation of unitary gates discussed in 3.4.9 extends to the case where we perform measurements: both



describe the same computation.

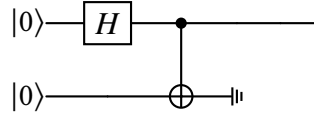
3.9 Discarding Quantum Registers

3.9.1 In regular programming languages, it makes sense to allocate and free (or *discard*) memory on the fly. With quantum registers, discarding amounts to perform a *destructive measure* and forget the result of the measurement. Discarding is represented in circuits with a small symbol reminiscent of “ground”. In the following example, we discard the second qubit after the application of a gate U :



⁶As a matter of fact, it can be formalized in the more general framework of POVM but this is out of the scope of this document.

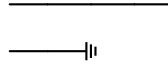
3.9.2 In general, discarding a qubit generates a probabilistic distribution of states. For instance, the circuit



returns a memory with only one qubit (the upper wire), whose state is $|0\rangle$ or $|1\rangle$ with probability $1/2$.

3.9.3 Safely discarding a wire requires to make sure that it is *not entangled* with the rest of the memory. As an example, one can make sure that it is in a canonical basis state. Indeed, if we were to discard the first qubit, the measurement process behind the discard operation would project the state of the memory on the subspaces $|0\rangle \otimes \mathcal{H}^{\otimes n}$ or $|1\rangle \otimes \mathcal{H}^{\otimes n}$. If the first qubit is separated from the rest of the memory, in state $|0\rangle$ or $|1\rangle$, the projection does not do anything beside focusing on the corresponding subspace: there is no loss of information.

3.9.4 For instance, suppose that the input of the circuit



in in state

$$\sum_x \alpha_x |x\rangle \otimes |0\rangle = \left(\sum_x \alpha_x |x\rangle \right) \otimes |0\rangle,$$

then the output is

$$\sum_x \alpha_x |x\rangle.$$

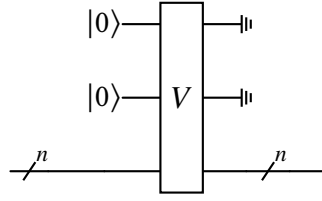
3.9.5 The canonical use-case for discard is for managing ancillas. A frequent situation is when a unitary operation U on n qubits might be cumbersome to implement with exactly n wires, but easier if we allocate more “room”, that is, more wires. The operation U is then seen as a block of a larger unitary V acting on $\mathcal{H}^{\otimes k} \otimes \mathcal{H}^{\otimes n}$. Assume $k = 2$ for simplicity. Then

$$V = \begin{pmatrix} U & 0 & 0 & 0 \\ 0 & W_{01,01} & W_{01,10} & W_{01,11} \\ 0 & W_{10,01} & W_{10,10} & W_{10,11} \\ 0 & W_{11,01} & W_{11,10} & W_{11,11} \end{pmatrix} \quad (32)$$

for matrices $W_{xy,zl}$ acting on $\mathcal{H}^{\otimes n}$. In the block decomposition of V , each column corresponds to a possible value for the two auxiliary qubits: $|00\rangle$ corresponds to the column with U , and the other columns are associated with $|01\rangle$, $|10\rangle$ and $|11\rangle$. Said otherwise, the action of V on a canonical basis vector is

$$V : \begin{cases} |00\rangle \otimes |x\rangle \mapsto |0 \cdots 0\rangle \otimes (U |x\rangle) \\ |yz\rangle \otimes |x\rangle \mapsto |01\rangle \otimes (W_{01,yz} |x\rangle) + |10\rangle \otimes (W_{10,yz} |x\rangle) + |11\rangle \otimes (W_{11,yz} |x\rangle) \end{cases}$$

when $yz \neq 00$. The circuit for U is then



It is design so that the only column used in Eq. (32) is the most-left one, corresponding to the case where the ancillas are $|00\rangle$. The action of V then return something of the form $|00\rangle \otimes \dots$, therefore making the ancillas qubits separated from the environment: we can discard them without perturbing the global state of the memory. Globally, we recover the action of U , as desired.

3.9.6 The circuit presented in 3.9.5 can be inverted using the technique of 3.4.17. Indeed, the inverse of V is blockwise:

$$V^{-1} = \begin{pmatrix} U^{-1} & 0 \\ 0 & W^{-1} \end{pmatrix}$$

where W is the block made out of the $W_{xy,zt}$'s. The ancillas are kept initialized at 0.

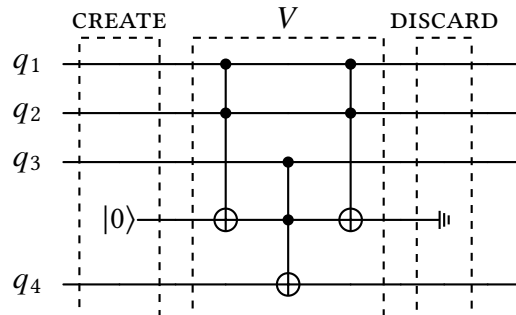
3.9.7 If V were to set the ancilla back to some canonical basis state other than $|0\rangle$, the procedure of 3.9.5 would still work: we could still measure without perturbing the global state. The inverse discussed in 3.9.6 would still be valid, modulo the fact that ancillas would have to be set to the correct, non-*ket0* value.

3.9.8 A standard case for an operation U described using a larger matrix V as in 3.9.5 is when the matrix U performs an action on the basis vectors, made of several subcomputations. We can store the subcomputations in ancilla qubits, building the final result step by step. Once it is done, we then uncompute the ancillas by reversing the local operations.

As an example, consider the gate $C-C-C-X$, a flip gate controlled by 3 qubits. On canonical basis vectors, it performs the operation

$$|x\rangle \otimes |y\rangle \otimes |z\rangle \otimes |t\rangle \mapsto |x\rangle \otimes |y\rangle \otimes |z\rangle \otimes |z \oplus xyz\rangle.$$

This operation can be decomposed into two conjunctions: first x with y , then xy with z . Each conjunction can be realized with a Toffoli gate, and we can store the result of the first conjunction inside an ancilla. The circuit is as follows.



It acts on 3 qubits, therefore on the space $\mathcal{H}^{\otimes 3} \otimes \mathcal{H}$, and consists of three parts:

$$\begin{aligned} \text{CREATE} : \mathcal{H}^{\otimes 3} \otimes \mathcal{H} &\rightarrow \mathcal{H}^{\otimes 3} \otimes |0\rangle \otimes \mathcal{H}, \\ V : \mathcal{H}^{\otimes 3} \otimes \mathcal{H} \otimes \mathcal{H} &\rightarrow \mathcal{H}^{\otimes 3} \otimes \mathcal{H} \otimes \mathcal{H}, \\ \text{DISCARD} : \mathcal{H}^{\otimes 3} \otimes \mathcal{H} \otimes \mathcal{H} &\rightarrow \mathcal{H}^{\otimes 3} \otimes \mathcal{H}. \end{aligned}$$

The operations CREATE and V composes since

$$\mathcal{H}^{\otimes 3} \otimes |0\rangle \otimes \mathcal{H} \subseteq (\mathcal{H}^{\otimes 3} \otimes |0\rangle \otimes \mathcal{H}) \oplus (\mathcal{H}^{\otimes 3} \otimes |1\rangle \otimes \mathcal{H}) = \mathcal{H}^{\otimes 3} \otimes \mathcal{H} \otimes \mathcal{H}.$$

In general, the behavior of DISCARD is *probabilistic*, as the 3rd qubit is measure. However, the state is not completely general: it is resulting from the function $V \circ \text{CREATE}$; let us compute its action on a *basis canonical state*:

$$\begin{aligned} |x\rangle |y\rangle |z\rangle \otimes |t\rangle &\mapsto |x\rangle |y\rangle |z\rangle \otimes |0\rangle \otimes |t\rangle && \text{by CREATE} \\ &\mapsto |x\rangle |y\rangle |z\rangle \otimes |0 \oplus xy\rangle \otimes |t\rangle && \text{by 1st Toffoli} \\ &= |x\rangle |y\rangle |z\rangle \otimes |xy\rangle \otimes |t\rangle \\ &\mapsto |x\rangle |y\rangle |z\rangle \otimes |xy\rangle \otimes |z \oplus xyz\rangle && \text{by 2nd Toffoli} \\ &\mapsto |x\rangle |y\rangle |z\rangle \otimes |xy \oplus xy\rangle \otimes |z \oplus xyz\rangle && \text{by 3rd Toffoli} \\ &= |x\rangle |y\rangle |z\rangle \otimes |0\rangle \otimes |t \oplus xyz\rangle. \end{aligned}$$

For compactness, we omitted some of the tensors. This maps therefore only populates a subspace of its co-domain:

$$V \circ \text{CREATE} : \mathcal{H}^{\otimes 3} \otimes \mathcal{H} \rightarrow \mathcal{H}^{\otimes 3} \otimes |0\rangle \otimes \mathcal{H}.$$

Whatever input $|\phi\rangle \in \mathcal{H}^{\otimes 3} \otimes \mathcal{H}$

$$|\phi\rangle = \sum_{x,y,z,t \in \{0,1\}} \alpha_{x,y,z,t} |x\rangle |y\rangle |z\rangle \otimes |t\rangle,$$

the resulting state lives in $\mathcal{H}^{\otimes 3} \otimes |0\rangle \otimes \mathcal{H}$:

$$(V \circ \text{CREATE}) |\phi\rangle = \sum_{x,y,z,t \in \{0,1\}} \alpha_{x,y,z,t} |x\rangle |y\rangle |z\rangle \otimes |0\rangle \otimes |t \oplus xyz\rangle.$$

If we perform a destructive measurement, the projection will have no effect, and we retrieve the vector

$$\sum_{x,y,z,t \in \{0,1\}} \alpha_{x,y,z,t} |x\rangle |y\rangle |z\rangle \otimes |t \oplus xyz\rangle.$$

3.9.9 Although the ancilla is not written in the same place, the circuit of 3.9.8 follows the specification given in 3.9.5: although we allocate and discard auxiliary registers, it corresponds to a unitary operation.

3.10 Cloning, Copy, Teleportation

3.10.1 Quantum information cannot be cloned: it is the so-called *no-cloning theorem*. More precisely, one cannot physically realize the map

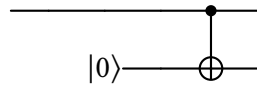
$$|\phi\rangle \otimes |0\rangle \mapsto |\phi\rangle \otimes |\phi\rangle$$

what would work for *all* ket vectors. One way to understand it is by writing the ket $|\phi\rangle$ as a column vector: the map should therefore send

$$\begin{pmatrix} \alpha \\ 0 \\ \beta \\ 0 \end{pmatrix} \mapsto \begin{pmatrix} \alpha^2 \\ \alpha\beta \\ \alpha\beta \\ \beta^2 \end{pmatrix}.$$

This operation is not linear: it cannot be synthesized with unitary maps...

3.10.2 If cloning is not possible, *copying* is possible. Consider the circuit



It sends a basis element $|x\rangle$ to $|xx\rangle$. This is clearly not the same map as in 3.10.1: it sends

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix} \mapsto \begin{pmatrix} \alpha \\ 0 \\ 0 \\ \beta \end{pmatrix}.$$

It is a linear map: in ket-notation, it does

$$\alpha |0\rangle + \beta |1\rangle \mapsto \alpha |00\rangle + \beta |11\rangle.$$

3.10.3 Teleportation In 3.8.2 we said that because of the projection that happens, some quantum information is lost when we perform a measurement. In particular, when we measure the qubit $\alpha |0\rangle + \beta |1\rangle$, the coefficients α and β disappear. This turns out to not be the case in general, and a non-intuitive effect can happen in the circuit presented in Fig. 6.

3.10.4 The circuit is typically read as follows. In the left dashed boxed, an entangled pair of qubits is built. The first qubit is sent to Alice, the other one to Bob. Alice has another qubit (the top wire) whose state she want to send to Bob. Unfortunately, they only share a classical communication channel! Thankfully, they can succeed: Alice performs the middle dashed box with the measurement of both of her qubits, she sends the result to Bob, and Bob uses the Boolean values he receives to execute a *correction* on his qubit: as we shall see, the qubit is now in the same state Alice's qubit was originally in. However, note that we did not clone anything: Alice's qubit has been measured, and his state is not a canonical basis element.

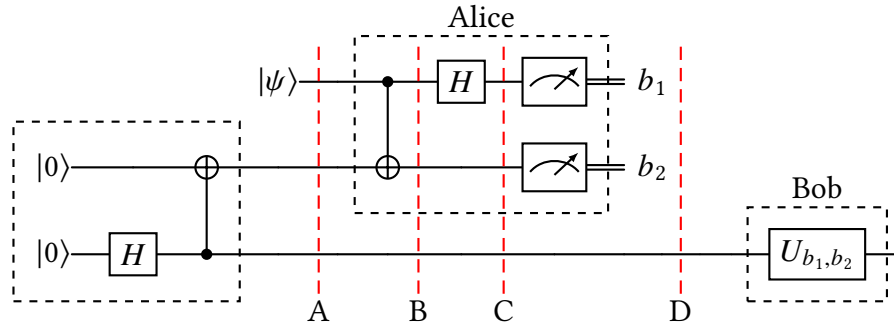


Figure 6: Scheme for Teleportation.

3.10.5 Let us develop the computation. Assume that $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$. Consider each layers in Fig. 6.

In (A).

$$\begin{aligned}
 |\psi\rangle \otimes |00\rangle &\xrightarrow{H} |\psi\rangle \otimes |0\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\
 &= |\psi\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle \otimes |0\rangle + |0\rangle \otimes |1\rangle) \\
 &\xrightarrow{CNOT} |\psi\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle) \\
 &= |\psi\rangle \otimes \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \\
 &= (\alpha |0\rangle + \beta |1\rangle) \otimes \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \\
 &= \frac{1}{\sqrt{2}} (\alpha |000\rangle + \alpha |011\rangle + \beta |100\rangle + \beta |111\rangle)
 \end{aligned}$$

In (B).

$$\frac{1}{\sqrt{2}} (\alpha |000\rangle + \alpha |011\rangle + \beta |110\rangle + \beta |101\rangle)$$

In (C).

$$\begin{aligned}
 &\frac{1}{\sqrt{2}} \left(\alpha \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |00\rangle + \alpha \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |11\rangle \right. \\
 &\quad \left. + \beta \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |10\rangle + \beta \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |01\rangle \right) \\
 &= \frac{1}{2} (\alpha |000\rangle + \alpha |100\rangle + \alpha |011\rangle + \alpha |111\rangle + \beta |010\rangle - \beta |110\rangle + \beta |001\rangle - \beta |101\rangle) \\
 &= \frac{1}{2} \left(|00\rangle \otimes (\alpha |0\rangle + \beta |1\rangle) + |01\rangle \otimes (\alpha |1\rangle + \beta |0\rangle) \right. \\
 &\quad \left. + |10\rangle \otimes (\alpha |0\rangle - \beta |1\rangle) + |11\rangle \otimes (\alpha |1\rangle - \beta |0\rangle) \right).
 \end{aligned}$$

In (D). The two first qubits got measured. The system lives in $\mathcal{H} \otimes \mathcal{H} \otimes \mathcal{H}$. The measurement projects the system on one of $|00\rangle \otimes \mathcal{H}$, $|01\rangle \otimes \mathcal{H}$, $|10\rangle \otimes \mathcal{H}$ and $|11\rangle \otimes \mathcal{H}$. Measuring, I then get with probability 1/4 two bits $b_1 b_2$ as follows:

- 00, and the state is now $|00\rangle \otimes (\alpha |0\rangle + \beta |1\rangle)$,

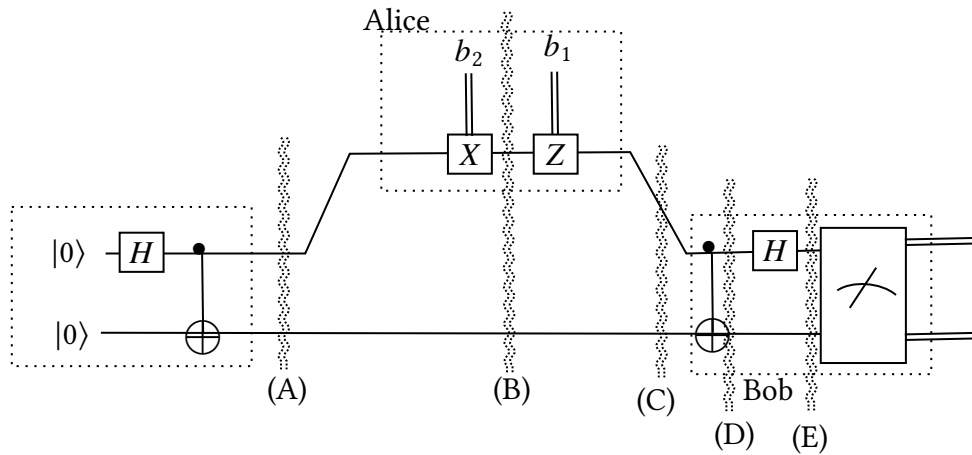


Figure 7: Scheme for Dense Coding.

- 01, and the state is now $|01\rangle \otimes (\alpha |1\rangle + \beta |0\rangle)$,
- 10, and the state is now $|10\rangle \otimes (\alpha |0\rangle - \beta |1\rangle)$,
- 11, and the state is now $|11\rangle \otimes (\alpha |1\rangle - \beta |0\rangle)$.

If Bob wants to get back $|\psi\rangle$, he needs to perform

$$U_{00} = Id, \quad U_{01} = X, \quad U_{10} = Z, \quad U_{11} = ZX.$$

3.10.6 Discussion Note that the teleportation protocol does not refer to the localization of Alice and Bob. The only constraint is for them to share an entangled pair of qubits. But once this is done, they can go as far of each other as they want, the protocol will succeed. One could argue that the teleportation of the state is instantaneous, thus breaking the physical law stating that information cannot move faster than the speed of light. But there is no contradiction here: the state of the qubit is only modified when Bob receives the results of the measures of Alice. These bits move inside a classical channel, subject to the law of physics, so nothing goes faster than the speed of light.

3.10.7 Recalling 3.8.18, Alice's action consists in measuring its two qubits in the *Bell basis*.

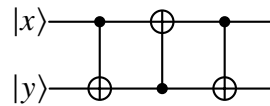
§ 3.3.7

3.10.8 Dense Coding. The teleportation algorithm can be somehow “inverted”: instead of sending a qubit through a classical channel, we can send two bits through a quantum channel, encoded on *one single qubit*. The algorithm is presented in Fig. 7. The gates X and Z are fired by Alice whenever the corresponding bit is set to 1. Bob should read the two bits at the end of the circuit in a non-probabilistic manner. See Exo. 3.11.12.

3.11 Exercises

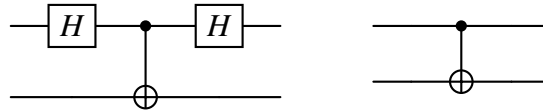
3.11.1 Show that the set of vectors in 3.3.4 is indeed a basis.

3.11.2 Consider the circuit



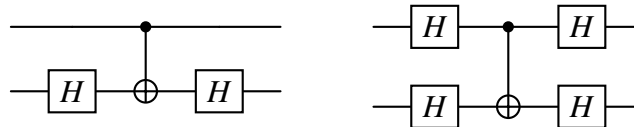
What does it compute? Give it in “function” form, and in matrix form (do not forget the basis ordering used for the representation!). Lay out the details in a convincing way.

3.11.3 Consider the following circuits.



Give an argument to show that they do not describe the same unitary operation. You might want to consider feeding the circuit with a well-chosen input state to ease the computation.

3.11.4 Consider the following circuits.

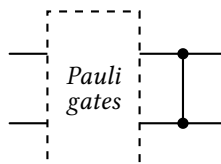


For each of them, give a simpler, equivalent circuit, and the corresponding linear map in function-style and in matrix form. Make sure to give the ordering of basis states you rely on. You might want to recall Exo 2.10.13, but it is not necessary.

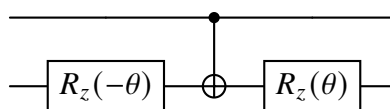
3.11.5 Consider the circuits



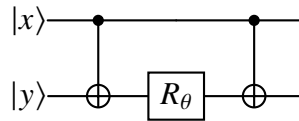
In each case, propose a new circuit where the gate C - Z is on the far right, of the form:



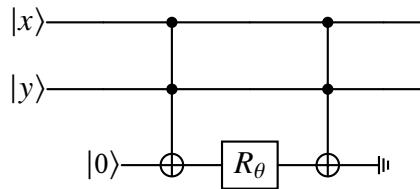
3.11.6 Consider the circuit



with θ a real number. Rewrite the circuit using only gates among CNOT and various $R_{\theta'}$, for well-chosen values of θ' .

3.11.7 Consider the circuit

with θ a real number. What does it compute? Give it in “function” form, and in matrix form (do not forget the basis ordering used for the representation!). Lay out the details in a convincing way.

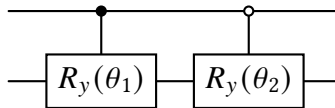
3.11.8 Consider the circuit

with θ a real number. What does it compute? Give it in “function” form, in (simpler) circuit-form, and in matrix form (do not forget the basis ordering used for the representation!). Lay out the details in a convincing way.

3.11.9 Section 3.9.8 gives a procedure to generate a multi-controlled NOT gate from Toffoli and ancillas. We want to realize a C^4 -NOT gate (a NOT gate with 4 controls). Give two circuits only consisting of Toffoli gates, making use of 3 ancillas for the first circuit and only 2 ancillas for the other circuit.

3.11.10 Consider the circuit

§ 3.5.7



1. Gives the corresponding matrices in the basis orderings $(|00\rangle, |01\rangle, |10\rangle, |11\rangle)$ and $(|00\rangle, |10\rangle, |01\rangle, |11\rangle)$.
2. Give a circuit made of multi-controlled 1-qubit gates and realizing the matrix

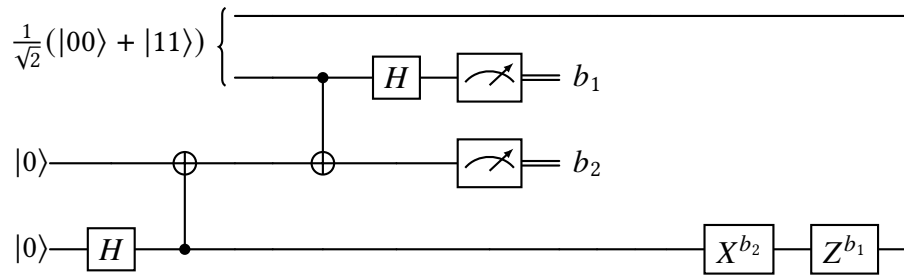
$$\begin{pmatrix} \cos(\theta_1) & 0 & 0 & 0 & -\sin(\theta_1) & 0 & 0 & 0 \\ 0 & \cos(\theta_2) & 0 & 0 & 0 & -\sin(\theta_2) & 0 & 0 \\ 0 & 0 & \cos(\theta_3) & 0 & 0 & 0 & -\sin(\theta_3) & 0 \\ 0 & 0 & 0 & \cos(\theta_4) & 0 & 0 & 0 & -\sin(\theta_4) \\ \sin(\theta_1) & 0 & 0 & 0 & \cos(\theta_1) & 0 & 0 & 0 \\ 0 & \sin(\theta_2) & 0 & 0 & 0 & \cos(\theta_2) & 0 & 0 \\ 0 & 0 & \sin(\theta_3) & 0 & 0 & 0 & \cos(\theta_3) & 0 \\ 0 & 0 & 0 & \sin(\theta_4) & 0 & 0 & 0 & \cos(\theta_4) \end{pmatrix}$$

when written in the canonical basis ordering

$$|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle.$$

Explain why it works.

3.11.11 Consider the teleportation algorithm of 3.10.3, where instead of teleporting a separated qubit we teleport one of the two qubits of a Bell pair:



We use here the representation that we shall meet in 4.6.6: X^{b_2} stands for the gate X when b_2 is true, and the identity otherwise. Similarly for Z^{b_1} : this is Z when b_1 is true and the identity when false.

Show that the output of this circuit is still a Bell pair, although on different qubits.

3.11.12 Spell out the details of the computation for the dense-coding algorithm in 3.10.8, when the circuit is fed with an arbitrary pair of bits b_1, b_2 .

4 Hardware Constraints and Circuit Synthesis

4.1 A bit of Complexity Theory

4.1.1 The notion of *complexity* refers to two related concepts: complexity of algorithms, and complexity of problems. In this section, we focus on the former, and we briefly recall what should be known about it in the context of quantum programming.

4.1.2 An algorithm is a mechanical procedure taking some input, and producing a result (possibly a simple “yes/no”) after a certain number of well-defined operations. The complexity of the algorithm is an estimate of the amount of resources required when the size of the inputs grows to infinity. The complexity can focus on the overall memory footprint of the algorithm (*space complexity*), or on the number of operations required to run it to completion (*time complexity*).

The “size of the input” is taken in a literal sense: if we were to store the input on a regular storage device (USB key, hard-drive, etc), the input size is the number of bits it takes to store it. For instance, storing a natural number N can be done in binary and this requires $\log_2(N)$ bits.

The notion of “number of operations” is of course a moving target, as it depends on the considered operations. In the conventional setting, one typically counts arithmetic operations and memory accesses. In the quantum setting, one considers the interaction with the quantum co-processor.

4.1.3 The complexity cares about the asymptotic behavior of the algorithm in a coarse way: If $f(x)$ is the quantity of resources required for processing an input of size x , we say that $f(x)$ is a *big-O* of $g(x)$ if there exists M and x_0 such that

$$\forall x \geq x_0, \quad f(x) \leq M \cdot g(x).$$

We write $f(x) = O(g(x))$.

In general, we consider functions of several variables: the amount of resources might be based on two parameters. For instance, an adder takes an input consisting of two natural numbers M and N . The complexity of the adder depends on N and M (for an adder, typically the complexity is $O(\log_2(M) + \log_2(N))$).

4.1.4 The definition of big-O in 4.1.3 hides constants and smaller growth. This makes it possible to write $x + c = O(x)$ and $c \cdot x = O(x)$, when c is a constant. We also have $x + \log(x) = O(x)$ for instance. The definition permits overapproximation, such as $x = O(x^2)$.

Such an approach makes the notion of “input size” and “amount of resources” somehow canonical. For instance, maybe the integer N requires $\log_2(N) + 4$ bits in memory to account for its type or other meta-data. Another, more involved example is when the input is a graph with v nodes and e edges: the resources needed to store the graph will grow in correlation with e and v , but we can omit the fact that node and edge identifiers are more than 1-bit long: as long as they are “small-enough” they won’t count in the complexity, and we can focus on v and e for the big-O formula.

4.1.5 The complexity of an algorithm gives some clue on how much useful it can be. For instance, one can expect an algorithm with an exponential complexity (i.e. a complexity of the form $O(e^x)$) to scale badly for large input sizes. On the contrary, an algorithm with a logarithmic, a linear or a low-degree polynomial complexity should be better-behaved: such complexities are the usual target for *reasonable* amount of resources.

4.1.6 The complexity of an algorithm is however not the last word: the complexity is in particular hiding constant terms that might end up being very large. Consider for instance an algorithm running in logarithmic time with, for an input of size x , the number of operations being $10^{40} + \log(x)$. This is $O(\log(x))$, an arguably very good complexity. In real-life though, even for very small x 's the algorithm requires a tantalizing amount of time to complete.

4.1.7 If the example of 4.1.6 might look contrived, such large, hidden overheads are unfortunately something rather common in the context of quantum algorithms. This is why concrete resource estimation is very important to assess the effectiveness of a quantum algorithm for a given task.

4.1.8 In the discussion so far, we are omitting an important fact: because of the potential use of measurement, quantum algorithms are *probabilistic algorithms*. We say that a quantum algorithm has a complexity of $O(f(x))$ when it *successfully* completes with probability at least $2/3$ in $O(f(x))$.

Note that the probability $2/3$ is somewhat arbitrary: any other high probability of success can be attained in $O(f(x))$ by executing the algorithm several times. For instance, executing twice the algorithm makes a new algorithm succeeding with probability $8/9$ in $O(f(x))$. We use the fact that $2 \cdot f(x) = O(f(x))$.

4.1.9 Finally, it is always good to keep in mind that the complexity of a quantum program might be radically changed along the compilation process. Indeed, the chosen hardware might require costly gate decompositions, various error correcting schemes and complex qubit mapping, blowing up the overall cost of running the program.

4.2 Low-level gate-sets

4.2.1 As discussed in 3.1.4 and 3.4.3, on a quantum memory one cannot directly implement arbitrary unitaries: each quantum co-processor is limited to a particular *gate-set*. These constraints can come from the physics of the implementation, but also from the error correcting scheme, limiting what is possible. Typically, these gate-sets are chosen so that any unitary operator can be synthesized, at least in an approximate manner.

4.2.2 Formally, we say that a gate-set S is *universal* if for all n , for all unitary operator U , there exists a circuit made out of gates from S realizing U . We say that the gate-set is *approximately universal* if for all n , for all unitary operator U , for all error $\varepsilon > 0$, there exists a circuit C made out of gates of S such that “ C approximates U up to ε ”, i.e.

$$\forall |\psi\rangle, \|(U - C)|\psi\rangle\| \leq \varepsilon \cdot \|\psi\rangle\|.$$

These two definitions generalize the 1-qubit case discussed in Sec. 3.5.5 and 3.5.7.

4.2.3 In order to perform quantum computation, the main gate-set that is required for every set-up is a set allowing one to implement *Clifford* operators. These are the operations generated by the gates HAD, S and CNOT. It is *not universal*, and in fact it is efficiently simulable on a classical machine.

Typical examples of universal gate sets extends Clifford gates with at least one non-Clifford gates, as follows.

- CNOT + 1-qubit rotations. This is the typical gate-set that can be realized in the context of linear optics or supraconducting qubits (the quantum co-processor of IBM, Google, Rigetti, *etc*). The universality of this gate set is proved in Sec. 4.3.
- $MS + R_z + R_x$ also form a universal gate set, with

$$MS(\theta) = \begin{pmatrix} \cos(\theta) & 0 & 0 & -i \sin(\theta) \\ 0 & \cos(\theta) & -i \sin(\theta) & 0 \\ 0 & -i \sin(\theta) & \cos(\theta) & 0 \\ -i \sin(\theta) & 0 & 0 & \cos(\theta) \end{pmatrix}$$

This is the Mølmer-Sørensen gate, also written XX . Parameterized by an angle θ , this gate is used in the context of the ion-trap technology⁷.

- Control-Z and the family of gates $J(\alpha)$, defined by

$$J(\theta) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & e^{i\theta} \\ 1 & -e^{i\theta} \end{pmatrix}.$$

This set of gates is in strong relation with *MBQC* (*Measurement-Based Quantum Computation*), discussed in Sec. 4.6.

- Clifford+ T . This is the typical gate-set for fault-tolerant quantum computation, *approximately* universal. This is discussed in Sec. 4.3.19.

Some more exotic universal gate-set exists, based on the Toffoli gate. A first result due to Kitaev [Kit97] shows that Toffoli, HAD and S are (approximately) universal. Maybe more surprisingly, Toffoli and any 1-qubit basis change such as HAD is also universal! [Shi03]. You might notice that none of these two gates contains coefficients with imaginary parts. The encoding of a generic unitary operation then requires an extraneous wire to “store” this imaginary part, but the encoding is efficient in the sense that one can turn a Clifford+ T circuit into a Toffoli+HAD circuit in a polynomial manner.

4.2.4 Clifford needs “a bit of help” to get to universality. Instead of using additional unitary gates such as Toffoli or 1-qubit gates, it is also possible to consider the use of *magic states* [BK05]. These are typically 1-qubit registers, initialized in a non-canonical basis state. The use of magic states requires measurements and the possibility to perform *some* form of classical processing in the quantum co-processor. We discuss magic states in Sec. 4.5.

⁷ see e.g. <https://arxiv.org/abs/1603.07678>

4.3 Universality of CNOT and 1-qubit rotations

4.3.1 In 3.4.3, we discussed how quantum internal operations are unitaries acting on the whole state space, while the quantum co-processor can only apply a handful of operations. In this section, we shall see how one can realize *any* unitary operation on *any* n -qubit states with only CNOT gates and 1-qubit rotations. The reader can refer to [arXiv:quant-ph/9503016](https://arxiv.org/abs/quant-ph/9503016) for a reference.

4.3.2 Theorem For all unitary U on 1-qubit, there exists an angle α and 3 1-qubit unitaries A_1, A_2, A_3 such that $A_1 A_2 A_3 = I$ and $U = e^{i\alpha} A_1 X A_2 X A_3$.

4.3.3 Proof. According to Th. 3.5.10, U can be written as

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta).$$

Define

$$\begin{aligned} A_1 &= R_z(\beta) R_y(\gamma/2), \\ A_2 &= R_y(-\gamma/2) R_z(-(\delta + \beta)/2), \\ A_3 &= R_z((\delta - \beta)/2), \end{aligned}$$

Then

$$\begin{aligned} A_1 A_2 A_3 &= R_z(\beta) R_y(\gamma/2) R_y(-\gamma/2) R_z(-(\delta + \beta)/2) R_z((\delta - \beta)/2) \\ &= R_z(\beta) R_y(\gamma/2) R_y(-\gamma/2) R_z(-\beta) \\ &= R_z(\beta) R_z(-\beta) \\ &= I. \end{aligned}$$

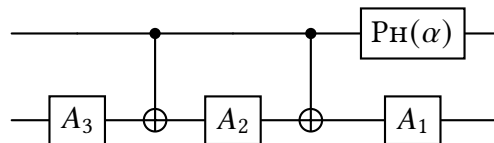
Then, we can realize that $XX = I$, and $XR_y(\alpha)X = R_y(-\alpha)$, and $XR_z(\alpha)X = R_z(-\alpha)$. The following the hold:

$$\begin{aligned} A_1 X A_2 X A_3 &= R_z(\beta) R_y(\gamma/2) X R_y(-\gamma/2) R_z(-(\delta + \beta)/2) X R_z((\delta - \beta)/2) \\ &= R_z(\beta) R_y(\gamma/2) \textcolor{blue}{X} R_y(-\gamma/2) \textcolor{blue}{X} X R_z(-(\delta + \beta)/2) X R_z((\delta - \beta)/2) \\ &= R_z(\beta) R_y(\gamma/2) \textcolor{blue}{R_y}(\gamma/2) \textcolor{green}{X} R_z(-(\delta + \beta)/2) \textcolor{green}{X} R_z((\delta - \beta)/2) \\ &= R_z(\beta) R_y(\gamma/2) R_y(\gamma/2) \textcolor{green}{R_z}((\delta + \beta)/2) \textcolor{green}{R_z}((\delta - \beta)/2) \\ &= R_z(\beta) R_y(\gamma) R_z(\delta), \end{aligned}$$

closing the proof. □

4.3.4 Theorem Let U be a 1-qubit unitary. The operation $C-U$ can be realized with only CNOT and 1-qubit gates.

4.3.5 Proof. Using Th. 4.3.2, one can decomposed U into $U = e^{i\alpha} A_1 X A_2 X A_3$. One can write the circuit:



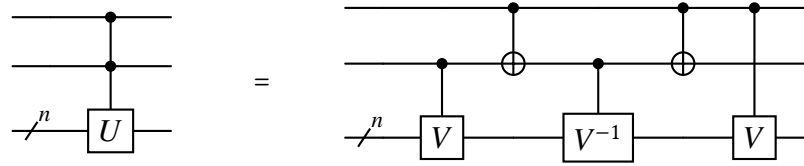
with $\text{PH}(\alpha)$ defined as in Sec. 3.5.9. Indeed, when the control wire is $|1\rangle$, we need to perform U , including the global phase. But this global phase should not be done with the control qubit is $|0\rangle$. This has already been discussed in 3.6.11. $\square$

4.3.6 Theorem Consider an n -qubit unitary U . There exists an n -qubit unitary V such that $V^2 = U$.

4.3.7 Proof. The existence of V derives from the decomposition of U as e^{iH} for some hermitian matrix H (see 2.9.8). One can then use $V = e^{iH/2}$, relying on 2.9.10. $\square$

4.3.8 Theorem Let U by an n -qubit unitary. The doubly-controlled gate $C-C-U$ can be realized with only CNOT and simply-controlled n -qubit gates.

4.3.9 Proof. Let V be defined as in Th. 4.3.6. A circuit realizing $C-C-U$ is



Indeed, the various cases for the controlling qubits are as follows:

- $00 \rightarrow$ on ne fait rien
- $01 \rightarrow$ on fait $C-V$ et $C-V_{\text{inv}} \rightarrow$ identité
- $10 \rightarrow$ d'abord $C-V_{\text{inv}}$ puis $C-V \rightarrow$ identité
- $11 \rightarrow C-V$ puis $C-V : C-U$

The operation U is then applied to the target qubit if and only if the control qubits are at 11. $\square$

4.3.10 Theorem Let U by a 1-qubit unitary. We can realize the multi-controlled gate $C-C-\dots-C-U$ with only CNOT and 1-qubit gates.

4.3.11 Proof. Using repeatedly Th. 4.3.8 we decompose $C-\dots-C-U$ into CNOT-gates and simply-controlled 1-qubit gates. We can conclude by using Th. 4.3.4 to decompose each simply-controlled 1-qubit gates into CNOT and 1-qubit unitaries, to reach the required form. $\square$

4.3.12 We are now almost ready to show that a $2^n \times 2^n$ unitary matrix can be realized (we say *synthesized*) with CNOT gates and 1-qubit unitaries. For this, we need a result allowing us to relate arbitrary unitary matrices and multi-controlled gates. The result is the following theorem, yielding Th. 4.3.15.

4.3.13 Theorem (*Cosine-Sine Decomposition*). Let U be any $n + 1$ -qubit unitary. One can decompose U as

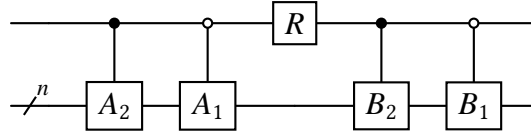
$$\begin{pmatrix} B_1 & 0 \\ 0 & B_2 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}$$

with A_1, B_1, A_2 and B_2 n -qubit unitaries, and C and S n -qubit diagonals such that $S^2 + C^2 = Id$.

4.3.14 Proof. See Appendix A.

4.3.15 Theorem. Any $n + 1$ -qubit unitary can be realized with multi-controlled, 1-qubit unitaries.

4.3.16 Proof. The proof proceeds by induction on n . In the base case, $n = 1$ and there is nothing to do. The inductive case makes use of the CS decomposition stated in Th. 4.3.13. This decomposition can be drawn as the circuit:



where R is a circuit realizing the matrix

$$\begin{pmatrix} C & -S \\ S & C \end{pmatrix}.$$

This circuit can be built using the technique on 3.11.10: the matrix C can be seen as a diagonal of Cosine functions, while the matrix S contains the corresponding Sine functions. This gives a *multiplexor* of multi-controlled R_y -rotations. $\square$

4.3.17 Theorem. Given a unitary matrices on n qubit, one can synthesize it with only CNOT gates and 1-qubit rotations.

4.3.18 Proof. The proof consists in chaining the decompositions. First, using Th. 4.3.15 the matrices yields a circuits consisting of multi-controls of 1-qubit gates. Then, using Th. 4.3.10 each of these multi-controls can be decomposed into CNOT and 1-qubit unitaries. Finally, each of these 1-qubit unitaries can be implemented with rotations using Th. 3.5.10. $\square$

4.3.19 Combining Th. 3.5.5 and 4.3.17, we can conclude that any n -qubit unitary can be implemented up to arbitrary error with only CNOT, Hadamard and T-gates.

4.3.20 Discussion The proof in Sec. 4.3.18 gives a construction for a circuit implementing an arbitrary unitary $2^n \times 2^n$ -matrix. The size of the circuit is clearly exponential on the number n of qubits. Is there a way of constructing a non-exponentially-sized circuit? Without any constraints on U , this is bound to fail. One can count the number of degrees of liberty for the system: the matrix contains 4^n complex numbers. Even if we

factor out the unitarity constraints, we still need on the order of 4^n angles to describe all of them. These angles are necessary: regardless of how we will handle them, they will end up crawling inside rotation gates in the circuit.

4.4 Tradeoffs: a Case-Study

4.4.1 Given a unitary matrix $2^n \times 2^n$, if one question is to be able to implement it with the available gate-set, the other is to be able to do it with a circuit of reasonable size. As discussed in 4.1.5, in computer science, “reasonable” is defined as “polynomial in n ”. The strategy proposed in 4.3.18 is clearly not reasonable with this definition.

In some situation, it is possible to do better when we can capitalize on the structure of the unitary. In order to see how this can work in practice, we propose to study two techniques to implement a multi-controlled R_θ -gate (where we write R_θ in place of $\text{PH}(\theta)$ for compactness).

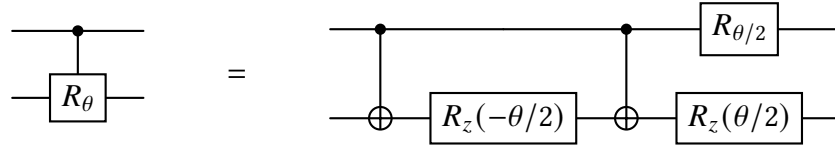
4.4.2 Exponentially large circuit Using Th. 4.3.4, note that we can write $R_\theta \triangleq \text{PH}(\theta)$ as

$$e^{i\frac{\theta}{2}} R_z(\theta/2) R_y(0) R_z(\theta/2).$$

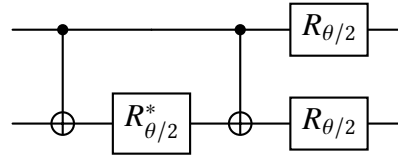
From the proof of Th. 4.3.2, we set

$$A_1 = R_z(\theta/2), A_2 = R_z(-\theta/2), A_3 = Id,$$

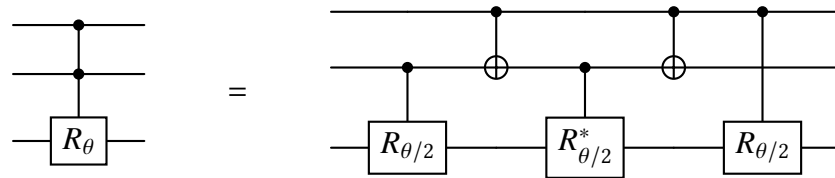
so that



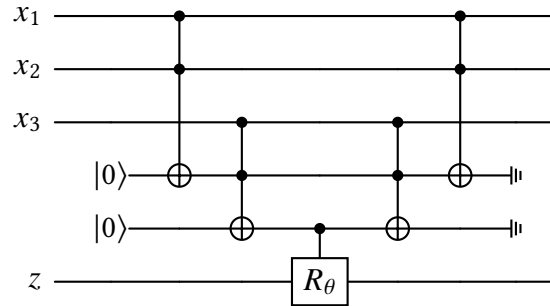
Remember 3.11.6: the circuit can be written using rotations $R_{\theta/2}$ instead of R_z -gates, as follows.



4.4.3 Let us now write a circuit for $C-C-R_\theta$: using Th. 4.3.8 with $U = R_\theta$, we get



follows.



In general, the gate $C^{n+1}-R_\theta$ with $n + 1$ controls requires n ancillas, $2n$ Toffoli gates and one $C-R_\theta$ gate. The former can be realized for instance with two HAD-gates and the circuit of Sec. 4.4.3, while the latter can be realized with the circuit of Sec. 4.4.2. We then have a linear-sized, much more compact circuit. We can also note that the angles that are required for the single-qubit rotation gates are at worst $\theta/8$: we are not working with exponentially small angles, as discussed in Sec 4.4.5. But again, this assumes that we can afford to use auxiliary registers.

4.4.8 Discussion We presented two concrete algorithms to synthesize C^n-R_θ gates: these are of course not the only existing techniques. However, the main idea remains: there is no silver bullet, and *tradeoffs* have to be decided upon, depending both on the backend and on the targetted use-case. In particular, one can note the following three tradeoffs

1. As discussed above, the reduction of the circuit size is typically correlated with an increase in the pool of ancillas.
2. The more compact the circuit is, the more *time* it takes to synthesize it. If one needs a steady stream of circuit generation on the fly, one might prefer a technique producing slightly larger circuits but in a fast manner rather than a slower technique producing more optimized circuits.
3. Informations on the *structure* of the unitary to synthesize might be capitalized upon to produce a smaller circuit. An example of such as case is discussed in 5.2.1 with the generation of *oracles*.

4.5 Quantum Computation with Magic States

4.5.1 If the low-level target is error corrected —such as with the *surface code* [Cle22]— non-Clifford gates might not be directly implementable at the logical level. If non-Clifford such as T -gates are not available, a solution consists in initializing the memory in a state that contains the phases that the rotations would have brought. Rotation gates are then not native, but instead built from these *magic states* [BK05], Clifford gates, and a bit of classical processing. Such a rotation gate is then way more costly than Clifford gates, requiring dedicated error correction and optimization scheme.

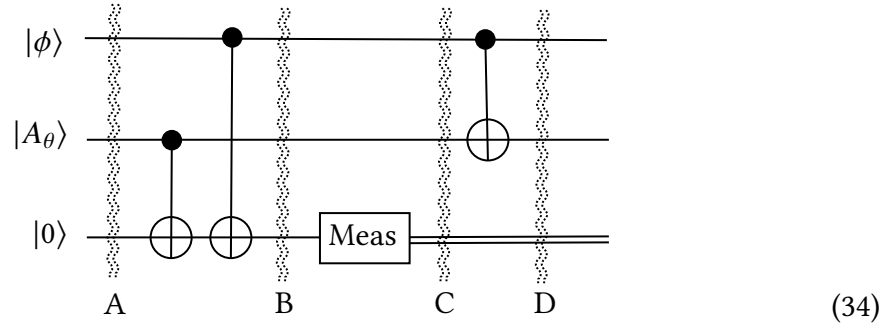
4.5.2 A typical magic state is

$$|A_\theta\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle).$$

If we have access to several copies of the magic states, it is possible to realize the rotation

$$R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}.$$

The procedure is summarized by the circuit:



Suppose that $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$. At each step, the state of the system is:

(A) $|\phi\rangle \otimes |A_\theta\rangle \otimes |0\rangle$, that is

$$\frac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha e^{i\theta}|010\rangle + \beta|100\rangle + \beta e^{i\theta}|110\rangle).$$

(B) The two CNOT-gates store in qubit 3 the parity respective of qubit 1 and 2: 0 for 00 and 11, and 1 for 01 and 10. The state becomes

$$\frac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha e^{i\theta}|011\rangle + \beta|101\rangle + \beta e^{i\theta}|110\rangle).$$

(C) The measurement gives with probability $\frac{1}{2}$ the states

$$(\alpha|00\rangle + \beta e^{i\theta}|11\rangle) \otimes |0\rangle$$

when 0 is measured, and

$$(\alpha e^{i\theta}|01\rangle + \beta|10\rangle) \otimes |1\rangle$$

when 1 is measured.

(D) The last CNOT-gate will unentangle the state of qubit 1 and 2: if we had measured 0 we would get

$$(\alpha|0\rangle + \beta e^{i\theta}|1\rangle) \otimes |0\rangle \otimes |0\rangle$$

and if we had measured 1 we would get

$$(\alpha e^{i\theta}|0\rangle + \beta|1\rangle) \otimes |1\rangle \otimes |1\rangle$$

that can be rewritten (by change of global phase) into

$$(\alpha|0\rangle + \beta e^{-i\theta}|1\rangle) \otimes |1\rangle \otimes |1\rangle.$$

The two last qubits are now in a canonical basis state: they can be recycled and reused without impacting the global state of the memory.

The effect of the circuit is then

$$(\alpha |0\rangle + \beta |1\rangle) \otimes |A_\theta\rangle \otimes |0\rangle \mapsto (\alpha |0\rangle + \beta e^{\pm i\theta} |1\rangle) \otimes |1\rangle \otimes |1\rangle,$$

with the sign depending on the result of the measurement.

4.5.3 Because the sign measurement is probabilistic, if we repeat the process (each time with a fresh magic state $|A_\theta\rangle$), we will get roughly the same number of 0 and 1 out of the measurements —let us respectively denote these numbers with n_0 and n_1 . The state is in this case:

$$(\alpha |0\rangle + \beta e^{(n_0 - n_1)i\theta} |1\rangle).$$

Because of the statistic of the process, we will eventually meet a point in time where $n_0 = 1 + n_1$: the state of qubit 1 then corresponds to the application of the gate R_θ to $|\phi\rangle$. We can stop there: we implemented the gate R_θ .

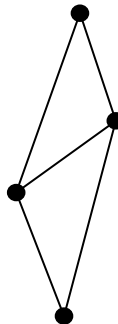
4.5.4 Of course, in general, the number of times needed to repeat the circuit is not known: we have a *repeat-until-success* scenario. In order for this scheme to work, the result of the measurement should not have to be sent to the classical host. Indeed, the delay would be way too costly. In this setting, the co-processor therefore needs to have *some* form of classical control: here, the possibility to perform some simple arithmetics, and the possibility to repeat a piece of circuit.

4.5.5 The magic state presented in Sec. 4.5.2 is a very simple one. There are many other proposals, possibly with simpler computational schemes. All of these schemes are however relying on the same underlying technique: *repeat-until-success*. See for instance [BK05].

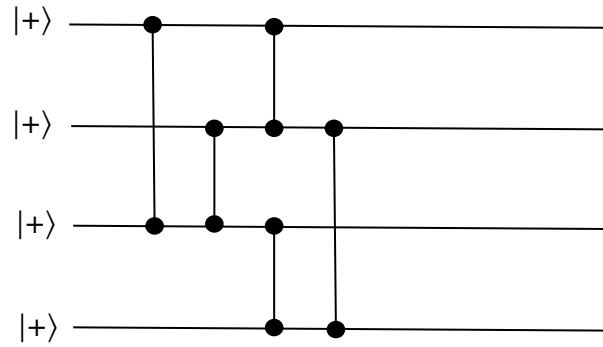
4.6 Measurement-Based Quantum Computation

4.6.1 In some situation, the problem is not with single-qubit rotations but with 2-qubit, entangling gates such as CNOT or C -Z. An example of such a situation is discussed for the case of quantum linear optics in 4.9. In this situation, similarly to what has been done in 4.5, we can possibly rely on something akin to magic states. In this situation, we talk instead of *graph states*.

4.6.2 A *graph state* is a set of entangled qubits according to an (undirected) graph: each node corresponds to a qubit initialized in state $|+\rangle$, and each edge corresponds to a C -Z between the two corresponding nodes. For instance, the graph



corresponds to the state generated with the circuit



4.6.3 Provided that we have access to graph states, measurements in the basis

[3.8.17](#)

$$|+\theta\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle)$$

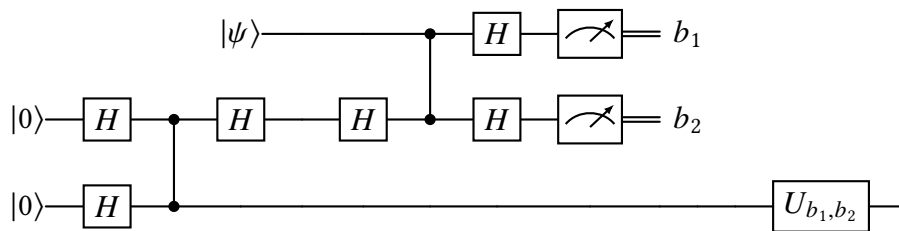
$$|-\theta\rangle = \frac{1}{\sqrt{2}}(|0\rangle - e^{i\theta}|1\rangle),$$

Pauli gates, general unitary operations can be realized. The computational model is called *MBQC*: Measurement-based Quantum Computation. The Pauli gates are *correction*, parameterized by measurement results as in the Teleportation Algorithm.

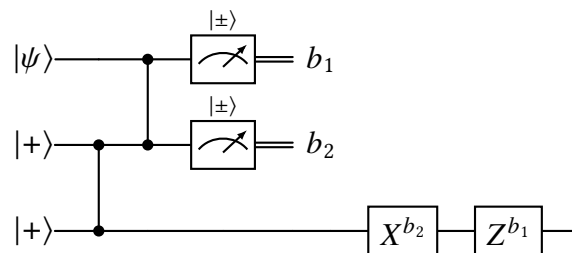
[3.10.3](#)

4.6.4 To understand how MBQC works, let us rewrite the Teleportation Algorithm using graph states. Each CNOT in the circuit drawn in Figure 6 can be rewritten to obtain the circuit

[3.11.4](#)

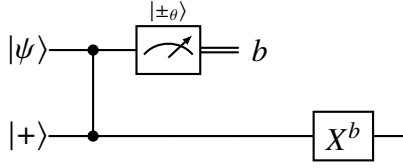


Tidying up, and unfolding the definition of U_{b_1, b_2} , we get



where the annotation $|\pm\rangle$ on the measurement gate means that the measure should happen in basis $|+\rangle$, $|-\rangle$, and the notation G^b means that the gate G should be applied whenever b is 1.

4.6.5 The scheme in 4.6.4 is a seemingly sophisticated MBQC *pattern* computing... the identity. One can follow the same idea to build a pattern for a non-trivial operation, as follows.



The measurement is defined in the basis $|+\theta\rangle, |-\theta\rangle$ of 4.6.3. The bit b is 1 whenever $|+\theta\rangle$ was measured.

If $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, the circuit computes the following operation:

$$\begin{aligned}
 |\psi\rangle \otimes |+\rangle &= \frac{1}{\sqrt{2}}(\alpha |0\rangle + \beta |1\rangle) \otimes (|0\rangle + |1\rangle) \\
 &= \frac{1}{\sqrt{2}}(\alpha |00\rangle + \alpha |01\rangle + \beta |10\rangle + \beta |11\rangle) \\
 &\xrightarrow{C-Z} \frac{1}{\sqrt{2}}(\alpha |00\rangle + \alpha |01\rangle + \beta |10\rangle - \beta |11\rangle) \\
 &= \frac{1}{\sqrt{2}}(\alpha |00\rangle + \alpha |01\rangle + \beta |10\rangle - \beta |11\rangle) \\
 &= \frac{1}{\sqrt{2}}(\alpha |0\rangle \otimes |0\rangle + \alpha |0\rangle \otimes |1\rangle + \beta |1\rangle \otimes |0\rangle - \beta |1\rangle \otimes |1\rangle) \\
 &= \frac{1}{2} \left(\begin{array}{l} \alpha(|+\theta\rangle + |-\theta\rangle) \otimes |0\rangle + \alpha(|+\theta\rangle + |-\theta\rangle) \otimes |1\rangle \\ + \beta e^{-i\theta}(|+\theta\rangle - |-\theta\rangle) \otimes |0\rangle - \beta e^{-i\theta}(|+\theta\rangle - |-\theta\rangle) \otimes |1\rangle \end{array} \right) \\
 &= \frac{1}{2} \left(\begin{array}{l} |+\theta\rangle \otimes ((\alpha + \beta e^{-i\theta}) |0\rangle + (\alpha - \beta e^{-i\theta}) |1\rangle) \\ + |-\theta\rangle \otimes ((\alpha - \beta e^{-i\theta}) |0\rangle + (\alpha + \beta e^{-i\theta}) |1\rangle) \end{array} \right)
 \end{aligned}$$

Measuring in basis $|+\theta\rangle, |-\theta\rangle$, we get

- either $(\alpha + \beta e^{-i\theta}) |0\rangle + (\alpha - \beta e^{-i\theta}) |1\rangle$ when the result of measurement is 0,
- or $(\alpha - \beta e^{-i\theta}) |0\rangle + (\alpha + \beta e^{-i\theta}) |1\rangle$ when the result of the measurement is 1.

In the latter case, the Pauli gate X is used as a correction: we deterministically get

$$(\alpha + \beta e^{-i\theta}) |0\rangle + (\alpha - \beta e^{-i\theta}) |1\rangle$$

at the end of the computation. The net effect of the pattern is then the unitary operation

$$J(-\theta) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & e^{-i\theta} \\ 1 & -e^{-i\theta} \end{pmatrix}.$$

As discussed in 4.2.3, the gate $J(\theta)$ and the gate $C-Z$ form a universal gate-set: we can realize any unitary operation with measurements, Pauli corrections and $C-Z$.

4.6.6 The language of MBQC can be described by the following instructions: $E_{i,j}$, M_i^α , ${}^t[M_i^\alpha]^s$, X_i^s and Z_i^s . The indices i and j corresponds to wire (or qubit) identifiers, and s and t are *signals*: boolean expressions built from the results of (previous) measurements. By convention, we write s_i for the result of the measurement of wire i . With these definitions, the instructions stand for

- $E_{i,j}$: a gate C - Z on qubits i and j ;
- M_i^α : measurement of qubit i in basis $|\pm_\alpha\rangle$;
- More generally ${}^t[M_i^\alpha]^s$, standing for $M_i^{(-1)^s\alpha+t\pi}$;
- X_i^s and Z_i^s : the corresponding Pauli gates on qubit i whenever s is 1.

Finally, a *program*, or *pattern*, is given by the tuple

$$(I, O, V, L)$$

where V is a set of qubits, $I \subseteq V$ is the set of *inputs* and $O \subseteq V$ the set of *outputs*. The sets I and O can intersect. The last element of the tuple is L : a list of instructions, read by convention from right to left. Running a program consists in

1. initializing the qubits in I with the input state,
2. initializing the qubits in $V \setminus I$ in state $|+\rangle$,
3. applying the instruction one at a time, from right to left, following the values of the *signals*,
4. the output of the computation is read in O .

4.6.7 Numbering the wires from top to bottom, the teleportation pattern in 4.6.4 can be written as the MBQC program

$$(\{1\}, \{3\}, \{1, 2, 3\}, Z_3^{s_1} X_3^{s_2} M_1^0 M_2^0 E_{1,2} E_{2,3}).$$

This pattern simply teleport the state of wire 1 to wire 3. The pattern 4.6.5 is the program

$$(\{1\}, \{2\}, \{1, 2\}, X_2^{s_1} M_1^\theta E_{1,2}).$$

If wire 1 is originally in state $|\psi\rangle$, the state of the wire 3 is set to $J(-\theta) |\psi\rangle$ at the end of the computation. In particular, realizing the Hadamard gate can be done with $\theta = 0$. As a last example, the (very simple) pattern

$$(\{1, 2\}, \{1, 2\}, \{1, 2\}, E_{1,2})$$

realizes... a C - Z : inputs and outputs are the same wires 1 and 2, and the action of the pattern is a single C - Z gate.

4.6.8 Not all sequences of instructions are valid. First, a measured wire cannot be reused later on. For instance, $E_{0,1} M_0^\theta$ is not valid (remember that instructions are read from right to left): once wire 0 has been measured, one cannot apply a gate to it. Another constraint is that a signal can only be used once emitted, so for instance $M_0^\theta X_2^{s_0}$ is not valid: the signal s_0 used in $X_2^{s_0}$ is only emitted later on, at the next instruction M_0^θ .

4.6.9 Note that any (valid) MBQC pattern corresponds to a valid sequence of quantum operations. However, because a pattern can contain measurements, it might or might not represent a purely unitary operation.

4.6.10 A pattern in MBQC has an input and an output: they can be composed in a natural way, remarking that the wire identifiers can be changed without modifying the overall action of the pattern:

$$(\{1, 2\}, \{1, 2\}, \{1, 2\}, E_{1,2}) \quad \text{and} \quad (\{5, 6\}, \{5, 6\}, \{5, 6\}, E_{5,6})$$

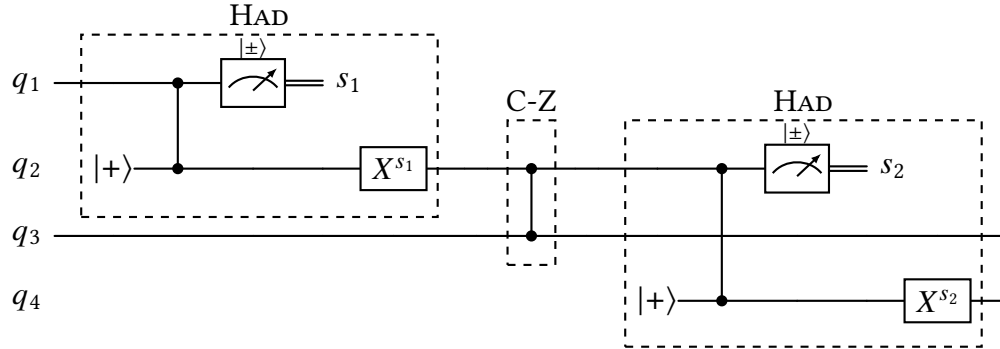
both perform a $C\text{-}Z$, the difference being that they do not act on the same set of wires. This makes it possible to *compose* patterns in order to emulate the behavior of quantum circuits, by “plugging” outputs to inputs. For instance composing patterns for Hadamard, then $C\text{-}Z$, then Hadamard yields a pattern for CNOT:

$$(\{2\}, \{4\}, \{2, 4\}, X_4^{s_2} M_2^\theta E_{2,4}) \circ (\{2, 3\}, \{2, 3\}, \{2, 3\}, E_{2,3}) \circ (\{1\}, \{2\}, \{1, 2\}, X_2^{s_1} M_1^\theta E_{1,2})$$

corresponds to a circuit whose overall input is $\{1, 3\}$ and overall output is $\{3, 4\}$, where qubit 1 is now qubit 4. The pattern is

$$(\{1, 3\}, \{3, 4\}, \{1, 2, 3, 4\}, X_4^{s_2} M_2^\theta E_{2,4} E_{2,3} X_2^{s_1} M_1^\theta E_{1,2})$$

and the corresponding circuit-like representation is



4.6.11 Equivalent Representation. It is not immediate to see that the pattern for CNOT in 4.6.10 is formed from a graph state: Instructions $E_{i,j}$ appear mixed with measurements and Pauli corrections. In order to recover a graph state, one has to realize that $C\text{-}Z$ and Pauli corrections commutes. From 3.11.5, we can derive the following equalities:

$$\begin{aligned} E_{i,j} X_i^s &= X_i^s Z_j^s E_{i,j}, & E_{i,j} Z_i^s &= Z_i^s E_{i,j}, \\ E_{i,j} X_j^s &= X_j^s Z_i^s E_{i,j}, & E_{i,j} Z_j^s &= Z_j^s E_{i,j}. \end{aligned}$$

We can also show that

$${}^t[M_i^\theta]^s X_i^r = {}^t[M_i^\theta]^{s+r}, \quad {}^t[M_i^\theta]^s Z_i^r = {}^{t+r}[M_i^\theta]^s.$$

Intuitively, an X gate corresponds to a π rotation and an Z gate a phase flip. Finally, instructions acting on distinct wires commute, assuming that this does not break dependencies on signals (as discussed in 4.6.8). For instance, $X_j^{s_i} M_i^\theta$ is a valid pattern, but although the measurement and the Pauli gate do not act on the same wires, they do not commute since the action of X depends on the result of the measurement. And in fact, the pattern $M_i^\theta X_j^{s_i}$ is not valid.

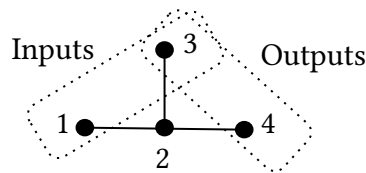
4.6.12 Normal Form. Using the equivalence rules in 4.6.11, it is possible to show⁸ that the instructions of any MBQC pattern can be factorized as follows.

- First, perform a series of entangling gates $E_{i,j}$ to generate a *graph state*;
- then, measure in some specific wire ordering, with angles that possibly changes according to previous measurements;
- finally, apply (parameterized) X and Z Pauli corrections.

For instance, the series of instructions in the pattern for CNOT discussed in 4.6.10 can be factorized as

$$X_4^{s_2} Z_4^{s_1} Z_3^{s_1} \quad {}^0[M_2^0]^{s_1} M_1^\theta \quad E_{2,4} E_{2,3} E_{1,2}.$$

The corresponding graph is



4.7 Classical Computation in the Co-Processor

4.7.1 In order to implement the techniques presented in 4.5, the circuit model presented in 3.4.7 has to be extended with classical registers, and operations to act on them. In this extended model, the bit generated from a measurement gets stored in a classical register, within the quantum co-processor. If the classical host needs the information, this bit needs to be extracted from the co-processor.

4.7.2 A quantum programming framework offering such a model is QISKIT, a PYTHON library developed by IBM⁹. An example of circuit mixing both quantum and classical control is as follows.

```

1 # 2 qubits for the Bell pair
2 epr = QuantumRegister(2, name="epr")
3
4 # The qubit to teleport
5 phi = QuantumRegister(1, name="phi")
6
7 # Classical register for storing the results
8 c = ClassicalRegister(2, name="c")
9
10 # We build a quantum circuit with both registers.
11 # By default, everything is initialized to 0 and to |0>
12 qc = QuantumCircuit(phi, epr, c)
13
14 # Generating the Bell pair

```

⁸See the very pedagogical presentation in [DKPP09b].

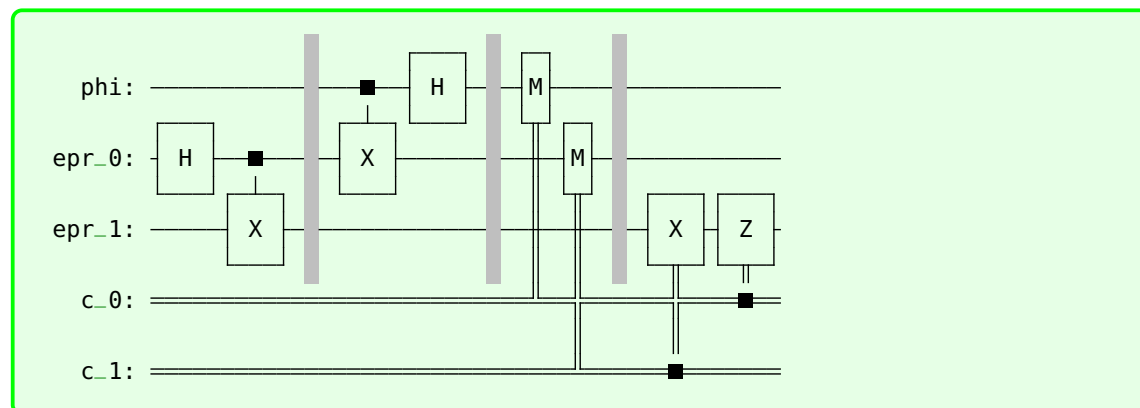
⁹<https://qiskit.org/>

```

15 qc.h(epr[0])
16 qc.cnot(epr[0],epr[1])
17 qc.barrier()
18
19 # Action of Alice
20 qc.cnot(phi, epr[0])
21 qc.h(phi)
22 qc.barrier()
23
24 # Measure of all of register q, storing results in c.
25 # This is still part of the circuit
26 qc.measure(phi, c[0])
27 qc.measure(epr[0], c[1])
28 qc.barrier()
29
30 # Now, action of Bob
31 qc.x(epr[1]).c_if(c[1],1)
32 qc.z(epr[1]).c_if(c[0],1)

```

For legibility of the printed circuit, we added “barriers” to regroup pieces of circuits together. The resulting circuit, as shown by QisKit in ASCII art is as follows. At the end of the circuit, the state of the qubit `phi` is in `epr_1`.



4.7.3 A question is the classical power of the quantum co-processor: what are the operations allowed on classical registers? We at least need to be able to apply quantum gates conditionally on the value of a Boolean register. To realize the scheme presented in Sec. 4.5, we also need (limited) arithmetic but also loops. For representing such a process, the circuit notation starts to show its limits, and if this were a more expressive language might be required. More generally, this question of where to place the limit between the classical and the quantum host is at the core of the discussions on hybrid computation.

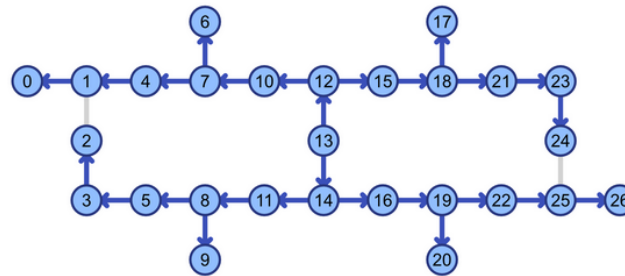
4.8 A Word on Hardware

4.8.1 Noise As discussed in 3.2.3, quantum information is encoded on the state of objects governed by the laws of quantum physics. Such objects are subject to a physical phenomenon called *decoherence*: due to an interaction with the environment, the state

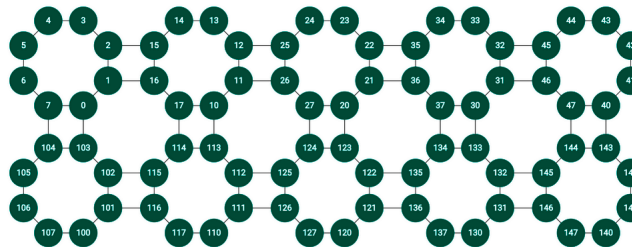
of the system evolves in uncontrolled ways. From a computer science point of view, this amounts to *noise* and the introduction of *errors* in the stored information.

4.8.2 NISQ and LSQ Noisy hardware is the current state of quantum co-processors. As of 2024, quantum memory holds at most a few hundred of noisy qubits: we are in the realm of *NISQ*, *Noisy Intermediate Scale Quantum Computation*. If we are arguably reaching the tipping point, the error rate and the (relatively) small memory size makes it still impossible to run quantum error correcting schemes. Building (logical) stable qubits for enabling large memory (dubbed *LSQ*: *Large Scale Quantum*) is still an active subject of research, both in academic and industrial labs.

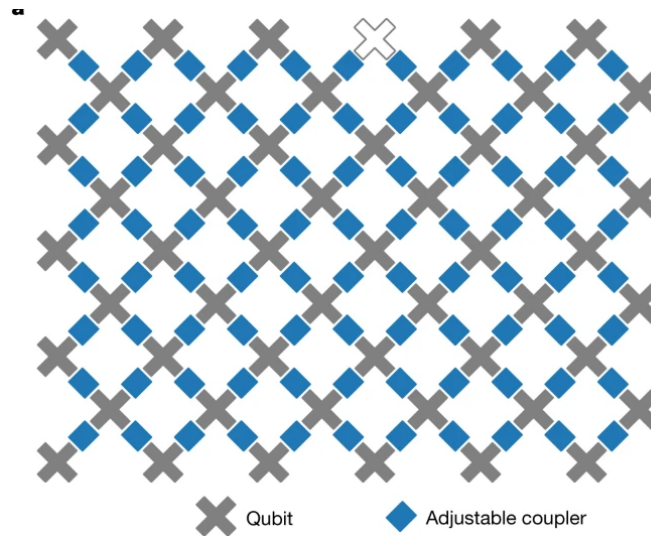
4.8.3 Topology For hardware (or some fault-tolerant scheme, such as surface code), the quantum bits have a fixed location. In these cases, two-qubits operations can only be performed on neighboring qubits: the notion of neighbors is described with a graph. We say that there is a *topology*. For instance, the IBM co-processor “Montreal” has 26 qubits arranged as follows:



The Rigetti co-processor “Aspen-M” is as follows:



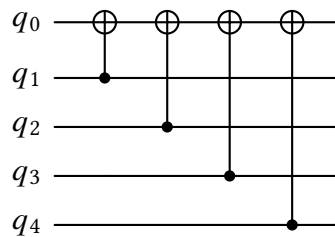
In these two cases, the circles are the qubit locations, while the edges are where CNOT-gates are allowed. Another example using the same technology is the Google coprocessor “sycamore”, with one non-functioning qubit (the white one):



Here, qubits are crosses, and the blue links are where 2-qubit operations are allowed.

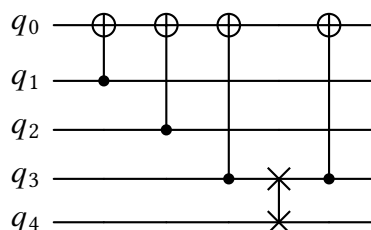
4.8.4 The problem this poses is the *mapping*, or *routing* of qubits. That is: given a (logical) circuits, to which physical qubit do we associate each wire? This question turns out to be an NP-complete problem: the *subgraph isomorphism problem*. In general not all circuits can be mapped on any given topology, and when not feasible the goal is to rewrite the circuit into an equivalent one such that a mapping exists: this turns the decision problem into an optimization problem.

4.8.5 For instance, consider the circuit



Wire q_0 needs to be mapped to a physical qubit of arity 4: if it is somewhat possible for Google Sycamore co-processor (modulo the fact that CNOT-gates are not native there), it is not possible to do it directly for IBM's Montreal and Rigetti's Aspen-M co-processor, as the largest possible arity is 3. To implement such a circuit, a solution consists in rewriting the circuit with swaps (themselves implementable with CNOT-gates) to reduce the arity of problematic gates.

4.8.6 The circuit presented in Sec. 4.8.5 can be rewritten as



with a swap between q_3 and q_4 . We can now find a routing for e.g. IBM Montreal:

$$q_0 \mapsto 12, \quad q_1 \mapsto 15, \quad q_2 \mapsto 13, \quad q_3 \mapsto 10, \quad q_4 \mapsto 7.$$

At the end of the computation, the wires are found in the following locations:

$$q_0 \mapsto 12, \quad q_1 \mapsto 15, \quad q_2 \mapsto 13, \quad q_3 \mapsto 7, \quad q_4 \mapsto 10.$$

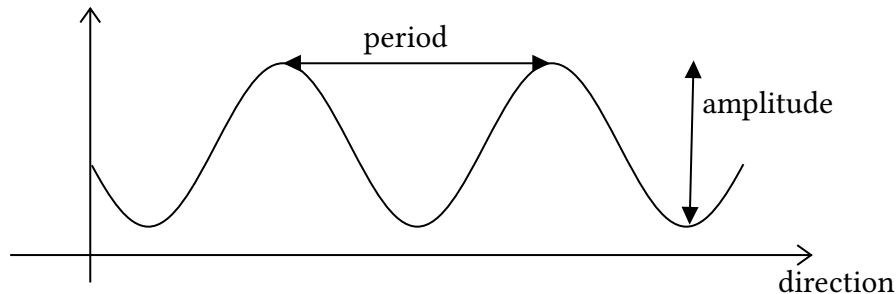
We can perform another swap if needed to place back q_3 and q_4 at their original locations if needed.

4.8.7 In general, finding an optimal *mapping* or *routing* for a given circuit is an NP-complete problem: for non-trivial circuits we have to rely on approximate solutions. Existing compilers such as T|KET from Quantinuum, QISKIT from IBM or MYQLM from Eviden implement state-of-the-art techniques for the mapping problem. Such tools can also handle additional constraints such as specific costs to minimize.

4.9 Focus: Linear Optical Quantum Computation

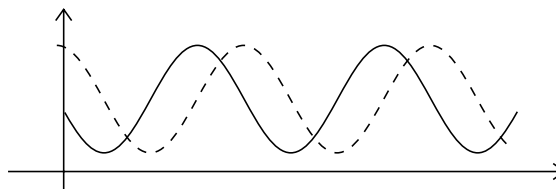
4.9.1 This section focuses on a hardware implementation maybe easier to apprehend than others: quantum linear optics. The idea consists in encoding information on the state of *photons*.

4.9.2 Notion of Phase. According to quantum mechanics principles that are out of the scope of this course, a photon is both a *particle* and a *wave*. A typical wave is a periodic phenomenon looking like the following.

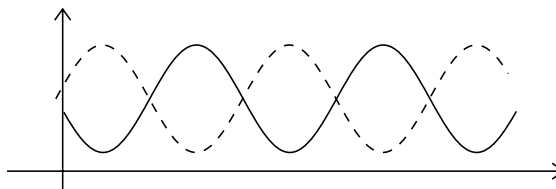


4.9.3 Relative Phase Two waves running in parallel might be shifted by a relative *phase*. In the following diagrams, the dashed wave is shifted from the solid wave with...

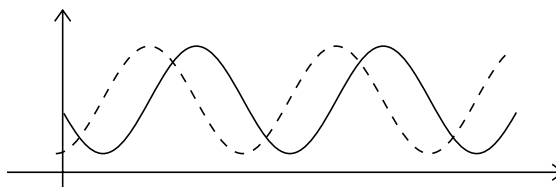
- an angle of $\pi/2$



- an angle of π

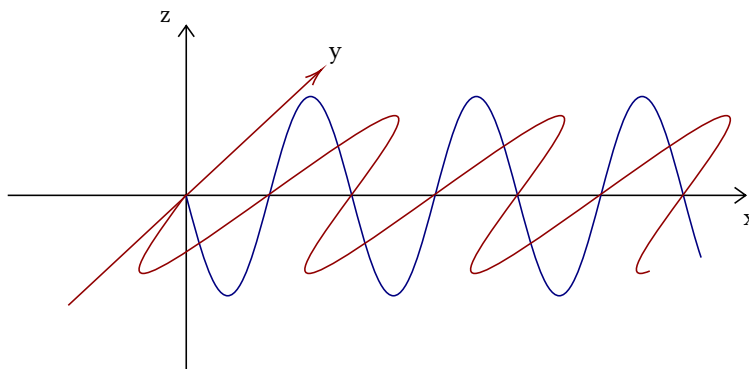


- an angle of $3\pi/2$



4.9.4 Global Phase In the context of 4.9.3, a *global phase* would here correspond to an horizontal shift of both waves at once. When we say that “global phase does not matter” in quantum computation, this can be understood as: there is no canonical horizontal position where to measure the amplitude of the wave. The only meaningful information that can possibly be retrieved is a relative phase, or an amplitude.

4.9.5 Coherent Light. Formally, coherent light is described by coupled oscillating electric and magnetic fields: a photon corresponds to an *electromagnetic wave*. It is composed of two synchronous waves running through space on orthogonal planes. In the following diagram, the x-axis is the direction of the light-wave, composed of a red wave in the xy-plane and a blue wave in the xz-plane. Physically, one of them corresponds to the electric part of the wave (say: the blue wave) and the other the magnetic part (so: the red wave).



4.9.6 Polarization When the electric field (in 4.9.5: the blue wave) evolves according to a particular orientation, we say that the photon is *polarized*. Polarization is a two-dimensional quantum property (the two dimensions orthogonal to the direction of the wave): it can be vertical (denoted with $|V\rangle$), or horizontal (denoted with $|H\rangle$). In general, the state of the system is a superposition of both: the wave admits a vertical projection (on the z axis) and an horizontal projection (on the y axis), as shown in Figure 8. The phase of these two projections is the relative phase between the $|H\rangle$ and the $|V\rangle$ coordinate, and we retrieve the invariance under global phase.

3.2.8

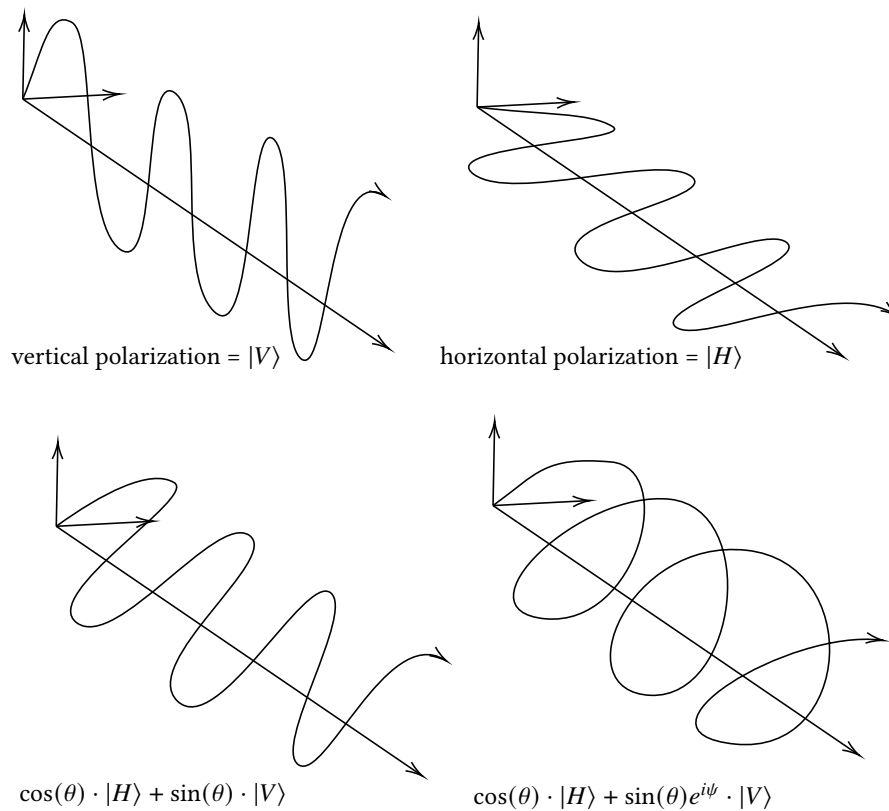


Figure 8: Encoding a qubit on the polarization of a photon.

4.9.7 Measuring Photon Properties For a photon, two properties can be easily measured. The first one is its existence at a particular location and at a particular time: this is done with a light detector. Depending on its sensitivity, such a detector might also be able to say how many photons were detected at once. The other easy property to measure is the polarization of a photon: this is done with a polarized glass followed with a light detector. The polarized glass only let through photon polarized in one particular orientation: depending on whether a photon was detected after the polarized medium, we can deduce the polarization of the photon.

Apart from polarization and position, it is possible to consider other properties, such as for instance crossing time (is the photon there at time t ?), frequency (i.e. color), etc. Polarization and position are however the easiest to manipulate, thanks to the already existing linear optical devices (some of them described in [4.9.9](#) and [4.9.11](#)).

4.9.8 Mathematical Formalism Compared to a qubit which is already a logical construction, a photon is a physical object: we can consider only two distinct states, such as its vertical and horizontal properties, but we can also be more general. For instance, given two optical fibers (or *modes*) A and B , a photon might be in four distinct states:

- horizontal polarization in mode A ,
- vertical polarization in mode A ,
- horizontal polarization in mode B ,

- vertical polarization in mode B .

4.9.9 Linear Optical Circuit In linear optical quantum computation (LOQC), the equivalent of a quantum circuit is an *optical circuit*, built from horizontal wires and gates. The wires are called *modes*, and they corresponds to optical fibers or optical paths in free space in which photons evolve. Gates are *linear optical components*: typically beamsplitters, polarizing beamsplitters, phase shifters and wave plates (and permutation). One can also consider non-linear components such as *sources* and *detectors*. A source introduces a fixed number of photons in the circuit, and a detector acts a measure: it detect incoming photons.

4.9.10 Encoding Information Encoding a quantum bit requires a medium and two orthogonal states. For photons, the natural choice involves *mode* and/or *polarization*. For instance, consider the setting of 4.9.8 with a single photon. Its state space is generated by

$$|H_A\rangle, |H_B\rangle, |V_A\rangle, |V_B\rangle.$$

where each basis ket corresponds to a particular pair of properties. For instance $|H_A\rangle$ means “the photon lies in wire A and is horizontally polarized”. One can encode a two-qubit system with the following strategy:

- qubit 1 uses the polarization of the photon: 0, 1 is for instance respectively H, V .
- qubit 2 uses the mode of the photon: 0, 1 is for instance respectively A, B .

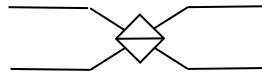
The state $|01\rangle$ is then represented with $|H_B\rangle$: a horizontally polarized photon in wire B . The Bell state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is $\frac{1}{\sqrt{2}}(|H_A\rangle + |V_B\rangle)$: a photon which is in superposition: horizontally polarized in wire A , and vertically polarized in wire B .

4.9.11 Action of Linear Optical Components The linear optical components mentioned in 4.9.9 act on polarization or on mode. Mathematically, the corresponding operation is a linear, unitary map on the state space of the photon.

4.9.12 Polarizing beamsplitters As their name implies, they act on polarization. They work on 2 wires, say A and B . When horizontally polarized the photon stays on the same mode and it changes mode when vertically polarized. Formally:

$$|H_A\rangle \mapsto |H_A\rangle, |H_B\rangle \mapsto |H_B\rangle, |V_A\rangle \mapsto |V_B\rangle, |V_B\rangle \mapsto |V_A\rangle.$$

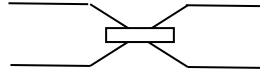
The graphical representation is



4.9.13 Beamsplitters Work on 2 wires. Parameterized by an angle θ , they only act on modes, not touching the polarization of the photon. Their action corresponds to a rotation in the two-dimensional space of modes $|A\rangle, |B\rangle$. Several conventions exist. When the basis is ordered as $|A\rangle, |B\rangle$, the action can be HAD-based, R_x -based or R_y -based, as follows.

$$\begin{array}{ccc} \begin{pmatrix} \cos(\theta/2) & \sin(\theta/2) \\ \sin(\theta/2) & -\cos(\theta/2) \end{pmatrix} & \begin{pmatrix} \cos(\theta/2) & i \sin(\theta/2) \\ i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} & \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \\ \text{HAD-based} & R_x\text{-based} & R_y\text{-based} \end{array}$$

The graphical representation is



For non-symmetric gates, it is important to specify which wire corresponds to A and which wire corresponds to B .

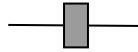
4.9.14 Wave plates Acting on a single wire, they perform a (2×2) unitary operation in the polarization space. For instance, a wave plate might perform a flip

$$|H\rangle \mapsto |V\rangle, \quad |V\rangle \mapsto |H\rangle,$$

or a Hadamard-like operation

$$|H\rangle \mapsto \frac{1}{\sqrt{2}}(|H\rangle + |V\rangle), \quad |V\rangle \mapsto \frac{1}{\sqrt{2}}(|H\rangle - |V\rangle).$$

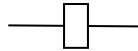
The representation is a greyed box as follows:



4.9.15 Phase shifters Acting on a single wire, they perform a global phase change on the photon. In the context of 4.9.10, a phase shift on A performs the action

$$|H_A\rangle \mapsto e^{i\theta} |H_A\rangle, |V_A\rangle \mapsto e^{i\theta} |V_A\rangle, |H_B\rangle \mapsto |H_B\rangle, |V_B\rangle \mapsto |V_B\rangle.$$

Note how it only changes the phase of the states corresponding to mode A . The representation is a white box as follows:

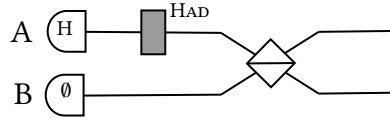


4.9.16 Permutation This operation corresponds literally to a permutation of wires. The states are changed accordingly. For instance, in the situation of 4.9.10, the permutation of A and B corresponds to the map

$$|H_A\rangle \mapsto |H_B\rangle, |V_A\rangle \mapsto |V_B\rangle, |H_B\rangle \mapsto |H_A\rangle, |V_B\rangle \mapsto |V_A\rangle.$$

It only modifies the mode of photons. The graphical representation is simply wire crossing.

4.9.17 Example In the encoding of 4.9.10, the following linear optical circuit generates the Bell-state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$:



A source generates a photon horizontally polarized while B does not emit anything. The photon then goes through the Hadamard wave plate of 4.9.14: its state is then

$$\frac{1}{\sqrt{2}}(|H_A\rangle + |V_A\rangle).$$

It finally goes through a polarizing beamsplitter, yielding the state

$$\frac{1}{\sqrt{2}}(|H_A\rangle + |V_B\rangle)$$

as only vertically polarized photons have their mode changed.

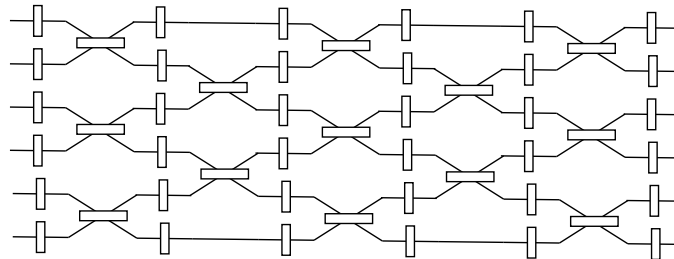
4.9.18 Mode versus Polarization When encoding only one qubit, one can choose either:

- Polarization: unitaries are realized with *wave plates*; or
- Two modes A and B : unitaries are realized with beamsplitters and phase shifters.

Polarizing beamsplitters can be regarded as a way to go from one encoding to the other.

4.9.19 Mode-Only Encoding The setting of 4.9.10 cannot easily be extended: going to 3 qubits would require to find more orthogonal properties. The easiest is to add more modes. From 4.9.18 we can even drop polarization and only focus on modes. When n wires are available, the state of a single photon is a vector in an n dimensional Hilbert space $\mathcal{K}$ described by (say) $|w_1\rangle, \dots, |w_n\rangle$.

A linear optical circuit consisting of beamsplitters and phase shifters on these n wires corresponds to a unitary operation on the Hilbert space $\mathcal{K}$. Such circuits are *universal* in the sense that every unitary matrix can be realized with a linear optical circuit. An example of shape on 6 wires is as follows:



4.9.20 According to 4.9.19, one can emulate a quantum circuit on n qubits using a linear optical circuit with 2^n wires and a single photon. For instance, 2 qubits can be encoded as follows:

- photon on wire w_1 : state is $|00\rangle$,
- photon on wire w_2 : state is $|01\rangle$,
- photon on wire w_3 : state is $|10\rangle$,
- photon on wire w_4 : state is $|11\rangle$.

A CNOT-gate where the controlled qubit is the first one is a permutation of w_2 and w_4 . If instead the controlled qubit is the second qubit, this can be done instead with permuting w_3 and w_4 .

4.9.21 Many photons With a single photon, a mode-only encoding is hardly scalable: we need an exponential number of wires on the number of qubits. A simple generalization is to consider several photons. In fact, in general, on a wire one might consider zero photon, a single one, or several of them. For quantum computation, we need photons to be *indistinguishable* so that they can interfere.

4.9.22 For simplicity, suppose that we have 4 modes. In a many-photon system, each wire might hold an arbitrary number of photons. Because these photons are indistinguishable, the only meaningful information to retain is the *photon number* of the corresponding wire. We use the notation

$$|n_1, n_2, n_3, n_4\rangle$$

for a many-photon state consisting of n_1 photons in mode 1, n_2 photons in mode 2, n_3 photons in mode 3 and n_4 photons in mode 4. Each n can be any non-negative number: the state $|0, 1, 0, 0\rangle$ stands for instance for a single photon in mode 2.

4.9.23 The notation of 4.9.22 describes the basis elements of an infinite dimensional Hilbert space called a *Fock space*. One can for instance consider the state

$$\frac{1}{\sqrt{2}}(|2, 0, 0, 0\rangle + |0, 0, 0, 2\rangle),$$

representing the state where 2 photons are in superposition in mode 1 and in mode 4.

4.9.24 Formally, a Fock space is a direct product of the form

$$\mathcal{F}(\mathcal{K}) = \mathbb{C} \oplus \mathcal{K} \oplus \mathcal{K}^{\otimes 2} \oplus \mathcal{K}^{\otimes 3} \oplus \dots$$

where $\mathcal{K}^{\otimes k}$ is the k th *symmetric tensor* of $\mathcal{K}$: the subspace of $\mathcal{K}^{\otimes k}$ where vectors are invariant under the permutation of the underlying tensors.

4.9.25 We can define a special bilinear operation to combine elements of the Fock space, keeping track of the “amount” of permuted elements that are in fact equal. Define the operation “ $\bullet$ ” as the commutative, associative and distributive operation such that

$$|1, 0, 0, 0\rangle^{\bullet n_1} \bullet |0, 1, 0, 0\rangle^{\bullet n_2} \bullet |0, 0, 1, 0\rangle^{\bullet n_3} \bullet |0, 0, 0, 1\rangle^{\bullet n_4} = \sqrt{n_1!n_2!n_3!n_4!} |n_1, n_2, n_3, n_4\rangle.$$

We can then derive

$$|0, n, 0, 0\rangle \bullet |0, m, 0, 0\rangle \triangleq \sqrt{\frac{(n+m)!}{n!m!}} |0, n+m, 0, 0\rangle,$$

where we “join” the photons that are in the same wire, and

$$|n, 0, 0, 0\rangle \bullet |0, m, 0, 0\rangle \triangleq |n, m, 0, 0\rangle,$$

where we merge the informations on distinct wires.

4.9.26 The standard presentation uses the notion of *creation operator*: instead of using $|n_1, \dots, n_k\rangle$ as primitive construction, one can use operators $\hat{w}_i^\dagger$, an operator whose behavior are (when $i = 1$):

$$\hat{w}_1^\dagger |n_1, n_2, n_3, n_4\rangle = \sqrt{n_1 + 1} |n_1 + 1, n_2, n_3, n_4\rangle.$$

All the discussion can then be done on this operator instead of the ket construction.

4.9.27 Linear Optical Component on Many Photons The action of a linear optical component on several photons can be derived (linearly) from the action on 1 single photon. To do so, we reuse the notation of 4.9.25. On 2 wires, if U is a unitary map on the space generated by $|w_1\rangle, |w_2\rangle$ as

$$U|w_1\rangle = \alpha|w_1\rangle + \beta|w_2\rangle, U|w_2\rangle = \gamma|w_1\rangle + \delta|w_2\rangle,$$

we can generalize the map U to the Fock space over these two wires by defining:

$$U|1, 0\rangle \triangleq \alpha|1, 0\rangle + \beta|0, 1\rangle, \quad U|0, 1\rangle \triangleq \gamma|1, 0\rangle + \delta|0, 1\rangle,$$

which is the simple translation of U on 1 wire, and then

$$U(|1, 0\rangle^{\bullet n} \bullet |0, 1\rangle^{\bullet m}) \triangleq (U|1, 0\rangle)^{\bullet m} \bullet (U|0, 1\rangle)^{\bullet n}.$$

for the general case, relying on the distributivity of the bullet operator.

4.9.28 For instance, consider a Hadamard beamsplitter U acting on two wires w_1 and w_2 . Its action is

$$|w_1\rangle \mapsto \frac{1}{\sqrt{2}}(|w_1\rangle + |w_2\rangle), \quad |w_2\rangle \mapsto \frac{1}{\sqrt{2}}(|w_1\rangle - |w_2\rangle).$$

Written with the Fock states $|1, 0\rangle$ and $|0, 1\rangle$, it is

$$|1, 0\rangle \mapsto \frac{1}{\sqrt{2}}(|1, 0\rangle + |0, 1\rangle), \quad |0, 1\rangle \mapsto \frac{1}{\sqrt{2}}(|1, 0\rangle - |0, 1\rangle).$$

On 2 photons, its action is on the subspace generated by $|2, 0\rangle$, $|1, 1\rangle$ and $|0, 2\rangle$. When the basis is ordered this way, its matrix is

$$\frac{1}{2} \begin{pmatrix} 1 & \sqrt{2} & 1 \\ \sqrt{2} & 0 & -\sqrt{2} \\ 1 & -\sqrt{2} & 1 \end{pmatrix}.$$

In particular:

$$\begin{aligned} U|1, 1\rangle &= U(|1, 0\rangle \bullet |0, 1\rangle) \\ &= (U|1, 0\rangle) \bullet (U|0, 1\rangle) \\ &= \frac{1}{2}(|1, 0\rangle + |0, 1\rangle) \bullet (|1, 0\rangle - |0, 1\rangle) \\ &= \frac{1}{2}(|1, 0\rangle \bullet |1, 0\rangle - |0, 1\rangle \bullet |1, 0\rangle + |1, 0\rangle \bullet |0, 1\rangle - |0, 1\rangle \bullet |0, 1\rangle) \\ &= \frac{1}{2}(\sqrt{2}|2, 0\rangle - |1, 1\rangle + |1, 1\rangle - \sqrt{2}|0, 2\rangle) \\ &= \frac{1}{2}(\sqrt{2}|2, 0\rangle - \sqrt{2}|0, 2\rangle). \\ &= \frac{1}{\sqrt{2}}(|2, 0\rangle - |0, 2\rangle). \end{aligned}$$

This effect is known as the *Hong-Ou-Mandel effect*: when two photons are sent through a Hadamard beamsplitter, one on each input wire, at the exit of the gate the two photons are both on the same wire: in superposition, they are both on the top wire and both on the bottom wire.

4.9.29 Dual Rail Encoding With many photons, a systematic approach for encoding qubits is called *dual rail*: each qubit corresponds to one photon over two possible modes. In the notation of 4.9.22: a logical $|0\rangle$ is represented with the photon state $|1, 0\rangle$ (one photon in first wire, zero in the second wire); a logical $|1\rangle$ is instead represented with $|0, 1\rangle$. For instance, a 3-qubit quantum circuit corresponds to a 6-mode linear optical circuit. The 8 possible logical qubit-states $|000\rangle$, $|001\rangle$, $|010\rangle$, $|011\rangle$, $|100\rangle$, $|101\rangle$, $|110\rangle$, $|111\rangle$ are represented by the 3-photons/6-modes states (written using the same order):

$$\begin{aligned} &|1, 0, 1, 0, 1, 0\rangle, |1, 0, 1, 0, 0, 1\rangle, |1, 0, 0, 1, 1, 0\rangle, |1, 0, 0, 1, 0, 1\rangle, \\ &|0, 1, 1, 0, 1, 0\rangle, |0, 1, 1, 0, 0, 1\rangle, |0, 1, 0, 1, 1, 0\rangle, |0, 1, 0, 1, 0, 1\rangle. \end{aligned}$$

Note that the Fock space contains many more states: they do not all correspond to a valid logical 3-qubits state. We have for instance

$$|3, 0, 0, 0, 0, 0\rangle, |2, 1, 0, 0, 0, 0\rangle, |0, 1, 0, 3, 0, 0\rangle, |1, 1, 0, 0, 1, 0\rangle, \text{ etc},$$

carrying no logical meaning.

4.9.30 Realizing one-qubit gates on dual rail is as easy as using phase gates and beam-splitters on the two wires corresponding to the target qubit. For two-qubit gates, this

turns out to be much harder, and sometimes impossible. Consider for instance the two-qubit gate CNOT. On dual rail, we need a 2-photons/4-modes linear optical circuit sending

$$\begin{aligned} |1, 0, 1, 0\rangle &\mapsto |1, 0, 1, 0\rangle, \\ |1, 0, 0, 1\rangle &\mapsto |1, 0, 0, 1\rangle, \\ |0, 1, 1, 0\rangle &\mapsto |0, 1, 0, 1\rangle, \\ |0, 1, 0, 1\rangle &\mapsto |0, 1, 1, 0\rangle. \end{aligned}$$

The second photon should then change its course depending on the state of the first photon. However, as we discussed in 4.9.27, with linear optical components the action on the two photons cannot be correlated.

4.9.31 In order to build 2-qubit gates in dual rails, we therefore have to rely on non-deterministic gates: gates succeeding *with some probability*. The idea consists in using auxiliary wires to extend the dimension of the state space, and consider the targetted unitary as a sub-part of a larger unitary. The general strategy consists in *post-selection*: at the end of the process, some or all of the wires are measured. The result of the measurement says whether the process succeeded or not. The gate is said *heralded* if one can decide on the success of the process by only measuring auxiliary wires.

4.9.32 An example of a post-selected 2-qubit gate on dual-rails is Ralph's CNOT encoding. The gate works with 6 modes, using 4 photons for 2 qubits and 2 auxiliary empty modes (thus initialized without any photon). Ralph's circuit implements the following unitary in the canonical basis $|w_1\rangle, |w_2\rangle, |w_3\rangle, |w_4\rangle, |w_5\rangle, |w_6\rangle$:

$$U = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 0 & 0 & 0 & \sqrt{2} & 0 \\ 0 & -1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ \sqrt{2} & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & -1 & 0 & -1 \end{pmatrix}.$$

We then focus on the 2-photons/6-modes state-space: it can be splitted into the *coincidence* subspace built from

$$|1, 0, 1, 0, 0, 0\rangle, \quad |1, 0, 0, 1, 0, 0\rangle, \quad |0, 1, 1, 0, 0, 0\rangle, \quad |0, 1, 0, 1, 0, 0\rangle,$$

corresponding to logical qubits and the remaining states:

$$\begin{aligned} &|2, 0, 0, 0, 0, 0\rangle, |0, 2, 0, 0, 0, 0\rangle, |0, 0, 2, 0, 0, 0\rangle, |0, 0, 0, 2, 0, 0\rangle, \\ &|1, 0, 0, 0, 0, 1\rangle, |0, 1, 0, 0, 0, 1\rangle, |0, 0, 1, 0, 0, 1\rangle, |0, 0, 0, 1, 0, 1\rangle, \\ &|1, 0, 0, 0, 1, 0\rangle, |0, 1, 0, 0, 1, 0\rangle, |0, 0, 1, 0, 1, 0\rangle, |0, 0, 0, 1, 1, 0\rangle, \\ &|1, 1, 0, 0, 0, 0\rangle, |0, 0, 1, 1, 0, 0\rangle, |0, 0, 0, 0, 1, 1\rangle, |0, 0, 0, 0, 2, 0\rangle, |0, 0, 0, 0, 0, 2\rangle, \end{aligned}$$

that do not represent any computation. On the coincidence subspace, the unitary U performs a logical CNOT with probability $\frac{1}{9}$. In other words, it sends a normalized vector

$$\alpha |1, 0, 1, 0, 0, 0\rangle + \beta |1, 0, 0, 1, 0, 0\rangle + \gamma |0, 1, 1, 0, 0, 0\rangle + \delta |0, 1, 0, 1, 0, 0\rangle$$

of the coincidence subspace to

$$\frac{\alpha}{3} |1, 0, 1, 0, 0, 0\rangle + \frac{\beta}{3} |1, 0, 0, 1, 0, 0\rangle + \frac{\gamma}{3} |0, 1, 0, 1, 0, 0\rangle + \frac{\delta}{3} |0, 1, 1, 0, 0, 0\rangle + \frac{\sqrt{8}}{3} |\psi\rangle$$

where $|\psi\rangle$ is orthogonal to the coincidence subspace, thus containing only non-logical states.

4.9.33 The limit of Ralph's CNOT gate is that although the gate generates the correct answer in some subspace, there is no way to project the state onto this subspace to get rid of the non-computational part $\frac{\sqrt{8}}{3} |\psi\rangle$ without measuring all of the modes (and therefore losing superposition of states).

4.9.34 An alternative strategy consists in using *heralded* states. This is the strategy employed by Knill. They design a linear optical circuit on 6 modes and 4 photons: 2 dual-rails qubits and two auxiliary mode (with one photon each). The logical, computational subspace $\mathcal{K}_{\text{Knill}}^{\text{log}}$ is generated by

$$|1, 0, 1, 0, 1, 1\rangle, |1, 0, 0, 1, 1, 1\rangle, |0, 1, 1, 0, 1, 1\rangle, |0, 1, 0, 1, 1, 1\rangle,$$

and the gate acts in such a way that measuring only the two last modes determines the success of the procedure. In other words, the coincidence subspace $\mathcal{K}_{\text{Knill}}$ is now larger than the logical subspace: it contains all basis elements where the two last modes are in state $|1, 1\rangle$: it is generated by

$$\begin{aligned} &|1, 0, 1, 0, 1, 1\rangle, |1, 0, 0, 1, 1, 1\rangle, |0, 1, 1, 0, 1, 1\rangle, |0, 1, 0, 1, 1, 1\rangle, \\ &|2, 0, 0, 0, 1, 1\rangle, |0, 2, 0, 0, 1, 1\rangle, |0, 0, 2, 0, 1, 1\rangle, |0, 0, 0, 2, 1, 1\rangle, \\ &|1, 1, 0, 0, 1, 1\rangle, |0, 0, 1, 1, 1, 1\rangle, \end{aligned}$$

while the orthogonal subspace $\mathcal{K}_{\text{Knill}}^{\perp}$ is generated by all the possible vectors of the form

$$|\star, \star, \star, \star, 0, 0\rangle, |\star, \star, \star, \star, 0, 1\rangle, |\star, \star, \star, \star, 0, 2\rangle, |\star, \star, \star, \star, 1, 0\rangle, |\star, \star, \star, \star, 2, 0\rangle.$$

4.9.35 Knill's gate is a C - Z : it is described in [Kni02]. In our framework, the matrix is as follows.

$$\frac{1}{3} \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -\sqrt{2} & \sqrt{2} & 2 \\ 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & \sqrt{2} & 0 & -1 & -2 & \sqrt{2} \\ 0 & -\sqrt{3+\sqrt{6}} & 0 & \sqrt{3-\sqrt{6}} & -\frac{1}{\sqrt{2}}\sqrt{3+\sqrt{6}} & \frac{1}{\sqrt{2}}\sqrt{3-\sqrt{6}} \\ 0 & -\sqrt{3-\sqrt{6}} & 0 & -\sqrt{3+\sqrt{6}} & -\frac{1}{\sqrt{2}}\sqrt{3-\sqrt{6}} & -\frac{1}{\sqrt{2}}\sqrt{3+\sqrt{6}} \end{pmatrix}.$$

It sends a normalized vector

$$\alpha |1, 0, 1, 0, 1, 1\rangle + \beta |1, 0, 0, 1, 1, 1\rangle + \gamma |0, 1, 1, 0, 1, 1\rangle + \delta |0, 1, 0, 1, 1, 1\rangle \in \mathcal{K}_{\text{Knill}}$$

to

$$\frac{\sqrt{2}}{3\sqrt{3}} (\alpha |1, 0, 1, 0, 1, 1\rangle + \beta |1, 0, 0, 1, 1, 1\rangle + \gamma |0, 1, 1, 0, 1, 1\rangle - \delta |0, 1, 0, 1, 1, 1\rangle) + \frac{\sqrt{5}}{3\sqrt{3}} |\psi\rangle$$

where $|\psi\rangle \in \mathcal{K}_{\text{Knill}}^\perp$. When measuring the two last modes, getting 1, 1 ensures the state got projected onto $\mathcal{K}_{\text{Knill}}$. We retrieve the superposed state

$$\alpha |1, 0, 1, 0, 1, 1\rangle + \beta |1, 0, 0, 1, 1, 1\rangle + \gamma |0, 1, 1, 0, 1, 1\rangle - \delta |0, 1, 0, 1, 1, 1\rangle :$$

a controlled Z-gate was indeed applied.

4.9.36 Note that the matrix in 4.9.35 is the identity on $|1, 0, 0, 0, 0\rangle$ and $|0, 0, 1, 0, 0, 0\rangle$. This can be understood by considering the fact that in the computational subspace, Knill's C-Z is only performing an action on $|0, 1, 0, 1, 1, 1\rangle$. A photon in the first or in the third mode can therefore be disregarded as they will not bring anything: we can as well set the action of the C-Z to be trivial.

4.10 Exercises

The exercises are grouped according to the various themes of the section.

Complexity

4.10.1 Consider the following quantum algorithm for factoring a number N :

- Allocate $n = \log_2(N)$ qubits in state $|0\rangle$
- Apply Hadamard on all of them
- Measure them and retrieve the corresponding bitstring
- This bitstring is the encoding of a natural number: test whether it is a factor of N (with the usual, efficient Euler division)
- If it is not, start over.

This algorithm eventually succeeds. Questions:

1. What is the probability of success?
2. How many times should we iterate the process in order to reach a high probability?
3. Derive the complexity of this (probabilistic) algorithm.

Gate decomposition

4.10.2 Using the scheme of 4.4.3, propose a circuit for C-C-Z, using CNOT and T -gates. Derive a circuit for the Toffoli gate.

Magic states

4.10.3 Consider the magic state $|A_{\pi/4}\rangle$ as defined in 4.5.2.

1. Inspired from the circuit in Eq (34), derive a deterministic procedure to implement a T -gate, using a single measurement and a S -gate classically controlled on the result of the measurement. Unlike the general case, for a T -gate there is no need to perform repeat-until-success (when S gates are natively available). We called such a controlled gate a *correction*.
2. Deduce from 4.10.2 a deterministic procedure for realizing a Toffoli gate using only magic states, measurements and Clifford gates. Draw the corresponding circuit, and spell out the convention you use for attaching each correction gate to its measurement result.

MBQC

4.10.4 In the circuit of 4.6.10 presenting a MBQC pattern for CNOT, where are the control and target qubits?

4.10.5 Show that $J(\theta)$ can be written as a composition of HAD and R_θ . Derive an MBQC pattern for R_θ .

4.10.6 First, solve 4.10.5. Then, using 4.4.2, write a circuit for C - S and derive an MBQC pattern to implement it. Write it in normal form. What differs for C - S^* ?

4.10.7 First, solve 4.10.6. Using the decomposition described in 4.4.2, one can write C - C - Z with C - S and C - S^* . Derive the graph state of an MBQC pattern realizing a Toffoli gate.

Quantum Linear Optics

4.10.8 In the encoding of 4.9.10, remembering 4.9.17. How can we encode:

1. a CNOT where the control is qubit 1 and the target qubit 2?
2. a Hadamard on qubit 1?
3. a Hadamard on qubit 2?
4. a CNOT where the control is qubit 2 and the target qubit 1?
5. a SWAP gate?

4.10.9 Consider a system with a single photon that can take any mode between $w_0, w_1, \dots, w_7$. Each one of these modes corresponds to a logical 3-qubits state: w_0 is $|000\rangle$, w_1 is $|001\rangle$, etc. How would you encode...

1. a CNOT where the control is qubit 1 and the target qubit 2?
2. a CNOT where the control is qubit 2 and the target qubit 3?
3. a CNOT where the control is qubit 3 and the target qubit 1?
4. a Hadamard on qubit 1?
5. a Hadamard on qubit 2?
6. a Hadamard on qubit 3?
7. a SWAP between qubit 2 and 3?
8. a permutation of the qubits $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$?

4.10.10 Consider a Hadamard beamsplitter on a 2-modes system as in 4.9.28. Give the action of the gate on the possible 3-photons states

$$|3, 0\rangle, \quad |2, 1\rangle, \quad |1, 2\rangle, \quad |0, 3\rangle$$

4.10.11 Consider Ralph's CNOT encoding in 4.9.32. The gate works with 6 modes, using 4 photons for 2 qubits and 2 auxiliary empty modes (thus initialized without any photon). The coincidence subspace is built from

$$|1, 0, 1, 0, 0, 0\rangle, \quad |1, 0, 0, 1, 0, 0\rangle, \quad |0, 1, 1, 0, 0, 0\rangle, \quad |0, 1, 0, 1, 0, 0\rangle,$$

and corresponds to logical qubits.

Questions.

1. Compute the projection on the coincidence subspace for the application of the gate to each of these vectors. Check that we indeed get a CNOT, and retrieve the probability of success.
2. In the same basis, what is a Hadamard gate on qubit 1? on qubit 2?
3. In the same basis, what is a SWAP between qubit 1 and qubit 2?
4. Consider the matrix:

$$V = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 0 & 0 & 0 & \sqrt{2} & 0 \\ 0 & -1 & \sqrt{2} & 0 & 0 & 0 \\ 0 & \sqrt{2} & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & \sqrt{2} \\ \sqrt{2} & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & \sqrt{2} & 0 & -1 \end{pmatrix}.$$

acting on the mode-space $|w_1\rangle, |w_2\rangle, |w_3\rangle, |w_4\rangle, |w_5\rangle, |w_6\rangle$. What is its behavior on the logical encoding of 2 qubits in Ralph's coincidence basis?

5 Structure of Quantum Algorithms

5.1 High-Level View

5.1.1 As any algorithm, the role of a *quantum algorithm* is to solve a regular, *classical problem*. Such a problem typically consists in

- An *instance*: a list of classical information: a graph, an integer, a matrix, *etc.* These can be given as datastructure: an unsigned integer on 64 bits, an array of floating point numbers, *etc.* or as a function: for constructing the neighbors of a node in a graph, for computing the coefficient of a matrix given a pair of indices, *etc.*
- A question, whose answer might be yes or no, or an object to construct.

Given a problem, an algorithm inputs an instance and outputs an answer to the question. The critical point is that the output should indeed answer the question: this requires a mathematical analysis of the algorithm and its adequacy with the problem.

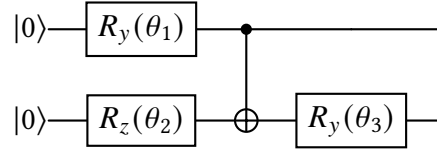
5.1.2 A quantum algorithm can be regarded as a regular algorithm which makes use of a quantum co-processor to build an answer to the question. The general structure of a quantum algorithm as found in the literature is as follows:

1. Input a problem instance.
2. From this instance, specify a bunch of quantum registers, and build a quantum circuit made of quantum (unitary) gates. Among the registers, one of them is meant to be measured at the end.
3. Send the circuit to the quantum co-processor, and measure the specified register: This gives back a bitstring.
4. Perform some post-processing using the bitstring. There are two cases:
 - (a) A solution to the problem instance was found! Exit with success.
 - (b) No solution was found: Loop back at Step (3).

The measurement is probabilistic: the algorithm is designed in such a way that we eventually branch out in (4a) with high enough probability. In such an algorithm, if the circuit depends on the problem instance, once it is built it is used over and over until a solution is found. The algorithm somehow builds its very own *faulty soothsayer*. Since it is faulty, several queries might be needed to eventually get a correct solution.

5.1.3 Note how the general structure presented in 5.1.2 generates a distinct circuit for every problem instance. If your algorithm aims at finding a specific node a graph, two different graphs will give two different circuits. Similarly, if you aim at factoring 15 or 110210873687 surely you wouldn't use the same circuit either. A quantum algorithm is therefore not describing *one* quantum circuit but a *family of quantum circuits*.

5.1.4 The situation presented in 5.1.2 is very simple: given a problem instance the circuit is defined once for all. A slightly more general procedure is found in *variational algorithms*. There, instead of constructing *one* circuit, the algorithm constructs a circuit parameterized by angles: some gates, such as rotation gates, can be changed specified at run-time. For instance, the following circuit is parameterized by three angles $\theta_1, \theta_2, \theta_3$:



When the angles are not decided yet, we just have a shape, or a structure of circuit. Such a circuit-shape is called an *ansatz*.

5.1.5 The interaction with the quantum coprocessor is relatively simple for both 5.1.2 and 5.1.4: a circuit is flushed to the quantum memory, followed by a measurement, and a complete reset. The quantum memory between calls to the co-processor is not preserved. More exotic algorithms require such a preservation between calls: the stream of gates sent to the co-processor is not a circuit built once for all, but is instead based on the result of intermediary measurements. More than a fixed structure, the circuit comes as a trace of (classical) execution.

5.1.6 In the rest of this section, we shall look at typical subroutines used in the design of quantum algorithms.

5.2 Oracles

5.2.1 As discussed in 5.1.3, a quantum circuit should somehow contain a description of the problem instance. As the problem instance is typically encoded on a regular, conventional memory made of bits, such description can typically be defined using a classical, conventional function acting on bitstrings. We are therefore given a Boolean function $f : \mathbb{B}^n \rightarrow \mathbb{B}^m$, and we need to make it available to the quantum co-processor. Of course, this has to be done with the use of unitary operations.

5.2.2 If the function f is reversible, then $n = m$ and we can rely on 2.10.3 to identify f with the action of a unitary map on basis vectors. For instance, consider the operation

x	000	001	010	011	100	101	110	111
$f(x)$	001	010	011	100	101	110	111	000

You can easily convince yourself that this function is a bijection on bitstrings of size 3 as it corresponds to single digit increment modulo 8. This operation can be generalized to the unitary operation

$$\begin{array}{llll}
 |000\rangle \mapsto |001\rangle & |001\rangle \mapsto |010\rangle & |010\rangle \mapsto |011\rangle & |011\rangle \mapsto |100\rangle \\
 |100\rangle \mapsto |101\rangle & |101\rangle \mapsto |110\rangle & |110\rangle \mapsto |111\rangle & |111\rangle \mapsto |000\rangle .
 \end{array}$$

With the standard basis in lexicographic ordering, the corresponding matrix is

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Referring again to 2.10.3, we can claim that this matrix is unitary. From what we discussed in Ch. 4, we know that we can implement this operation on a quantum co-processor.

5.2.3 In general, the function f of 5.2.1 might not be reversible: we cannot identify it with a unitary.

Without loss of generality, one can assume that $m = 1$. Indeed, if $m > 1$, the function f can be regarded as a family of functions $(f_1, \dots, f_m)$, all of codomain $\mathbb{B}$. Therefore assuming $m = 1$, as the function f is usually not invertible the standard way to do this is to instead implement

$$\begin{aligned} \tilde{f} : \mathbb{B}^n \times \mathbb{B} &\rightarrow \mathbb{B}^n \times \mathbb{B} \\ (x, y) &\mapsto (x, y \oplus f(x)). \end{aligned}$$

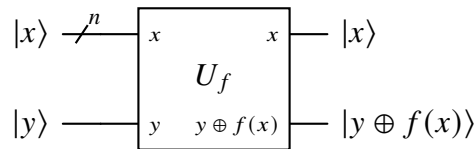
This function is invertible, as

$$\tilde{f}(\tilde{f}(x, y)) = \tilde{f}(x, y \oplus f(x)) = (x, y \oplus f(x) \oplus f(x)) = (x, y)$$

since $z \oplus z = 0$ for all z , and $z \oplus 0 = z$ for all z . This function being invertible, the unitary

$$U_f : \mathcal{H}^{\otimes n} \otimes \mathcal{H} \rightarrow \mathcal{H}^{\otimes n} \otimes \mathcal{H}$$

defined by



Note that U_f is indeed a unitary, it sends a basis vector to a basis vector,

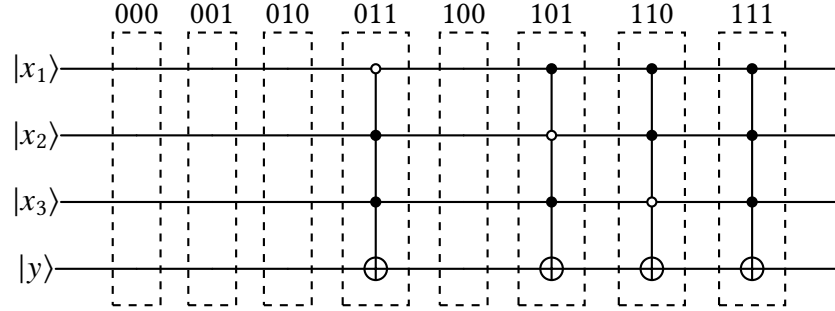
5.2.4 In general, such a box is called an *oracle*: it captures the (classical) structure of the problem instance. For instance, it can correspond to an arithmetic operation, or the neighboring relation for a graph, etc.

5.2.5 From 4.3, we know that regardless of its internal structure, the unitary U_f is realizable using CNOT and 1-qubit rotations. The procedure however requires in general to perform Cosine-Sine decompositions. In the case of U_f , we can rely on the fact that it was built from the classical, boolean function f .

5.2.6 In the most general situation, f is given as a truth table. As an example, consider the function $f : \mathbb{B}^3 \rightarrow \mathbb{B}$ defined as follows.

$x_1x_2x_3$	000	001	010	011	100	101	110	111
$f(x_1, x_2, x_3)$	0	0	0	1	0	1	1	1

Given the truth table, a simple way to build the circuit U_f is to add one multi-control for each value entry yielding 1, as follows.

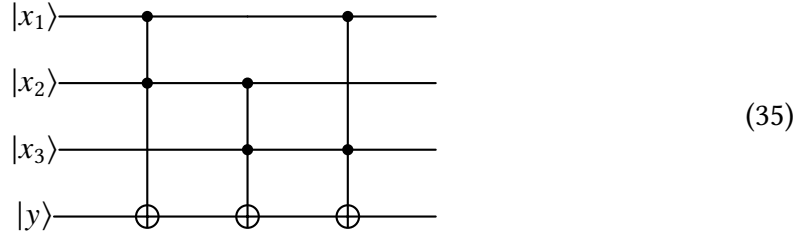


As you can easily see, this is not optimal in general: the size of the circuit is typically exponential on the number of inputs of f .

5.2.7 This function is the *majority function*: $f(x_1x_2x_3)$ is 0 if there are more 0's than 1 in $x_1x_2x_3$, and 1 otherwise. Instead of a truth table, the function can be described with the formula:

$$f(x_1, x_2, x_3) = (x_1x_2) \oplus (x_2x_3) \oplus (x_3x_1).$$

Compared to the circuit presented in 5.2.6, we can then do better with only 3 Toffoli gates:

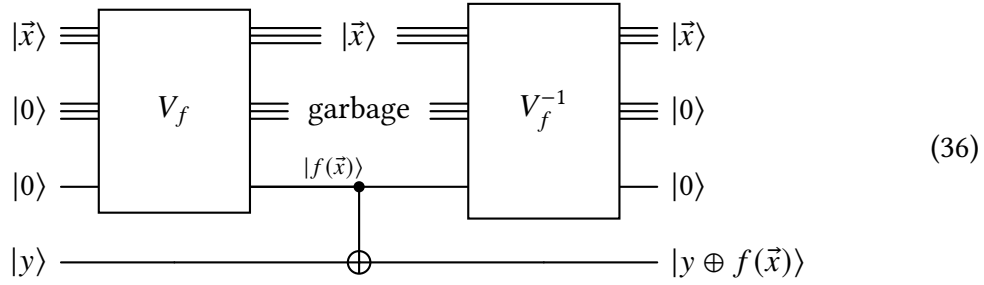


The idea behind this circuit is akin to what was described in 4.4.7: subformulas are computed and their results are stored for use later on. For the circuit of Eq. 35 we did not have to use auxiliary wires, but in general we can: this is the topic of 5.2.8.

5.2.8 If the Boolean function f is given as a boolean formula made of conjunctions and negations, using ancillas one can build a circuit of size linear to the size of the formula. The idea is that

- Conjunctions: implementable with Toffolis;
- Negations: implementable with X -gates and CNOT-gates;
- Composition: corresponds to circuit composition.

The procedure follows the description given in 4.4.7: we first *compute* the final result using a sub-circuit V_f , aggregating subcomputations in ancillas (these are called *garbage qubits*), we *copy* the result to the dedicated register, and we finally *uncompute* the ancillas:



5.2.9 The operator V_f has a strict specification: the input qubits should be left untouched (when in canonical basis state), the last wire should contain the result (when fed with $|0\rangle$), while the middle qubits (the *garbage qubits*) store the subcomputations. In particular, the circuit V_f sends canonical basis states to canonical basis states.

5.2.10 The construction presented in 5.2.8 is using the trick discussed in 3.9.8: we temporarily work inside a larger space to have more rooms, in this case to be able to store intermediary computations. If V_f is sending

$$|\vec{x}\rangle \otimes |0\rangle \otimes |0\rangle \mapsto |\vec{x}\rangle \otimes |f_{\text{garbage}}(\vec{x})\rangle \otimes |f(\text{vec } x)\rangle$$

then the sequence of operations given in Eq. 36 is doing the following:

$$\begin{aligned}
 & \sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |y\rangle \\
 \mapsto & \sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |0\rangle \otimes |0\rangle \otimes |y\rangle && \text{(ancilla allocation)} \\
 \mapsto & \sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |f_{\text{garbage}}(\vec{x})\rangle \otimes |f(\vec{x})\rangle \otimes |y\rangle && \text{(applying } V_f) \\
 \mapsto & \sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |f_{\text{garbage}}(\vec{x})\rangle \otimes |f(\vec{x})\rangle \otimes |y \oplus f(\vec{x})\rangle && \text{(CNOT-gate)} \\
 \mapsto & \sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |0\rangle \otimes |0\rangle \otimes |y \oplus f(\vec{x})\rangle && \text{(applying } V_f^{-1})
 \end{aligned}$$

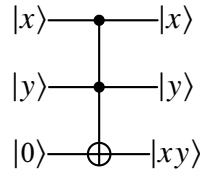
The middle qubits are back to $|0\rangle$, so there are not entangled with the rest of the system: if we discard them, following the discussion of 3.9, the result is deterministic and yield with probability 1

$$\sum_{\vec{x}, y} \alpha_{\vec{x}, y} \cdot |\vec{x}\rangle \otimes |y \oplus f(\vec{x})\rangle$$

which is precisely the behavior expected for U_f .

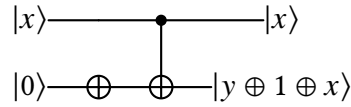
5.2.11 One can build the circuit for the operator V_f is built inductively on the structure of f , as follows:

- If f is the conjunction: $f(x, y) = x \wedge y = xy$, we build V_f with a Toffoli gate:



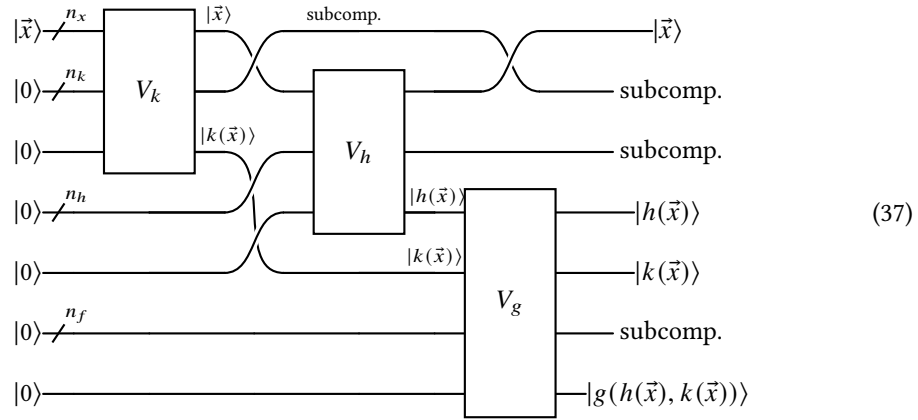
It leaves the inputs untouched, while storing the $f(x, y)$ in the last wire if it was initialized with $|0\rangle$.

- If f is the negation: $f(x) = \neg x = 1 \oplus x$, we can build V_f with a CNOT-gate:



Note that we could have used a simple X -gate, but this would have modified the input qubit, therefore breaking the specification.

- Finally, if f is a composition of sub-formulas, such as $f(x) = g(h(x), k(x))$, one can build V_f out of V_g , and V_h and V_k :



The first wires are set back to $|\vec{x}\rangle$, the last wire contains the result, while the middle wires contain all of the sub-computations, including the two values $g(\vec{x})$ and $k(\vec{x})$ computed along the composition.

5.2.12 Note how in Eq.(37) the input $|\vec{x}\rangle$ is used twice for V_h and V_k . This is why, in general, using this method one cannot use an X -gate to realize the negation: one has to keep the original value around. One can use another strategy to produce a more compact circuit, but this circuit construction is however efficient in the sense that the number of gates and auxiliary wires is linear on the size of the formula describing f .

5.2.13 Instead of a basis change, the operation U_f can also be regarded as a phase-flip, as follows:

$$\begin{aligned}
 U_f |\vec{x}\rangle \otimes |-\rangle &= U_f \frac{1}{\sqrt{2}} |\vec{x}\rangle \otimes (|0\rangle - |1\rangle) \\
 &= U_f \frac{1}{\sqrt{2}} (|\vec{x}\rangle \otimes |0\rangle - |\vec{x}\rangle \otimes |1\rangle) \\
 &= \frac{1}{\sqrt{2}} (U_f(|\vec{x}\rangle \otimes |0\rangle) - U_f(|\vec{x}\rangle \otimes |1\rangle)) \\
 &= \frac{1}{\sqrt{2}} (|\vec{x}\rangle \otimes |f(\vec{x})\rangle - |\vec{x}\rangle \otimes |1 \oplus f(\vec{x})\rangle) \\
 &= \frac{1}{\sqrt{2}} |\vec{x}\rangle \otimes (|f(\vec{x})\rangle - |1 \oplus f(\vec{x})\rangle) \\
 &= (-1)^{f(\vec{x})} \frac{1}{\sqrt{2}} |\vec{x}\rangle \otimes (|0\rangle - |1\rangle) \quad (\text{From 2.10.2}) \\
 &= (-1)^{f(\vec{x})} |\vec{x}\rangle \otimes |-\rangle.
 \end{aligned}$$

5.3 Encoding Natural Numbers

5.3.1 A typical classical function to turn into a unitary consists in an arithmetic operation such as

$$\begin{aligned}
 f_{a,N} : \quad \{0 \dots N-1\} &\longrightarrow \{0 \dots N-1\} \\
 x &\longmapsto a \cdot x \pmod{N}
 \end{aligned}$$

for a and N natural numbers. If a and N are co-prime, the function is a bijection (this function is the oracle for Shor's algorithm, see 6.2). For instance, $f_{5,8}$ consists in the map

x	0	1	2	3	4	5	6	7
$f_{5,8}(x)$	0	5	2	7	4	1	6	3

The map $f_{5,8}$ is a bijection of $\{0, \dots, 7\}$: we are in the setting of 2.10.4 (where the map was $f_{3,8}$) and 5.2.2: the function can be regarded as a unitary on a 8-dimensional Hilbert space. However, the map acts on numbers whereas the discussion in 5.2 is based on Boolean values: we need a bit *encoding* of numbers as *bitstrings*.

5.3.2 A natural number x can be decomposed in *base 2* as

$$x = \sum_{k=0}^{N-1} b_k \cdot 2^k$$

for some number N and some Boolean values $b_0, \dots, b_{N-1}$. We say that b_0 is the *least significant bit*. The Boolean values b_k 's are enough to recover the number x , provided that we choose the ordering of the bits in the list. There are two canonical ones, that we can refer to as

- *big-endian*: the number x is represented with a list starting with the most significant bits:

$$b_{N-1}, \dots, b_1, b_0.$$

This is the mathematical representation (used in base 10 for instance), and used in most paper and textbooks in quantum computation.

- *little-endian*: the number x is represented with a list starting with the *least* significant bit:

$$b_0, b_1, \dots, b_{N-1}.$$

This is useful if the number is stored in an array: b_0 is then literally $b[0]$. This convention is used in QISKIT for instance.

5.3.3 Beware In summary, there are at least two difficulties to look for when debugging quantum programs:

- conflicting ordering for binary representations;
- conflicting basis ordering for matrix and vector representations.

5.3.4 The domain and codomain of the map $f_{5,8}$ in 5.3.1 can be encoded on bitstrings of size 3. As discussed in 5.3.2, we can for instance decide on big-endian, so that 3 is represented by 011. The lexicographic ordering of the canonical basis on $\mathcal{H}^{\otimes 3}$ then exactly corresponds to the natural number ordering, and the matrix for $f_{5,8}$ can be constructed in the same way as in 2.10.4:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

This matrix corresponds to the map $f_{5,8}$ acting on

$$\{|e_0\rangle, |e_1\rangle, |e_2\rangle, |e_3\rangle, |e_4\rangle, |e_5\rangle, |e_6\rangle, |e_7\rangle\}$$

as in 2.10.3, but also as the corresponding action on $\mathcal{H} \otimes \mathcal{H} \otimes \mathcal{H}$ with the basis ordering

$$|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle.$$

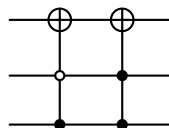
It performs the action:

$$\begin{array}{llll} |000\rangle \mapsto |000\rangle & |010\rangle \mapsto |010\rangle & |100\rangle \mapsto |100\rangle & |110\rangle \mapsto |110\rangle \\ |001\rangle \mapsto |101\rangle & |011\rangle \mapsto |111\rangle & |101\rangle \mapsto |001\rangle & |111\rangle \mapsto |011\rangle \end{array}$$

5.3.5 For the case of 5.3.4, it is possible to infer a circuit “by hand” by realizing that the map can be factored into

$$|x\rangle \otimes |01\rangle \mapsto |x\rangle \otimes |01\rangle \quad |x\rangle \otimes |11\rangle \mapsto |x\rangle \otimes |11\rangle$$

and identity otherwise. A circuit implementing the action is therefore



5.3.6 Example: Reversible Adder The oracle in 5.3.1 is reversible: it was therefore possible to build a circuit without ancillas as for instance shown in 5.3.5. In general however, the classical function we care about is not reversible: we therefore need a more sophisticated circuit. We discuss here how to encode an *adder*: a function inputting two numbers a and b and outputting $a + b$.

5.3.7 There are still several design choices for arithmetic on a bitstring encoding.

- We want a circuit: an *encoding* of our numbers as bitstrings. But which numbers? Integers (*signed* integers), natural numbers (*unsigned* integers)? On which range? Is it the same range for a and for b ?
- How do we treat overflows? Is the range of the output expanded by one bit to take care of it, or do we consider arithmetic modulo instead?
- There are two “standard” ways to turn addition into a reversible operation. One can first turn the adder into a reversible function as in 5.2.3:

$$(\underline{a}, \underline{b}, z) \mapsto (\underline{a}, \underline{b}, z \oplus (\underline{a} + \underline{b})) \quad (38)$$

where it is understood that $\underline{n}$ is a bitstring encoding of the number n . Of course, we still need to characterize the size of each bitstring, and the actual behavior of the “+” operator (signed, unsigned, etc).

The procedure shown in Eq. (38) does not modify the input registers a and b but requires an additional register z . If one does not need to retain the input register, for the addition one can follow a possibly more compact strategy and store the output in one of the registers:

$$(\underline{a}, \underline{b}) \mapsto (\underline{a}, \underline{a} + \underline{b}). \quad (39)$$

Of course, this only makes sense if the register b is not used later on.

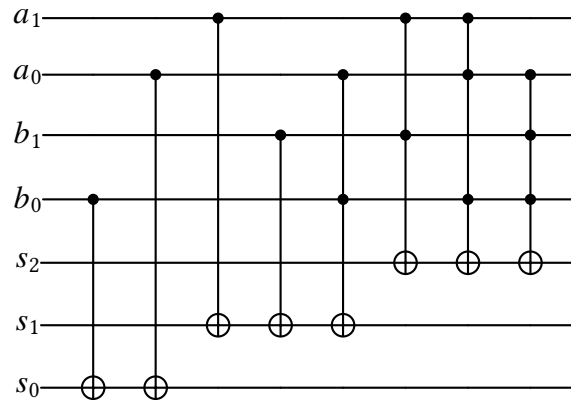
5.3.8 Consider an adder for numbers coded on bitstrings of size 2: $\underline{a} = a_1a_0$ and $\underline{b} = b_1b_0$ (with a_k s and b_k s Boolean values). We have

$$a = a_1 \cdot 2 + a_0, \quad b = b_1 \cdot 2 + b_0.$$

With a bit of thinking one can show that

$$a + b = (a_1b_1 \oplus a_0b_0a_1 \oplus a_0b_0b_1) \cdot 2^2 + (a_1 \oplus b_1 \oplus a_0b_0) \cdot 2 + (a_0 \oplus b_0) \quad (40)$$

The operator U_{add} as in 5.2.3 can be realized with the circuit



The answer is read in register s : we have $\underline{a + b} = s_2s_1s_0$.

5.3.9 The circuit presented in 5.3.8 was built “by hand”. In order to go further, we have to *program* the adder. The simplest method is the *ripple-carry adder* procedure—the procedure followed when performing addition “by hand” on a sheet of paper. There are two cases:

- Base case: adding a_0 and b_0 , yielding a result s_0 and a carry c_0 . This is a *half-adder*, and the formulas are

$$s_0 = a_0 \oplus b_0, \quad c_0 = a_0 b_0. \quad (41)$$

- Inductive case: adding a_{k+1} , b_{k+1} and the carry c_k from the layer below, yielding a result s_{k+1} and a new carry c_{k+1} . This is a *full-adder*, and the formulas are

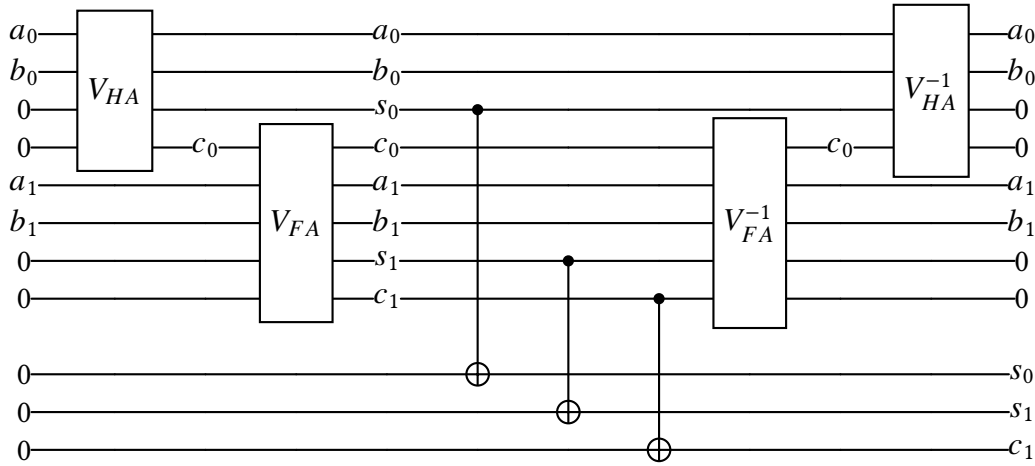
$$s_{k+1} = a_k \oplus b_k \oplus c_k, \quad c_{k+1} = a_k b_k \oplus c_k a_k \oplus c_k b_k. \quad (42)$$

Unsurprisingly, from these formulas we can recover Eq. (40). Such a construction makes what is called a *ripple-carry adder*. The 2-bit adder of 5.3.8 can then be written in pseudo-code as

```
s0,c0 = HA(a0,b0)
s1,c1 = FA(c0,a1,b1)
return (c1,s1,s0)
```

where HA is a function coding the half-adder and FA is a function coding the full-adder. One can easily extend the procedure to any number of bits by chaining together additional full-adders.

Using the procedure described in 5.2.8, assuming V_{HA} and V_{FA} don’t require any ancillas, we can write a reversible 2-bits adder as follows.



5.3.10 In term of circuit size, it is possible to show that $V_{HA} = 2 \cdot \text{CNOT} + C^2 \cdot X$ and $V_{FA} = 3 \cdot \text{CNOT} + 3 \cdot C^2 \cdot X$. For n -bits integers, the ripple-carry procedure yields a number of gates of

$$n \cdot V_{HA} + n \cdot V_{FA} + (n + 1) \cdot \text{CNOT} = (6n + 1) \cdot \text{CNOT} + (4n) \cdot C^2 \cdot X.$$

For the “naive” procedure of 5.3.8, we have for 2-bits integers: $4 \cdot \text{CNOT} + 4 \cdot C^2 \cdot X$. Compared to the ripple-carry adder, the computation $a_0 b_0$ is performed 2 times. For n -bits integers, we will also need $C^3 \cdot X$, $C^4 \cdot X$, $\dots$ $C^n \cdot X$, and all sub-computations would be repeated several times. It in fact corresponds to unfolding Eqs (41) and (42): we can see that we would end up with a quadratic number of gates.

A last strategy to generate U_{add} would consists in using the decomposition summarized in 4.3.17. As discussed in 4.3.20, the size of the generated circuit would then be of the order of 4^N , where N is the size of the matrix. In our case, $N = 3n + 1$ (3 registers of size n and an additional wubit to store the final carry). The circuit is therefore of size $O(4 \cdot 64^n)$.

In summary, we are back to the trade-off discussion in 4.4. The ripple-carry produces a linear-sized circuit but with ancillas. It is possible to get rid of the ancillas, but at the expense of a circuit of larger size. However, in this particular case, the unitary is not arbitrary: we can rely on its internal structure to dodge an exponential asymptotic complexity.

5.4 Amplitude Amplification

5.4.1 A typical situation met in the design of quantum algorithms is the case where the quantum memory is in a superposed state

$$\sum_{k=0}^{2^n-1} \alpha_i |k\rangle \otimes |b_k\rangle$$

living in the space $\vec{H}^{\otimes n} \otimes \mathcal{H}$ ($b_k \in \{0, 1\}$) and where we are looking for one of the k such that $b_k = 1$: we are interested in the subspace where the last qubit is $|1\rangle$.

5.4.2 A state as in 5.4.1 arises for instance in the case where the oracle U_f as in 5.2.3 was applied to the state

$$\left(\frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle \right) \otimes |0\rangle.$$

We then look for (one of) the k such that $f(k) = 1$.

5.4.3 For instance, consider the problem SQUARENAT:

- Input: a natural number N , with the promise that it is a square;
- Output: the square-root of N : the non-negative number a such that $a^2 = N$.

We can define f as follow: it inputs a bitstring $\vec{x}$ and reads it as the binary representation of a number j . The value $f(\vec{x})$ is then 1 if $j^2 = N$, and 0 otherwise.

5.4.4 Following the strategy presented in 5.4.1, a possible quantum algorithm to solve SQUARENAT can be:

- Start with $|0 \dots 0\rangle \in \vec{H}^n \otimes \vec{H}$, with n large enough, for instance $n = \log_2(N)$.
- Apply a tower of Hadamard on the n th first qubits to get

$$\frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle \otimes |0\rangle.$$

- Apply U_f to get

$$\frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |j\rangle \otimes |f(j)\rangle.$$

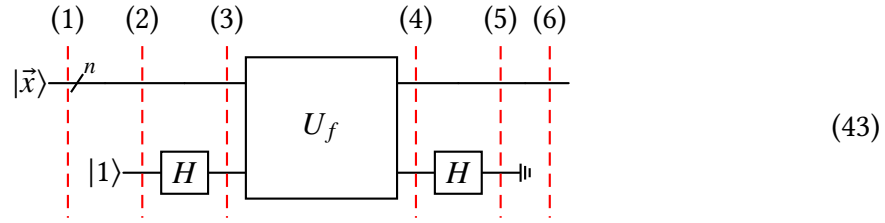
- Measure: if the last bit is 1, we succeed, if not, we start again.

Provided that the promise “N is a square” is held, this algorithm eventually converges to a solution. However, because all of the coefficients are equal, the probability of success for each run is very low: $\frac{1}{2^n}$.

5.4.5 The procedure called *Amplitude Amplification*, or *Grover’s algorithm* (from the first person to have described it), aims at speeding up the process presented in 5.4.4 by augmenting the amplitudes of the “good states”, i.e. the ones for which $f(j)$ is 1. For this, we need 3 operations on $\vec{H}^{\otimes n}$:

- an oracle $O : |x\rangle \mapsto (-1)^{f(x)} \cdot |x\rangle$;
- an operator $U_{0^\perp}$ sending $|0 \dots 0\rangle$ to $|0 \dots 0\rangle$ and every other canonical basis vector $|x\rangle$ to $-|x\rangle$;
- a tower of Hadamard $H^{\otimes n}$.

5.4.6 Note that O is not under the canonical form $U_f : |x\rangle \otimes |z\rangle \mapsto |x\rangle \otimes |z \oplus f(x)\rangle$, but one can build it from U_f using the circuit



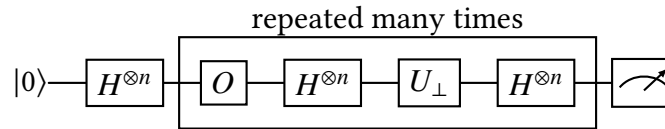
Starting from a canonical basis state $|\vec{x}\rangle$, we get:

$$\begin{aligned}
 & |\vec{x}\rangle & (1) \\
 \mapsto & |\vec{x}\rangle \otimes |1\rangle & (2) \\
 \mapsto & |\vec{x}\rangle \otimes |-\rangle & (3) \\
 \mapsto & (-1)^{f(\vec{x})} |\vec{x}\rangle \otimes |-\rangle & (4) \text{ (from 5.2.13)} \\
 \mapsto & (-1)^{f(\vec{x})} |\vec{x}\rangle \otimes |1\rangle & (5)
 \end{aligned}$$

The measurement operation on the second qubit is deterministic since the state is not entangled with the rest of the system. We then get at (6) desired state $(-1)^{f(\vec{x})} |\vec{x}\rangle$.

5.4.7 In Eq. 43, we measure the auxiliary register. If we have to repeat the circuit (and we will have to do so for Grover), we do not have to measure it: we can reuse it since it is back to the state $|1\rangle$. If we reuse it for the same circuit, we can even simplify it by not applying intermediary Hadamard gates.

5.4.8 Using the building blocks of 5.4.5, Grover's algorithm is as follows:



How many should “many times” be depends on f .

5.4.9 Let us denote $H^{\otimes n} U_{0^\perp} H^{\otimes n}$ with $U_\psi^\perp$. What is its action? Let us consider $|\psi\rangle = H^{\otimes n} |0 \dots 0\rangle$. Let $V_{|\psi\rangle}$ be the subspace generated by $|\psi\rangle$ and $V_{|\psi\rangle}^\perp$ the orthogonal subspace. We then have

$$H^{\otimes n} U_{0^\perp} H^{\otimes n} |\psi\rangle = |\psi\rangle$$

Indeed:

$$\begin{aligned}
 H^{\otimes n} U_{0^\perp} H^{\otimes n} |\psi\rangle &= H^{\otimes n} U_{0^\perp} H^{\otimes n} (H^{\otimes n} |0\rangle) \\
 &= H^{\otimes n} U_{0^\perp} (H^{\otimes n} H^{\otimes n}) |0\rangle \\
 &= H^{\otimes n} U_{0^\perp} Id |0\rangle \\
 &= H^{\otimes n} U_{0^\perp} |0\rangle \\
 &= H^{\otimes n} |0\rangle
 \end{aligned}$$

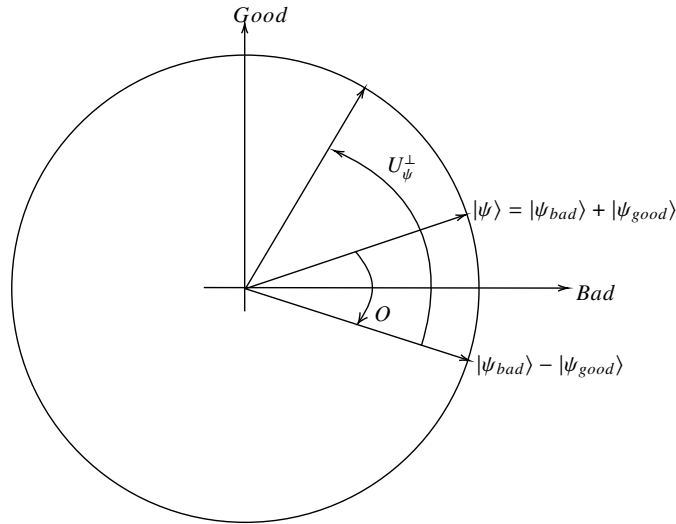


Figure 9: Geometric Intuition for Amplitude Amplification

which is $|\psi\rangle$ by definition. Now, if $|\phi\rangle \in V_{|\psi\rangle}^\perp$, the vector $H^{\otimes n} |\phi\rangle$ does not contain any instance of $|0\dots 0\rangle$ in its canonical decomposition. Indeed, supposed that there were such an instance. We would then have $H^{\otimes n} |\phi\rangle = \alpha |0\rangle + \beta |0^\perp\rangle$ with $|0^\perp\rangle \perp |0\rangle$. So

$$\begin{aligned} H^{\otimes n} (H^{\otimes n} |\phi\rangle) &= H^{\otimes n} (\alpha |0\rangle + \beta |0^\perp\rangle) \\ &= \alpha H^{\otimes n} |0\rangle + \beta H^{\otimes n} |0^\perp\rangle \\ &= \alpha |\psi\rangle + \beta |\psi^\perp\rangle \end{aligned}$$

with $|\psi^\perp\rangle \in V_{|\psi\rangle}^\perp$. But then $H^{\otimes n} (H^{\otimes n} |\phi\rangle) = |\phi\rangle$ is not in $V_{|\psi\rangle}^\perp$: contradiction. Therefore, $H^{\otimes n} U_{0^\perp} H^{\otimes n} |\phi\rangle = -|\phi\rangle$. Indeed, $U_{0^\perp} (H^{\otimes n} |\phi\rangle) = -H^{\otimes n} |\phi\rangle$ since $H^{\otimes n} |\phi\rangle$ does not contain any instance of $|0\dots 0\rangle$, and is therefore orthogonal to $|0\dots 0\rangle$. So $H^{\otimes n} U_{0^\perp} H^{\otimes n} |\phi\rangle = H^{\otimes n} (-H^{\otimes n} |\phi\rangle) = -H^{\otimes n} (H^{\otimes n} |\phi\rangle) = -|\phi\rangle$.

5.4.10 We are now ready to see how the algorithm is working. We start by buiding the state $|\psi\rangle$. It can be decomposed into

$$|\psi\rangle = |\psi_{good}\rangle + |\psi_{bad}\rangle \quad (44)$$

The former consists in the canonical basis kets for which f gives 1, the latter consists in the canonical basis kets for which f yields 0.

The state $|\psi_{good}\rangle$ can be decomposed into $\epsilon |\psi\rangle + \delta |\psi_\perp\rangle$, with $|\psi_\perp\rangle \in V_{|\psi\rangle}^\perp$. From Eq. (44) we derive that $|\psi_{bad}\rangle = |\psi\rangle - |\psi_{good}\rangle = (1 - \epsilon) |\psi\rangle - \delta |\psi^\perp\rangle$. If there are not “too many” solutions for f , the amplitude ϵ is small.

Let us apply G to $|\psi\rangle$:

- we first apply O : it flips around the “bad” states and turns $|\psi\rangle = |\psi_{good}\rangle + |\psi_{bad}\rangle$ into $-|\psi_{good}\rangle + |\psi_{bad}\rangle$, that is,

$$(1 - 2\epsilon) |\psi\rangle - 2\delta |\psi_\perp\rangle.$$

- We then apply $U_{\psi}^{\perp}$: it flips around the $|\psi\rangle$ axis and changes the sign of $|\psi_{\perp}\rangle$: we get

$$(1 - 2\epsilon) |\psi\rangle + 2\delta |\psi_{\perp}\rangle,$$

that is,

$$(3 - 4\epsilon) |\psi_{good}\rangle + (1 - 4\delta) |\psi_{bad}\rangle$$

If ϵ is small enough, the amplitude of “good” states got increased. We can draw a geometrical intuition as shown in Fig. 9: Two symmetries gives a rotation in the direction of the “good” states. If we iterate the process, we get closer and closer. If we do too much, we go too far and the amplitude starts decreasing.

5.4.11 One can compute the optimal number of iterations of G . If there are k values x such that $f(x) = 1$, the optimal is

$$r \sim \frac{\pi}{4} \sqrt{\frac{2^n}{k}}$$

and we get a quadratic speedup compared to a classical approach.

5.5 Quantum Fourier Transform

5.5.1 The *Quantum Fourier Transform* (QFT) is a circuit that moves information between the phase and the canonical basis-ket. The circuit is the QFT , while the reversed one is the QFT inverse. The latter can presented as a problem to solve as follows. You are given a state on n qubits hiding a number $x \in \{0, \dots, 2^n - 1\}$ as follows.

$$\frac{1}{\sqrt{2}^n} \sum_{k=0}^{2^n-1} \omega_x^k |k\rangle,$$

where

$$\omega_x = e^{i \cdot \frac{2\pi}{2^n} \cdot x}.$$

Can you (classically) retrieve the value of x ?

5.5.2 The answer is yes: we can build a circuit computing the operation

$$\frac{1}{\sqrt{2}^n} \sum_{k=0}^{2^n-1} \omega_x^k |k\rangle \mapsto |x\rangle$$

(with the least significant bit on the left in $|x\rangle$).

5.5.3 Let us try with $n = 1$: the hidden value x is then 0 ou 1. The state of the system can be rewritten as

$$\sum_{k=0}^1 e^{\frac{2i\pi}{2} kx} |k\rangle = |0\rangle + (-1)^x |1\rangle.$$

To get back $|x\rangle$, an Hadamard gate is enough.

5.5.4 Let us try with $n = 2$. The hidden value x is then 0, 1, 2 or 3. In binary notation, $[x]_2 = x_1x_2$, i.e. $x = 2x_1 + x_2$. The input state on two qubits is

$$\begin{aligned}
& \sum_{k=0}^{2^n-1} e^{\frac{2i\pi}{2^n} kx} |k\rangle \\
&= \sum_{k=0}^3 e^{2i\pi \left(\frac{x_1}{2} + \frac{x_2}{4}\right) k} |k\rangle \\
&= \sum_{k=0}^3 e^{2i\pi \frac{x_1}{2} k} e^{2i\pi \frac{x_2}{4} k} |k\rangle \\
&= \sum_{k=0}^3 e^{i\pi x_1 k} e^{i\pi \frac{x_2}{2} k} |k\rangle \\
&= \frac{1}{2} (|00\rangle + e^{i\pi x_1} e^{i\pi \frac{x_2}{2}} |01\rangle + e^{2i\pi x_1} e^{i\pi x_2} |10\rangle + e^{3i\pi x_1} e^{3i\pi \frac{x_2}{2}} |11\rangle) \\
&= \frac{1}{2} \left(|00\rangle + e^{i\pi x_1} e^{i\pi \frac{x_2}{2}} |01\rangle + e^{i\pi x_2} |10\rangle + e^{i\pi x_1} e^{3i\pi \frac{x_2}{2}} |11\rangle \right) \\
&= \frac{1}{2} (|0\rangle + e^{i\pi x_2} |1\rangle) \otimes (|0\rangle + e^{i\pi x_1} e^{i\pi \frac{x_2}{2}} |1\rangle) \\
&= \frac{1}{2} (|0\rangle + (-1)^{x_2} |1\rangle) \otimes (|0\rangle + (-1)^{x_1} e^{i\pi \frac{x_2}{2}} |1\rangle).
\end{aligned}$$

Note how the memory state is separable:

- Applying Hadamard on the first qubit, we retrieve $|x_2\rangle$.
- If it were not for the phase $e^{i\pi\frac{x_2}{2}}$, we could get $|x_1\rangle$ with an Hadamard. We first need to get rid of the phase, and this can be done with the controlled rotation $C\text{-}R_2^{-1}$, where

$$R_n = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{2\pi}{2^n}} \end{pmatrix}.$$

Summarizing, to get back $|x_2x_1\rangle$ one can use the circuit

$$\begin{array}{c}
\frac{1}{\sqrt{2}}(|0\rangle + (-1)^{x_2}|1\rangle) \text{ --- } [H] \text{ --- (1) --- } \bullet \text{ --- (2) --- } |x_2\rangle \\
\frac{1}{\sqrt{2}}(|0\rangle + (-1)^{x_1}e^{i\pi\frac{x_2}{2}}|1\rangle) \text{ --- } [R_2^{-1}] \text{ --- } [H] \text{ --- (3) --- } |x_1\rangle
\end{array}
\quad (45)$$

At each step, the memory is changed towards the goal:

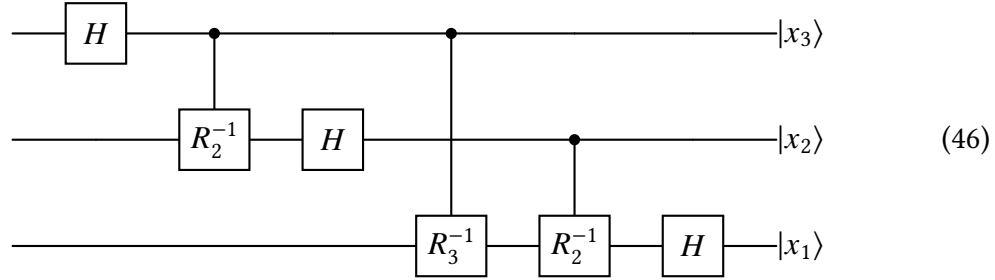
- at (1): $|x_2\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + (-1)^{x_1} e^{i\pi \frac{x_2}{2}} |1\rangle)$;
- at (2): $|x_2\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + (-1)^{x_1} |1\rangle)$;
- at 3: $|x_2\rangle \otimes |x_1\rangle$.

Note that the bits of x are read from right to left!

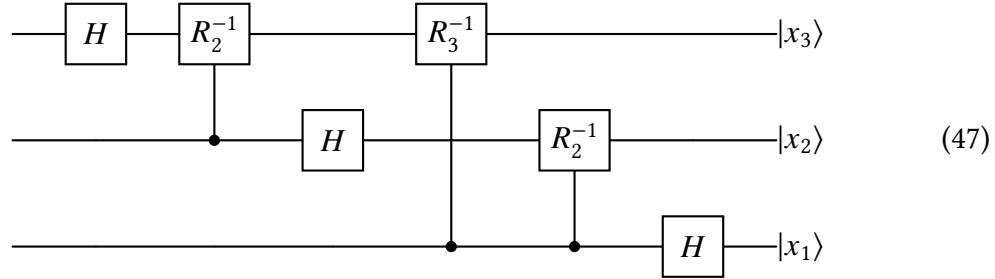
5.5.5 The situation carries over for larger n : the state of the system is in fact separable, and the bits of x can be recovered one by one, using more and more controlled rotations to correct for the additional phases. For instance, at $n = 3$ the state of the system is

$$\frac{1}{\sqrt{2^3}} (|0\rangle + (-1)^{x_3} |1\rangle) \otimes \left(|0\rangle + (-1)^{x_2} e^{i\pi \frac{x_3}{2}} |1\rangle \right) \otimes \left(|0\rangle + (-1)^{x_1} e^{i\pi \frac{x_2}{2}} e^{i\pi \frac{x_3}{4}} |1\rangle \right).$$

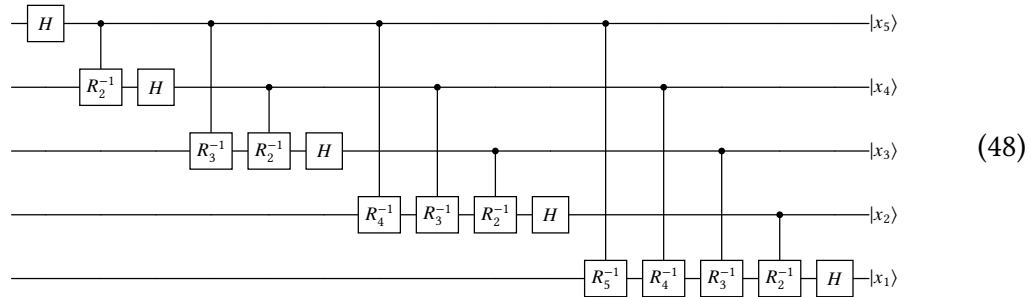
One can recover $|x_3 x_2 x_1\rangle$ using the circuit



5.5.6 Remember 3.6.9: the circuit of Eq. (46) can be equivalently rewritten as



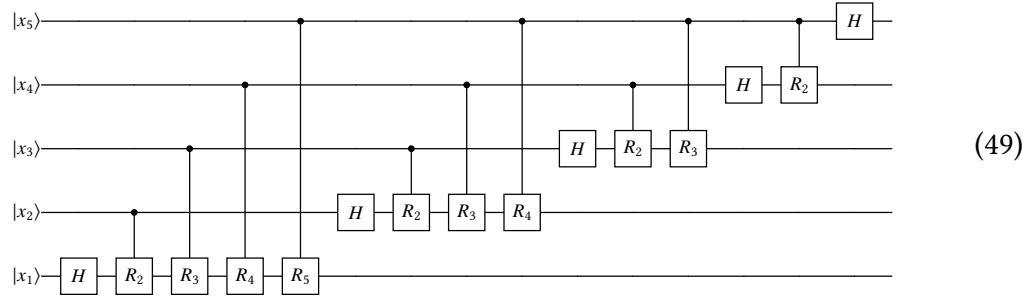
5.5.7 This structure generalizes to n qubits: the circuit called *QFT inverse* can be defined for all n in a similar manner, with blocks of increasing sizes. For instance, for $n = 5$ we get the circuit



5.5.8 Reversing the circuit, we get the *Quantum Fourier Transform*, or *QFT*. It implements the map

$$|x_n \dots x_1\rangle \mapsto \sum_{k=0}^{2^n-1} e^{\frac{2i\pi}{2^n} k [x_1 \dots x_n]_2} |k\rangle,$$

where $[x_1 \dots x_n]_2 = \sum_{k=1}^n x_k 2^{n-k}$. For $n = 5$, the circuit is then the inverse of the one in Eq. (48):



5.6 Phase Estimation

5.6.1 Consider the problem PHASEESTIMATION, defined as follows

Input A unitary U , an eigenvector $|\psi\rangle$ and a natural number $n \in \mathbb{N}$.

Output The phase of the corresponding eigenvalue up to n bits of precision.

2.9.3

The algorithm *QPE* (for *Quantum Phase Estimation*) proposes a circuit for solving the problem.

5.6.2 The algorithm is based on the following property of $C-U$:

$$\begin{aligned}
 C-U((\alpha|0\rangle + \beta|1\rangle) \otimes |\psi\rangle) &= C-U(\alpha|0\rangle \otimes |\psi\rangle + \beta|1\rangle \otimes |\psi\rangle) \\
 &= \alpha \cdot C-U(|0\rangle \otimes |\psi\rangle) + \beta \cdot C-U(|1\rangle \otimes |\psi\rangle) \\
 &= \alpha \cdot (|0\rangle \otimes |\psi\rangle) + \beta \cdot (|1\rangle \otimes (U|\psi\rangle)) \\
 &= \alpha \cdot (|0\rangle \otimes |\psi\rangle) + \beta \cdot (|1\rangle \otimes (e^{2i\pi\omega}|\psi\rangle)) \\
 &= \alpha \cdot (|0\rangle \otimes |\psi\rangle) + \beta e^{2i\pi\omega} \cdot (|1\rangle \otimes |\psi\rangle) \\
 &= (\alpha|0\rangle + \beta e^{2i\pi\omega}|1\rangle) \otimes |\psi\rangle.
 \end{aligned}$$

Note the maybe counterintuitive fact that on this particular shape of input, $C-U$ only changes the phase of the control qubit.

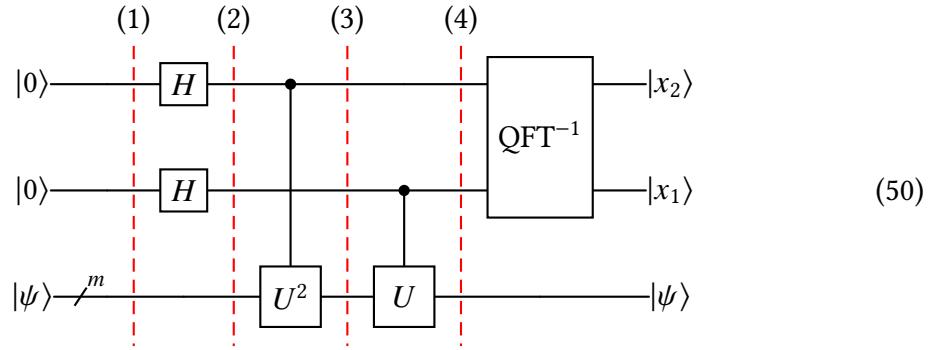
5.6.3 To understand how this works, let us consider that ω is $0.x_1x_2$ in binary form:

$$\omega = \frac{x_1}{2} + \frac{x_2}{4}.$$

Let $|\psi\rangle$ be the corresponding eigenvector. We then have

$$\begin{aligned}
 U|\psi\rangle &= e^{2i\pi\omega}|\psi\rangle \\
 &= (-1)^{x_1} e^{i\pi \frac{x_2}{2}} |\psi\rangle, \\
 U^2|\psi\rangle &= (-1)^{x_1} e^{i\pi \frac{x_2}{2}} (U|\psi\rangle) \\
 &= (-1)^{x_2} |\psi\rangle.
 \end{aligned}$$

Note how the two coefficients are akin to the input of the circuit of Eq. (45). It is not exactly of the right form, but with the remark of 5.6.2, we can derive the circuit

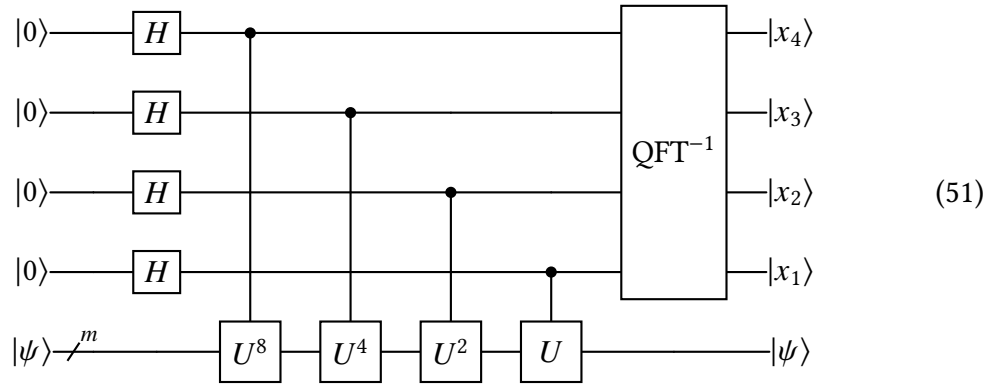


At each step, the state of the system is

1. $|0\rangle \otimes |0\rangle \otimes |\psi\rangle$.
2. $|+\rangle \otimes |+\rangle \otimes |\psi\rangle = \frac{1}{2}(|0\rangle + |1\rangle) \otimes (|0\rangle + |1\rangle) \otimes |\psi\rangle$.
3. $\frac{1}{2}(|0\rangle + (-1)^{x_2}|1\rangle) \otimes (|0\rangle + |1\rangle) \otimes |\psi\rangle$.
4. $\frac{1}{2}(|0\rangle + (-1)^{x_2}|1\rangle) \otimes (|0\rangle + (-1)^{x_1}e^{i\pi\frac{x_2}{2}}|1\rangle) \otimes |\psi\rangle$.

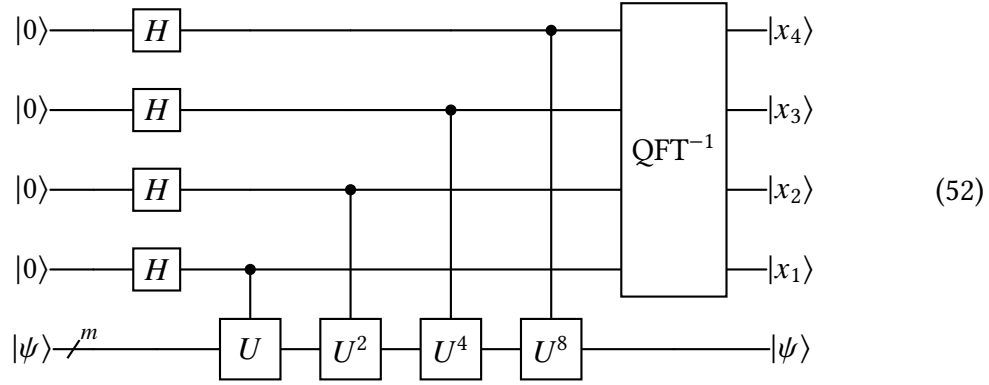
and at this step, the two first qubits are exactly the input of the circuit in Eq. (45): Applying the QFT inverse yields $|x_2x_1\rangle \otimes |\psi\rangle$: we can retrieve ω with a measurement.

5.6.4 Once again, this generalizes. One can also show that this is also working (albeit probabilistically) if ω is not writable on precisely n bits. The circuit for 4 bits of precision is

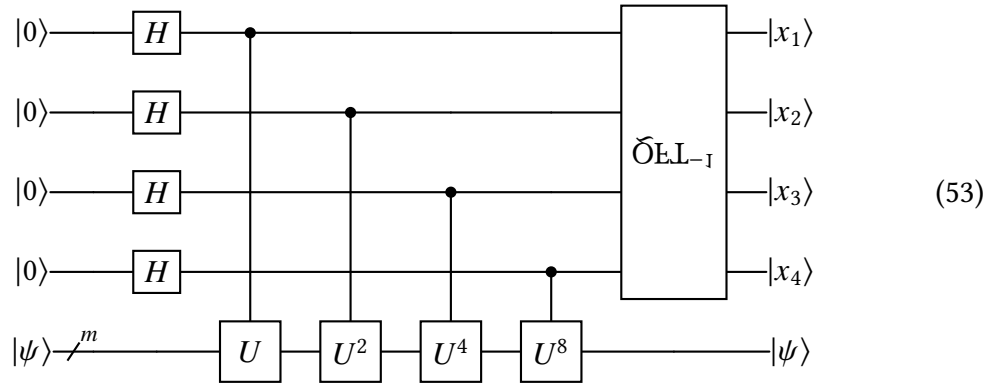


Note how the powers of U are *powers of 2*. Indeed, each power of U corresponds to one bit of ω .

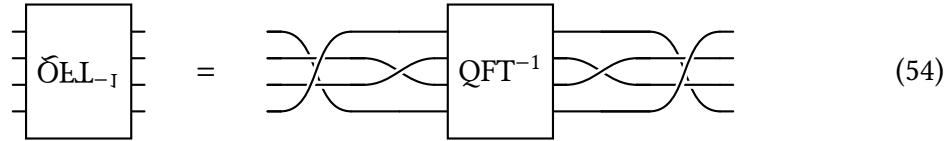
5.6.5 Note how in Circ. (51) the $C-U^k$ commutes: the circuit can be equivalently written as



We can also reorder the top wires, *provided that the circuit QFT^{-1} is written upside down*:



with



5.6.6 Let us write QPE for the unitary realized by the circuit of 5.6.4. In particular

$$\text{QPE}(|0000\rangle \otimes |\psi\rangle) = |x_4 x_3 x_2 x_1\rangle \otimes |\psi\rangle.$$

Note how this operator is linear. Therefore, if $|\phi\rangle$ is another eigenvector with eigvalue $[0.y_1 y_2 y_3 y_4]_2$, then

$$\text{QPE}(|0000\rangle \otimes |\phi\rangle) = |y_4 y_3 y_2 y_1\rangle \otimes |\phi\rangle,$$

and

$$\text{QPE}\left(|0000\rangle \otimes \frac{1}{\sqrt{2}}(|\phi\rangle + |\psi\rangle)\right) = \frac{1}{\sqrt{2}}(|y_4 y_3 y_2 y_1\rangle \otimes |\phi\rangle + |x_4 x_3 x_2 x_1\rangle \otimes |\psi\rangle).$$

If we were to measure the first 4 qubits, we would get both $x_4 x_3 x_2 x_1$ and $y_4 y_3 y_2 y_1$ with probability $1/2$.

5.6.7 Looking at the shape of the circuit in 5.6.4, it might not be immediately clear how this can be efficient. Indeed, for n bits of precision one has to produce U^{2^n} —how doesn't this produce an exponential blowup?

This is one of the reason for which QPE is not a magic bullet: it all depends on the structure of the matrices U^{2^k} : the objective consists in avoiding to blindly replicate many times a circuit for U and instead craft a dedicated circuit for each U^{2^k} . One strategy we shall discuss is Trotterization (Section 5.7) and its use in the algorithm HHL (Section 6.3); another is when the matrices U^{2^k} comes from an oracle: this is discussed in the presentation of Shor's algorithm (Section 6.2).

5.7 Trotterization

5.7.1 A common problem in the implementation of quantum algorithm consists in having to synthesize a circuit representing the unitary e^{itA} , for a given Hermitian matrix A . An often used trick is called the *Trotter-Suzuki* decomposition. It states that when $\delta \rightarrow 0$,

2.9.8

$$e^{i\delta \cdot (A+B)} = e^{i\delta \cdot A} e^{i\delta \cdot B} + O(\delta^2).$$

So realizing a circuit for $e^{i\delta \cdot (A+B)}$ can be reduced to realizing two subcircuits: one for $e^{i\delta \cdot B}$ and one for $e^{i\delta \cdot A}$, and composing them (at the cost of a small error). This is called the *Order 1 decomposition*. There are also higher-order decompositions with smaller errors, but they are out of scope for this course.

5.7.2 When A and B commute: $AB = BA$, the decomposition is exact and we have

$$e^{i\delta \cdot (A+B)} = e^{i\delta \cdot A} e^{i\delta \cdot B}.$$

5.7.3 From the relation in 5.7.1 we can generalize to

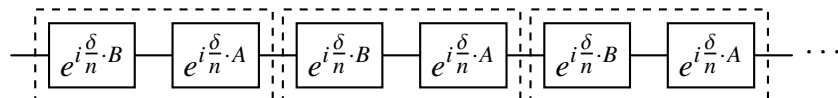
$$e^{i\delta \cdot \sum_{k=0}^n A_k} = \prod_{k=0}^n e^{i\delta \cdot A_k} + O(\delta^2).$$

The main use-case is the decomposition of Hermitian into linear combination of Pauli strings, as discussed in 2.9.13. Indeed, exponentials of Pauli strings are “easy” to synthesize, as discussed starting at 5.7.12.

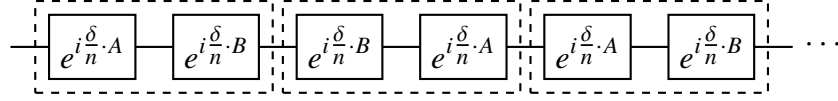
5.7.4 If A and B do not commute, and if one cannot make δ small, one can still make use of the decomposition with the following trick:

$$e^{i\delta \cdot (A+B)} = \left(e^{i\frac{\delta}{n} \cdot A} e^{i\frac{\delta}{n} \cdot B} \right)^n + O\left(\frac{\delta^2}{n}\right)$$

The corresponding circuit is then made of a repetition of n time the same pair of circuits:



5.7.5 The choice of ordering $e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B}$ in 5.7.4 is arbitrary: one could as well as chosen $e^{i\frac{\delta}{n}\cdot B}e^{i\frac{\delta}{n}\cdot A}$ instead. Each iteration can have its own ordering and yield another approximation of $e^{i\delta\cdot(A+B)}$:



5.7.6 The equation in 5.7.4 can be derived using the fact that $A+B$ commutes with itself, as follows.

$$\begin{aligned} e^{i\delta\cdot(A+B)} &= e^{i\frac{\delta}{n}\cdot(A+B)+\dots+i\frac{\delta}{n}\cdot(A+B)} \\ &= \left(e^{i\frac{\delta}{n}\cdot(A+B)}\right)^n \\ &= \left(e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B} + O\left(\frac{\delta^2}{n^2}\right)\right)^n \\ &= \sum_{k=0}^n \binom{n}{k} \cdot \left(e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B}\right)^k \cdot \left(O\left(\frac{\delta^2}{n^2}\right)\right)^{n-k}. \end{aligned}$$

If we extract the terms corresponding to $k = n$ and $k = n - 1$, and if we note that $e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B} = 1 + O\left(\frac{\delta}{n}\right)$, we get

$$\begin{aligned} e^{i\delta\cdot(A+B)} &= \left(e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B}\right)^n + n \cdot \left(1 + O\left(\frac{\delta}{n}\right)\right)^{n-1} \cdot O\left(\frac{\delta^2}{n^2}\right) \\ &\quad + \sum_{k=0}^{n-2} \binom{n}{k} \cdot \left(1 + O\left(\frac{\delta}{n}\right)\right)^k \cdot \left(O\left(\frac{\delta^2}{n^2}\right)\right)^{n-k}. \end{aligned}$$

Realizing that $\binom{n}{k} = \frac{n(n-1)}{(n-k)(n-1-k)} \binom{n-2}{k}$ whenever $k \in \{0 \dots n-2\}$, we can derive that the sum is in $O\left(\frac{\delta^2}{n^2}\right)$, which is $O\left(\frac{\delta^2}{n}\right)$. We also have that $\left(1 + O\left(\frac{\delta}{n}\right)\right)^{n-1} = O(1)$, and that $n \cdot O(1) \cdot O\left(\frac{\delta^2}{n^2}\right) = O\left(\frac{\delta^2}{n}\right)$. We can conclude

$$e^{i\delta\cdot(A+B)} = \left(e^{i\frac{\delta}{n}\cdot A}e^{i\frac{\delta}{n}\cdot B}\right)^n + O\left(\frac{\delta^2}{n}\right).$$

5.7.7 Generalizing 5.7.4, we have

$$e^{i\delta\cdot\sum_{k=0}^n A_k} = \left(\prod_{k=0}^n e^{i\frac{\delta}{N}\cdot A_k}\right)^N + O\left(\frac{\delta^2}{N}\right).$$

5.7.8 As a 2x2 example, consider the matrix

$$A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}.$$

We can decompose A into

$$A = I - \frac{1}{3}X.$$

The matrices I and X commutes, thus

$$e^{i\delta A} = e^{i\delta I}e^{-i\frac{1}{3}\delta X} = e^{i\delta}R_X\left(\frac{2}{3}\delta\right),$$

that is, a rotation around axis X modulo a global phase of δ .

5.7.9 A slightly less trivial 2×2 example is

$$B = \begin{pmatrix} 1 & -1/3 \\ -1/3 & -1 \end{pmatrix} = Z - \frac{1}{3}X$$

since X and Z do not commute anymore. We can then derive that

$$e^{i\delta B} \simeq \left(R_Z \left(-\frac{2}{n}\delta \right) R_X \left(\frac{2}{3n}\delta \right) \right)^n,$$

and the error is in $O\left(\frac{\delta^2}{n}\right)$.

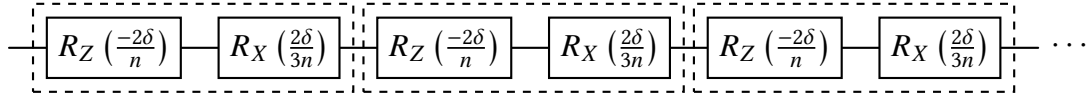
5.7.10 In 5.7.9, we arbitrary decided to decompose each of the n repeated operations

$$e^{i\frac{\delta}{n}Z - i\delta\frac{1}{3n}X}$$

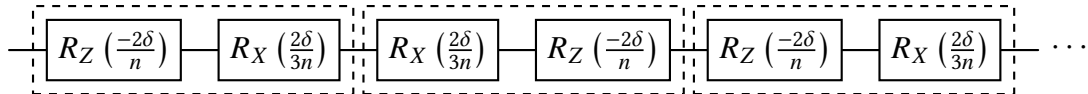
into the same sequence

$$e^{i\frac{\delta}{n}Z} e^{-i\delta\frac{1}{3n}X}.$$

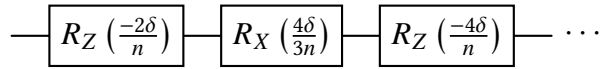
This means that the repetition yields a (long) sequence of R_Z then R_X then R_Z then R_X then R_Z then R_X etc.



As discussed in 5.7.5, the order could have been chosen differently. For instance one can alternate as follows:



This then makes it possible to merge consecutive rotation gates:



5.7.11 If in 5.7.10 we then get a sequence twice as short as in 5.7.9. The convergence of the approximation is however still following a similar asymptotic behavior: the question is whether this actually offers any practical advantage. This is the kind of trade-off that typically needs to be evaluated in real-world quantum compilation scenarios.

In the special case of a 2×2 matrix such as B , one could also consider computing directly an approximation of the matrix $e^{i\delta B}$, and then rely on 3.5.10 to derive a circuit. For larger matrix, one could also rely on a universality result such as 4.3.17 to derive a circuit. The problem is however two-fold. First, in general universality results provide a circuit of size exponential on the size of the matrix. Secondly, the procedure requires to have fixed the value δ in advance, something that might not be desirable in the case of e.g. variational algorithms as in Section 7.

5.7.12 The typical use-case for the Trotter-Suzuki decomposition is when Hermitian matrices are decomposed as linear combinations of Pauli strings. The archetypical example is

$$e^{i\delta Z \otimes Z}$$

Following the same strategy as in 3.5.8, we can decompose this unitary as

$$\cos(\theta) \cdot Id + i \sin(\theta) \cdot Z \otimes Z.$$

The operator $Z \otimes Z$ is the matrix

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

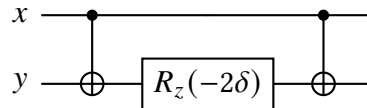
Using the decomposition of $e^{i\delta}$ shown in 2.3.3, $e^{i\delta Z \otimes Z}$ corresponds to the diagonal matrix

$$\begin{pmatrix} \cos(\delta) & 0 & 0 & 0 \\ 0 & \cos(\delta) & 0 & 0 \\ 0 & 0 & \cos(\delta) & 0 \\ 0 & 0 & 0 & \cos(\delta) \end{pmatrix} + i \cdot \begin{pmatrix} \sin(\delta) & 0 & 0 & 0 \\ 0 & -\sin(\delta) & 0 & 0 \\ 0 & 0 & -\sin(\delta) & 0 \\ 0 & 0 & 0 & \sin(\delta) \end{pmatrix} = \begin{pmatrix} e^{i\delta} & 0 & 0 & 0 \\ 0 & e^{-i\delta} & 0 & 0 \\ 0 & 0 & e^{-i\delta} & 0 \\ 0 & 0 & 0 & e^{i\delta} \end{pmatrix}$$

when the basis is in the lexicographic ordering $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. The action of the operator is then

$$\begin{aligned} |00\rangle &\mapsto e^{i\delta} |00\rangle, \\ |01\rangle &\mapsto e^{-i\delta} |01\rangle, \\ |10\rangle &\mapsto e^{-i\delta} |10\rangle, \\ |11\rangle &\mapsto e^{i\delta} |11\rangle. \end{aligned}$$

Said otherwise, its action is a coefficient of $e^{-i\delta}$ when the parity of the bits in the ket is 1, and a coefficient of $e^{i\delta}$ otherwise. A circuit implementing the operator is as follows:



Indeed, the R_z gate fires a negative phase when $x \oplus y = 1$, a positive gate otherwise.

5.7.13 A Pauli string of 3 Z -gates is treated similarly:

$$e^{i\delta Z \otimes Z \otimes Z} = \cos(\theta) \cdot Id + i \sin(\theta) \cdot Z \otimes Z \otimes Z,$$

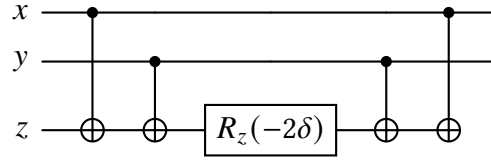
where $Z \otimes Z \otimes Z$ is

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{pmatrix}.$$

The gate $e^{i\delta Z \otimes Z \otimes Z}$ then set the phase to $-\delta$ on

$$|001\rangle, |010\rangle, |100\rangle, |111\rangle,$$

and the phase to δ otherwise. Said otherwise, it fires a negative phase on $|xyz\rangle$ whenever $x \oplus y \oplus z = 1$. This gives the circuit



5.7.14 In the case of a Pauli strings with Id and Z , the structure is simpler: the wires corresponding to Id do not intervene. For instance,

$$e^{i\delta Id \otimes Z} = \cos(\theta) \cdot Id + i \sin(\theta) \cdot Id \otimes Z$$

corresponds to the map

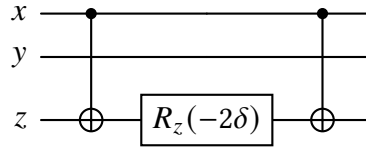
$$\begin{aligned} |00\rangle &\mapsto e^{i\delta} |00\rangle, \\ |01\rangle &\mapsto e^{-i\delta} |01\rangle, \\ |10\rangle &\mapsto e^{i\delta} |10\rangle, \\ |11\rangle &\mapsto e^{-i\delta} |11\rangle. \end{aligned}$$

This is simply a gate $R_z(-2\delta)$ on the second qubit.

5.7.15 Similarly,

$$e^{i\delta Z \otimes Id \otimes Z}$$

is synthesized by the circuit



The middle wire is mute: the behavior is otherwise the same as in [5.7.12](#).

5.7.16 If Pauli strings might contains more than Id and Z , one can however reduce the problem to this case. Indeed, the following equalities

$$X = \text{HAD} \cdot Z \cdot \text{HAD}, \quad Y = (S \cdot \text{HAD}) \cdot Z \cdot (\text{HAD} \cdot S^*),$$

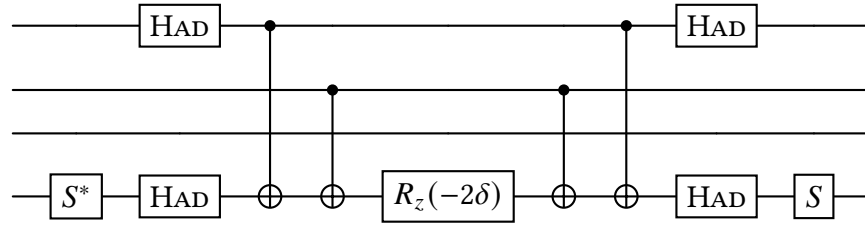
carries over to their exponential counterparts thanks to [2.9.11](#):

$$e^{i\delta X} = \text{HAD} \cdot e^{i\delta Z} \cdot \text{HAD}, \quad e^{i\delta Y} = (S \cdot \text{HAD}) \cdot e^{i\delta Z} \cdot (\text{HAD} \cdot S^*).$$

They also generalize to Pauli strings. For instance, we have

$$e^{i\delta X \otimes Z \otimes Id \otimes Y} = (\text{HAD} \otimes Id \otimes Id \otimes (S \cdot \text{HAD})) \cdot e^{i\delta Z \otimes Z \otimes Id \otimes Z} \cdot (\text{HAD} \otimes Id \otimes Id \otimes (\text{HAD} \cdot S^*)),$$

which is then realized by the circuit



5.7.17 Let us now build a full example: a circuit for e^{iA} , where

$$A = \begin{pmatrix} 0 & 0 & 3 & 5 \\ 0 & 0 & 7 & 9 \\ 3 & 7 & 0 & 0 \\ 5 & 9 & 0 & 0 \end{pmatrix}.$$

The matrix A can be written as a linear combination of

$$\begin{aligned} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} &= \frac{1}{2}X \otimes (I + Z), & \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} &= \frac{1}{2}X \otimes (I - Z), \\ \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} &= \frac{1}{2}(X \otimes X + Y \otimes Y), & \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} &= \frac{1}{2}(X \otimes X - Y \otimes Y), \end{aligned}$$

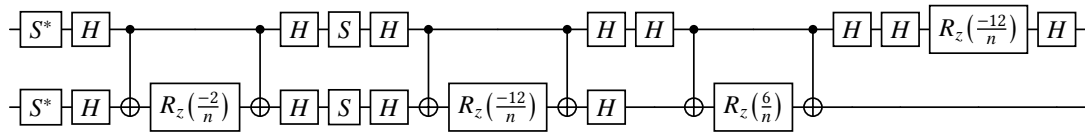
yielding the decomposition into Pauli strings:

$$A = 6X \otimes I - 3X \otimes Z + 6X \otimes X + Y \otimes Y.$$

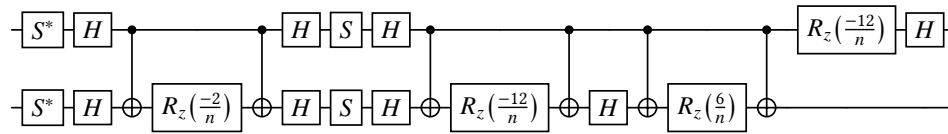
The Trotter-Suzuki decomposition of e^{iA} states that

$$e^{iA} \simeq \left(e^{\frac{6}{n}iX \otimes I} e^{-\frac{3}{n}iX \otimes Z} e^{\frac{6}{n}iX \otimes X} e^{\frac{1}{n}iY \otimes Y} \right)^n$$

where we somehow arbitrarily decided on the ordering of the Pauli strings. The corresponding circuit is a repetitions of n times:



which can be made slightly simpler by removing the consecutive Hadamard gates:



5.8 Exercises

5.8.1 Consider a n -qubit register, and identify the n -bitstring $x_1 \cdots x_n$ with the binary representation of an integer $x \in \{0, \dots, n\}$ in big-endian notation (remember 5.3.2). Using controlled-rotations, implements the operation

$$|x_1 \cdots x_n\rangle \mapsto e^{2i\pi \frac{x}{2^n}} |x\rangle.$$

Explain how it works.

5.8.2 Using Trotter-Suzuki, derive a circuit approximating e^{itA} , for t a real number and A the hermitian operator (when the basis is ordered lexicographically):

$$A = \begin{pmatrix} 5 & 9 & 0 & 0 \\ 9 & 5 & 9 & 0 \\ 0 & 9 & 5 & 9 \\ 0 & 0 & 9 & 5 \end{pmatrix}$$

6 Algorithms for LSQ era

6.1 Simple Oracle-Based Algorithms

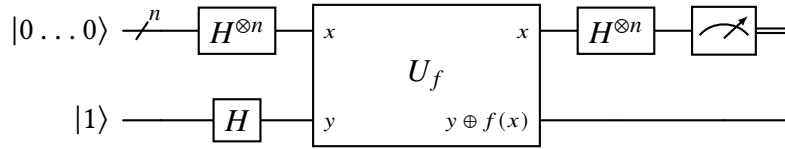
6.1.1 Deutsch-Josza algorithm. Suppose that we are given the set-function

$$f : \text{bool}^n \rightarrow \text{bool},$$

with the promise that f is either constant or balanced (i.e. the sets of inputs mapping to 0 and 1 are of equal size). We are looking for an algorithm deciding on the status of f : is it constant or balanced? The catch is that f is given as a *blackbox*: we only know how to call f , and we don't have any information on how it is built. For instance, you can consider f as a call to an external server. In such an *oracle-based* algorithm, we care about the complexity in term of calls to the oracle (the function f).

6.1.2 With a classical algorithm, the only thing we can do is call f repeatedly on various inputs. We need $2^{n-1} + 1$ calls to f : we might be very unlucky, and our classical procedure might only pick the inputs mapping to the same Boolean value for the 2^{n-1} first calls. So we really need one more to be sure that the function is indeed constant or balanced.

6.1.3 In the quantum case, we rely on the trick discussed in Section 5.2.1, and instead of f we use U_f . Deutsch-Josza algorithm is very simple: run the circuit below, and measure the n first qubits. f est constant if $|0\dots 0\rangle$ was measured, and balanced otherwise. This requires one single run of the algorithm!



Of course, one could argue that this changes the algorithm since we somehow have to *build* the sub-circuit U_f out of f . But for the purpose of the measure of the oracle-complexity, this is irrelevant.

6.1.4 Case $n = 1$. Let us compute what happens when $n = 1$. There are 4 possible functions f : The two constant functions of value 0 and 1, the identity and the bit-flip. Originally, the state is

$$|0\rangle \otimes |1\rangle.$$

After applying the Hadamard gates, we get

$$\begin{aligned} & \frac{1}{2} (|0\rangle + |1\rangle) \otimes (|0\rangle - |1\rangle) \\ &= \frac{1}{2} (|00\rangle + |10\rangle - |01\rangle - |11\rangle). \end{aligned}$$

The oracle is applied:

$$\frac{1}{2} \begin{pmatrix} |0\rangle \otimes |0 \oplus f(0)\rangle + |1\rangle \otimes |0 \oplus f(1)\rangle \\ - |0\rangle \otimes |1 \oplus f(0)\rangle - |1\rangle \otimes |1 \oplus f(1)\rangle \end{pmatrix}$$

$$= \frac{1}{2} \begin{pmatrix} |0\rangle \otimes |f(0)\rangle + |1\rangle \otimes |f(1)\rangle \\ - |0\rangle \otimes |1 \oplus f(0)\rangle - |1\rangle \otimes |1 \oplus f(1)\rangle \end{pmatrix},$$

followed by yet another Hadamard gate on the first wire:

$$\begin{aligned} & \frac{1}{2\sqrt{2}} \begin{pmatrix} (|0\rangle + |1\rangle) \otimes |f(0)\rangle + (|0\rangle - |1\rangle) \otimes |f(1)\rangle \\ - (|0\rangle + |1\rangle) \otimes |1 \oplus f(0)\rangle - (|0\rangle - |1\rangle) \otimes |1 \oplus f(1)\rangle \end{pmatrix} \\ &= \frac{1}{2\sqrt{2}} \begin{pmatrix} |0\rangle \otimes |f(0)\rangle + |0\rangle \otimes |f(1)\rangle \\ - |0\rangle \otimes |1 \oplus f(0)\rangle - |0\rangle \otimes |1 \oplus f(1)\rangle \\ + |1\rangle \otimes |f(0)\rangle - |1\rangle \otimes |f(1)\rangle \\ - |1\rangle \otimes |1 \oplus f(0)\rangle + |1\rangle \otimes |1 \oplus f(1)\rangle \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes \frac{1}{2} (|f(0)\rangle + |f(1)\rangle - |1 \oplus f(0)\rangle - |1 \oplus f(1)\rangle) \\ + |1\rangle \otimes \frac{1}{2} (|f(0)\rangle - |f(1)\rangle - |1 \oplus f(0)\rangle + |1 \oplus f(1)\rangle) \end{pmatrix}. \end{aligned} \quad (55)$$

6.1.5 Case $n = 1$, with constant f . Assume f is constant: there is some Boolean value b such that $f(x) = b$ for all x . The formula in Eq. (55) becomes

$$\begin{aligned} & \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes \frac{1}{2} (|b\rangle + |b\rangle - |1 \oplus b\rangle - |1 \oplus b\rangle) \\ + |1\rangle \otimes \frac{1}{2} (|b\rangle - |b\rangle - |1 \oplus b\rangle + |1 \oplus b\rangle) \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes \frac{1}{2} (|b\rangle + |b\rangle - |1 \oplus b\rangle - |1 \oplus b\rangle) \\ + |1\rangle \otimes \frac{1}{2} (0 - 0) \end{pmatrix} \\ &= |0\rangle \otimes \frac{1}{\sqrt{2}} (|b\rangle - |1 \oplus b\rangle). \end{aligned}$$

Measuring the first qubit yield 0 with probability 1, as claimed in Sec. 6.1.3.

6.1.6 Case $n = 1$, with non-constant f . If the function f is not constant, then it is either the identity or the bit flip. In both cases, we have $f(1) = 1 \oplus f(0)$. The formula in Eq. (55) then becomes

$$\begin{aligned} & \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes \frac{1}{2} \begin{pmatrix} |f(0)\rangle + |1 \oplus f(0)\rangle \\ - |1 \oplus f(0)\rangle - |1 \oplus 1 \oplus f(0)\rangle \end{pmatrix} \\ + |1\rangle \otimes \frac{1}{2} \begin{pmatrix} |f(0)\rangle - |1 \oplus f(0)\rangle \\ - |1 \oplus f(0)\rangle + |1 \oplus 1 \oplus f(0)\rangle \end{pmatrix} \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes \frac{1}{2} \begin{pmatrix} |f(0)\rangle + |1 \oplus f(0)\rangle \\ - |1 \oplus f(0)\rangle - |f(0)\rangle \end{pmatrix} \\ + |1\rangle \otimes \frac{1}{2} \begin{pmatrix} |f(0)\rangle - |1 \oplus f(0)\rangle \\ - |1 \oplus f(0)\rangle + |f(0)\rangle \end{pmatrix} \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} |0\rangle \otimes 0 \\ + |1\rangle \otimes \frac{1}{2} (2|f(0)\rangle - 2|1 \oplus f(0)\rangle) \end{pmatrix} \\ &= |1\rangle \otimes \frac{1}{\sqrt{2}} (|f(0)\rangle - |1 \oplus f(0)\rangle). \end{aligned}$$

Measuring the first qubit, we obtain 1 with probability 1.

6.1.7 Generalization to any n Through the Hadamard, the state $|0 \dots 0\rangle \otimes |1\rangle$ is sent to

$$\frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k=0}^{2^n-1} |k\rangle \otimes |0\rangle - \sum_{k=0}^{2^n-1} |k\rangle \otimes |1\rangle \right)$$

and the action of the oracle gives

$$\frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k=0}^{2^n-1} |k\rangle \otimes |f(k)\rangle - \sum_{k=0}^{2^n-1} |k\rangle \otimes |1 \oplus f(k)\rangle \right). \quad (56)$$

If f is constant of Boolean value b , we get

$$\begin{aligned} & \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k=0}^{2^n-1} |k\rangle \otimes |b\rangle - \sum_{k=0}^{2^n-1} |k\rangle \otimes |1 \oplus b\rangle \right) \\ &= \frac{1}{\sqrt{2^n}} \left(\sum_{k=0}^{2^n-1} |k\rangle \right) \otimes \frac{1}{\sqrt{2}} (|b\rangle - |1 \oplus b\rangle), \end{aligned}$$

and the last tower of Hadamard yield

$$|0 \dots 0\rangle \otimes \frac{1}{\sqrt{2}} (|b\rangle - |1 \oplus b\rangle).$$

Measuring, we indeed get 00000..000

6.1.8 Now, if f were balanced, we can partition the set of indices $\{0 \dots 2^n - 1\}$ into $S_0 \cup S_1$, with $S_0 \cap S_1 = \emptyset$, with $f(x) = b$ whenever $x \in S_b$. From $|0 \dots 0\rangle \otimes |1\rangle$, applying the Hadamard gates and the oracle, we get

$$\begin{aligned} & \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k=0}^{2^n-1} |k\rangle \otimes |f(k)\rangle - \sum_{k=0}^{2^n-1} |k\rangle \otimes |1 \oplus f(k)\rangle \right) \\ &= \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k \in S_0} |k\rangle \otimes |f(k)\rangle - \sum_{k \in S_0} |k\rangle \otimes |1 \oplus f(k)\rangle + \sum_{k \in S_1} |k\rangle \otimes |f(k)\rangle - \sum_{k \in S_1} |k\rangle \otimes |1 \oplus f(k)\rangle \right) \\ &= \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{k \in S_0} |k\rangle \otimes |0\rangle - \sum_{k \in S_0} |k\rangle \otimes |1\rangle + \sum_{k \in S_1} |k\rangle \otimes |1\rangle - \sum_{k \in S_1} |k\rangle \otimes |0\rangle \right) \\ &= \frac{1}{\sqrt{2^n}} \left(\sum_{k \in S_0} |k\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) + \sum_{k \in S_1} |k\rangle \otimes \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle) \right) \\ &= \frac{1}{\sqrt{2^n}} \left(\sum_{k \in S_0} |k\rangle - \sum_{k \in S_1} |k\rangle \right) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{\sqrt{2^n}} \left(\sum_{k \in S_0} (-1)^{f(k)} |k\rangle + \sum_{k \in S_1} (-1)^{f(k)} |k\rangle \right) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{\sqrt{2^n}} \left(\sum_{k=0}^{2^n-1} (-1)^{f(k)} |k\rangle \right) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \end{aligned}$$

Yielding, after the last Hadamard gates, to

$$|\psi\rangle \otimes \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle)$$

with

$$|\psi\rangle = \frac{1}{2^n} \sum_{k=0}^{2^n-1} (-1)^{f(k)} \sum_{j=0}^{2^n-1} (-1)^{j \odot k} |j\rangle = \frac{1}{2^n} \sum_{j=0}^{2^n-1} \left(\sum_{k=0}^{2^n-1} (-1)^{f(k)+j \odot k} \right) |j\rangle$$

since $H^{\otimes n} |k\rangle = \sum_{j=0}^{2^n-1} (-1)^{j \odot k} |j\rangle$ (identifying indices with bitstrings, recall 2.10.8). We are interested in the coefficient of $|0 \dots 0\rangle$: it corresponds to $j = 0$, so it is

$$\left(\sum_{k=0}^{2^n-1} (-1)^{f(k)+0 \odot k} \right) = \left(\sum_{k=0}^{2^n-1} (-1)^{f(k)} \right)$$

and since f is balanced, there are as many k for which $f(k) = 0$ as there are for which $f(k) = 1$: the coefficient is 0. Thus, we if were to measure $|\psi\rangle$, we cannot obtain $0 \dots 0$ since the corresponding probability is 0.

6.1.9 Bernstein-Vazirani. An algorithm following the same structure is *Bernstein-Vazirani*. The idea is the same as for Deutsch-Josza, except that this time you are told that the function f acting on a bitstring of size n is of the form $f(x_1, \dots, x_n) = (a_1 \dots a_n) \oplus (x_1 \dots x_n) = a_1 \wedge x_1 \oplus \dots \oplus a_n \wedge x_n$, for an unknown bitstring a . The question is to figure out this bitstring a hidden in the oracle. With a classical algorithm, n calls to f are needed... With Bernstein-Vazirani only one is needed!

6.1.10 The circuit is literally the same as for Deutsch-Josza, shown in Sec. 6.1.3. Right before the last tower of Hadamard, according to Eq. (56) in Sec. 6.1.7 the state is

$$\frac{1}{\sqrt{2^n}} \left(\sum_{k=0}^{2^n-1} (-1)^{f(k)} |k\rangle \right) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle).$$

But now $f(k) = a \odot k$: when we apply the last tower of Hadamard (see Exercise 2.10.8), we get

$$|a\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle),$$

and measuring gives back the bitstring a .

6.2 Shor

6.2.1 Maybe the first work to have placed the subject of quantum algorithms on the table is Shor's factoring algorithm [Sho97]. Although it is not known whether it is NP-complete or not, the fact is that we do not know of any classical algorithm able to factor a number $N = pq$ (p and q prime numbers) coded on n digit in a time polynomial on n . The problem is deemed hard enough that it is at the root of the main cryptographic protocol used to encrypt data over the internet: RSA.

FACTORIZATION	
Input	A number N , product of two unknown primes.
Output	A divisor of N .
ORDERFINDING	
Input	Two integers N and a , co-primes.
Output	The period r of a , i.e. the smallest $r > 0$ such that $a^r \equiv 1 \pmod{N}$.
PHASEESTIMATION	
Input	A unitary U and an eigenvector $ \phi\rangle$.
Output	The corresponding eigenvalue.

Table 3: Each problem is reduced to the one below.

6.2.2 Provided that we have at disposal a quantum co-processor with a large enough memory holding stable logical quantum qubits, factoring is at reach in polynomial time (at least theoretically). The algorithm is based on the concept —standard in complexity theory— of *polynomial reduction*: the “difficult” part of factorization is encoded in another problem that we know how to solve. This process is done in two steps: Factorisation is first reduced to the problem of order finding, and order finding is itself reduced to the problem of phase estimation: see Table 3.

6.2.3 Step 1: Reducing FACTORIZATION to ORDERFINDING. This really means: “If I know how to (efficiently) solve ORDERFINDING, I can easily factor an integer N ”. In fact, this part of the algorithm is purely classical, relying on mathematical properties. Suppose that I can solve ORDERFINDING. I am given $N = pq$ to factor. The algorithm proceeds as follows.

1. Select a number $1 < a < N$ at random. If it is not co-prime with N , we are done: we have a non-trivial factor.
2. Otherwise, it is co-prime with N . We then invoke our algorithm for ORDERFINDING. It outputs the smallest number r such that $a^r = 1 \pmod{N}$.
3. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, proceed to Step 4. Otherwise, start over at Step 1. Because of mathematical properties that are out of scope, this happens with probability greater or equal to $1/2$ [NC02, Appendix A].
4. We have $a^r = (a^{r/2})^2 = 1 \pmod{N}$, so $(a^{r/2} - 1)(a^{r/2} + 1) = 0 \pmod{N}$. Thus, N divides $(a^{r/2} - 1)(a^{r/2} + 1)$.
5. N cannot divide $a^{r/2} + 1$ since we assumed that $a^{r/2} \not\equiv -1 \pmod{N}$. It cannot divide $a^{r/2} - 1$ either since that would make $a^{r/2} = 1 \pmod{N}$, and r was supposed to be the smallest such integer. Therefore, the only remaining possibility is that one of the factors of N divide $a^{r/2} - 1$, and the other $a^{r/2} + 1$.
6. These factors can be computed with $\gcd(N, a^{r/2} \pm 1)$.

6.2.4 Step 2: Solving ORDERFINDING using PHASEESTIMATION. Note that if a and N are co-primes, then $x \mapsto a \cdot x \bmod N$ is a reversible function (it is a permutation of $\{0 \dots N-1\}$, see 2.10.3). Build the unitary $U_a : |x\rangle \mapsto |a \cdot x \bmod N\rangle$ (multiplication modulo N) According to what we discussed in 5.2, we know that we can implement it efficiently. We claim that PHASEESTIMATION can be used on U_a to recover the order of a modulo N .

6.2.5 The claim is that a suitable eigenvector of U_a has an eigenvalue from which the order of a modulo N can be recovered. Let us consider several possibilities.

- $U_a |0 \dots 0\rangle = |0 \dots 0\rangle$: the vector $|0 \dots 0\rangle$ is an eigenvector of eigenvalue 1. We cannot derive anything from this.
- Let us compute:

$$\begin{aligned}
 U_a \left(\frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} |a^k \bmod N\rangle \right) &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} |a^{(k+1)} \bmod N\rangle \\
 &= \frac{1}{\sqrt{r}} \sum_{k=1}^r |a^k \bmod N\rangle \\
 &= \frac{1}{\sqrt{r}} (|a^r \bmod N\rangle + \sum_{k=1}^{r-1} |a^k \bmod N\rangle) \\
 &= \frac{1}{\sqrt{r}} (|1 \bmod N\rangle + \sum_{k=1}^{r-1} |a^k \bmod N\rangle) \\
 &= \frac{1}{\sqrt{r}} (|a^0 \bmod N\rangle + \sum_{k=1}^{r-1} |a^k \bmod N\rangle) \\
 &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} |a^k \bmod N\rangle
 \end{aligned}$$

We have another eigenvector of U_a , with eigenvalue 1. If this is still useless, we can nonetheless use this technique and add a phase to the various elements in the sum, in 6.2.6

6.2.6 Fix $s \in \{0, \dots, r-1\}$ and define

$$|\phi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2i\pi \frac{s \cdot k}{r}} |a^k \bmod N\rangle$$

Let us compute:

$$\begin{aligned}
 U_a |\phi_s\rangle &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2i\pi \frac{s \cdot k}{r}} U_a |a^k \bmod N\rangle \\
 &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2i\pi \frac{s \cdot k}{r}} |a^{(k+1)} \bmod N\rangle \\
 &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2i\pi \frac{s \cdot (k-1)}{r}} |a^k \bmod N\rangle \\
 &= \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{2i\pi \frac{s}{r}} e^{-2i\pi \frac{s \cdot k}{r}} |a^k \bmod N\rangle \\
 &= e^{2i\pi \frac{s}{r}} |\phi_s\rangle
 \end{aligned}$$

This eigenvalue is better since it contains a phase parametrized by r . As QPE gives us the phase, with some luck we can retrieve r out. U_a have r such eigenvectors, one for each value of s between 0 and $r - 1$. We would just have to use the fact that

$$|0\dots 0\rangle \otimes |\phi_s\rangle \xrightarrow{QPE(U_a)} |x_1\dots x_n\rangle \otimes |\phi_s\rangle$$

where $x_1\dots x_n$ is a binary representation of the phase of the eigenvalue corresponding to $|\phi_s\rangle$, from which one could infer r .

6.2.7 The problem is that one cannot directly use these eigenvectors $|\phi_s\rangle$ since we need to know r to construct them. However, what we can do is use the fact that the QPE is a linear map, so we can place them in superposition:

$$|0\dots 0\rangle \otimes (\alpha |\phi_s\rangle + \beta |\phi_{s'}\rangle) \xrightarrow{QPE(U_a)} \alpha |x_1\dots x_n\rangle \otimes |\phi_s\rangle + \beta |y_1\dots y_n\rangle \otimes |\phi_{s'}\rangle$$

(where $y_1\dots y_n$ is a binary representation of the phase of the eigenvalue corresponding to $|\phi_{s'}\rangle$). The trick consists in realizing that if we place them all in (equal) superposition, as follows:

$$\begin{aligned} \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\phi_s\rangle &= \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2i\pi \frac{s*k}{r}} |a^k \bmod N\rangle \\ &= \frac{1}{r} \sum_{k=0}^{r-1} \sum_{s=0}^{r-1} e^{-2i\pi \frac{s*k}{r}} |a^k \bmod N\rangle \\ &= \frac{1}{r} \sum_{k=0}^{r-1} \left(\sum_{s=0}^{r-1} e^{-2i\pi \frac{s*k}{r}} \right) |a^k \bmod N\rangle \end{aligned} \quad (57)$$

then the inner (red) sum is equal to r if $k = 0$, and 0 otherwise (because it is a sum of all of the roots of unity). Therefore, in Eq (57) all the terms are nul except when $k = 0$: we get

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\phi_s\rangle = \frac{1}{r} (r |a^0 \bmod N\rangle) = |1\rangle_n$$

(where the encoding of 1 on n qubits is $|0\dots 01\rangle$).

If we run $QPE(U_a)$ on this input, we “compute” all of the phases at once. Consider two registers, the first one for retrieving the ω of the eigenvalue and the second one for the eigenvector. We have

$$QPE(U_a) (|0\dots 0\rangle \otimes |\phi_s\rangle) = |s/r\rangle \otimes |\phi_s\rangle$$

where by $|s/r\rangle$ we mean the approximation of s/r over the corresponding number of qubits. Then

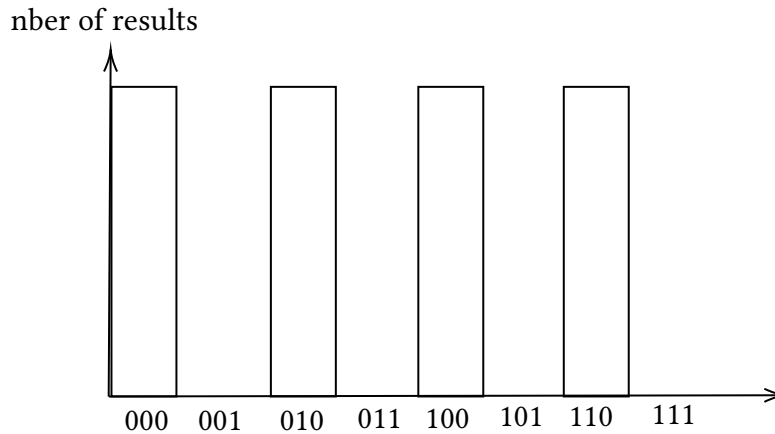
$$\begin{aligned} QPE(U_a) \left(|0\dots 0\rangle \otimes \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\phi_s\rangle \right) &= \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} QPE(U_a) (|0\dots 0\rangle \otimes |\phi_s\rangle) \\ &= \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |s/r\rangle \otimes |\phi_s\rangle \end{aligned}$$

Measuring the first register, we get one of the $\frac{s}{r}$ (for the sake of the discussion, assume that the decomposition on n bits is exact)

For instance, if $r = 4$, there are exactly 4 elements in the sum: measuring, we get $0/4$, $1/4$, $2/4$ and $3/4$.

- On 2 bits, this is 00, 01, 10 and 11.
- On 3 bits, this is 000, 010, 100 and 110 (since $0.x_1x_2x_3 = \frac{x_1}{2} + \frac{x_2}{4} + \frac{x_3}{8}$)

If we were to perform many measurements (for 3 bits) and collecting the results, we would get the following plot



with equiprobable results 000, 010, 100 and 110. If the decomposition were not exact (for instance when $r = 3$), we would instead get a less precise plot with 3 peaks but not as sharp. Possibly then 3 bits would not be enough to distinguish them, and we would need to get to 5 or 6 bits of precision.

In any case, when the precision is high enough, one can “read out” the period r of $a \bmod N$ from the plot, if we were to run enough computation.

6.2.8 However, in a concrete use-case we cannot afford to perform enough computations to draw such a plot. Instead, Shor’s algorithm is run once, and then one retrieves a possible estimate for s/r , one uses the algorithm of continued fractions.¹⁰ to get a putative value r , and one tests whether this gives a factor of N . If not, we start over. With a high-enough probability, this succeeds.

6.3 HHL

6.3.1 A slightly more involved algorithm relying on QPE is *HHL*, initials of the author’s names: *Harrow, Hassidim, Lloyd* [HHL09]. This algorithm solves a linear system of equation with a complexity arguably better than the one offered by classical algorithms.

¹⁰See e.g. https://en.wikipedia.org/wiki/Continued_fraction#Best_rational_approximations

6.3.2 The problem can be stated as follows. Consider an hermitian matrix A and a vector $\vec{b}$: we want to solve the equation

$$A \cdot \vec{x} = \vec{b}.$$

Note that if A were not hermitian, we can reduce the problem to the case where the matrix is

$$\begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}$$

(see 6.4.1)

6.3.3 To make use of a quantum co-processor, the idea is to code the vectors $\vec{b}$ and $\vec{x}$ as the coefficients of a ket-vector. For instance assume that $\vec{b}$ is the vector

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix}.$$

The vector is stored in a 2-qubit register as

$$|b\rangle = b_0 |00\rangle + b_1 |01\rangle + b_2 |10\rangle + b_3 |11\rangle$$

modulo some renormalization. In the rest, we assume that $\vec{b}$ is of norm 1.

6.3.4 Complexity-wise, the algorithm is better than classical ones in the following sense. If

- N is the size of the system
- s is the number of non-zero elements in a line of A
- κ is the condition number of A (i.e. the ration between the largest and the smallest eigenvalue of A)
- ϵ the allowed error

then the complexity are

- In the classical case: $\mathcal{O}(Ns\kappa \log(1/\epsilon))$
- In the quantum case: $\mathcal{O}(\log(N)s^2\kappa^2/\epsilon)$ for HHL.

In summary, we get an exponential gain with respect to the size of the matrix. In theory.

6.3.5 General idea. Since the matrix A is hermitian, according to 2.9.4 it can be written as

$$A = \sum_{j=0}^{N-1} \lambda_j |u_j\rangle \langle u_j|$$

with the λ_j s being real numbers and $\{|u_j\rangle\}_j$ an orthonormal basis. Provided that none of the λ_j are zero (which would make the condition number infinite), we can therefore safely consider that A admits an inverse, and in fact

$$A^{-1} = \sum_{j=0}^{N-1} \lambda_j^{-1} |u_j\rangle \langle u_j|$$

Now, $|\vec{b}\rangle$ can be decompose in the basis $\{|u_j\rangle\}_j$. For instance,

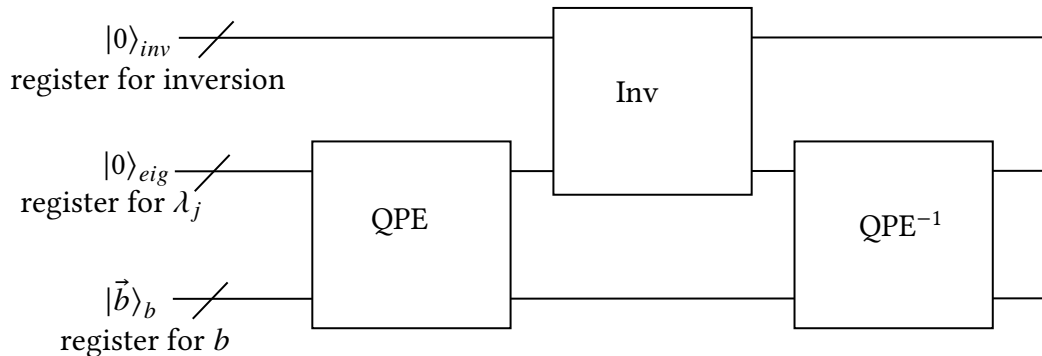
$$|\vec{b}\rangle = \sum_{j=0}^{N-1} b_j |u_j\rangle$$

with $b_j \in \mathbb{C}$. We can then write $|x\rangle$ as

$$|\vec{x}\rangle = A^{-1} |\vec{b}\rangle = \sum_{j=0}^{N-1} \lambda_j^{-1} b_j |u_j\rangle.$$

6.3.6 Can we consider that we solve the problem? If this mathematical development gives us a formal presentation of $|x\rangle$, it does not say how to *compute* the various pieces: it only say that “they exist” and that when combined they give a solution $|x\rangle$ to the problem. The objective of the algorithm HHL is to provide a computational mean to attain such a $|x\rangle$. It is however important to emphasize right away that HHL will *not* derive the $|u_j\rangle$ s, the b_i s and the λ_j s. It will only rely on implicit mechanisms that manipulate them, without having to spell them out. At the end of the computation, a register will be set in state $|x\rangle$, solution to the problem. But everything will happen implicitly.

6.3.7 Structure of the circuit. We need two parameters, real values to calibrate the system: t and C . Their mean will be explained later on. The structure of the circuit is as follows.



The register inv contains a single qubit. The register b contains n bits, when $N = 2^n$. The size of the register eig depends on the desired precision.

The sub-circuit QPE stands for the quantum phase estimation as in 5.6.1, applied to the unitary e^{itA} . The parameter t aims at ensuring that the eigenvalues of tA are “not too large” and that QPE succeeds. The subcircuit Inv stands for any circuit implementing the operation

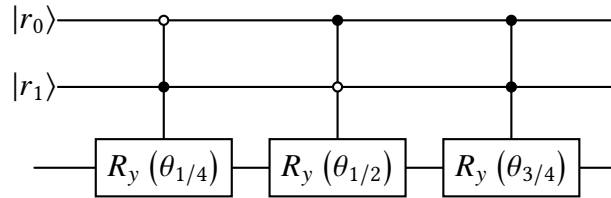
$$\text{Inv} : |0\rangle_{\text{inv}} \otimes |r\rangle_{\text{eig}} \mapsto \left(\sqrt{1 - \frac{C^2}{r^2}} |0\rangle_{\text{inv}} + \frac{C}{r} |1\rangle_{\text{inv}} \right) \otimes |r\rangle_{\text{eig}}$$

where C is chosen small enough so that

- this makes sense: we need $\frac{C}{r}$ to be within 0 and 1 for the values of r we care about (see below).
- yet the amplitudes corresponding to the subspace $|1\rangle_{\text{inv}}$ are as large as possible (so that Step 5 in 6.3.10 succeeds with the highest probability).

6.3.8 The operation Inv . If we define the angle θ_r as $2 \arcsin(C/r)$, the action of Inv is “just” a rotation $R_y(\theta_r)$ parameterized by r . In 5.2, instead of a rotation R_y the action was a bit-flip: we can suggest two methods to realize a circuit for Inv replacing the bit-flip with suitable rotations. 3.5.7

Following the strategy in 5.2.6, if the eig register holds two qubits, and if we consider that $|r_0 r_1\rangle$ corresponds to the real value $\frac{r_0}{2} + \frac{r_1}{4}$, the value r can take 4 values: 0, $1/4$, $1/2$ and $3/4$, corresponding respectively to the states $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. The circuit is then



The size of the circuit is however exponential on the size of the register eig . If we can afford auxiliary qubits, one can rely on the structure of the function θ to build a circuit V_θ as in 5.2.8, and instead produce a circuit of size polynomial on the size of the register eig (albeit with a high overhead).

6.3.9 Beware! The problem is always the same: the encoding of natural numbers (or, for that matter, real numbers) on bitstrings relies on seemingly arbitrary conventions (see 5.3.2). When implementing an algorithm, one has to be careful about choosing the same convention for all of its subparts. Here, we have to choose the same ones for QPE and Inv .

6.3.10 Overview of the algorithm.

1. At first we have $|0\rangle_{\text{inv}} \otimes |0\rangle_{\text{eig}} \otimes |\vec{b}\rangle_b = \sum_j b_j |0\rangle_{\text{inv}} \otimes |0\rangle_{\text{eig}} \otimes |u_j\rangle_b$

2. We apply QPE with the matrix

$$U = e^{iAt} = \sum_{j=0}^{N-1} e^{i\lambda_j t} |u_j\rangle \langle u_j| = \sum_{j=0}^{N-1} e^{2i\pi \frac{\lambda_j t}{2\pi}} |u_j\rangle \langle u_j|$$

(remember: QPE recovers the phase ω of an eigenvalue $e^{2i\pi\omega}$).

Note how we use here the calibration parameter t . Its purpose is to adjust the values λ_j s to have them fit inside the register eig and minimizing the error.

Assuming that there are no precision error, we now have

$$\sum_j b_j |0\rangle_{inv} \otimes \left| \frac{\lambda_j t}{2\pi} \right\rangle_{eig} \otimes |u_j\rangle_b$$

3. We then use the sub-circuit Inv discussed in 6.3.7 and whose action is

$$\text{Inv: } |0\rangle_{inv} \otimes |r\rangle_{eig} \mapsto \left(\sqrt{1 - \frac{C^2}{r^2}} |0\rangle_{inv} + \frac{C}{r} |1\rangle_{inv} \right) \otimes |r\rangle_{eig}$$

This uses the second calibration parameter: the value r is potentially very small: we need to renormalize to get to a number between 0 and 1.

In any case, after the action of Inv, we have

$$\sum_j b_j \left(\sqrt{1 - \frac{(2\pi C)^2}{(\lambda_j t)^2}} |0\rangle_{inv} + \frac{2\pi C}{\lambda_j t} |1\rangle_{inv} \right) \otimes \left| \frac{\lambda_j t}{2\pi} \right\rangle_{eig} \otimes |u_j\rangle_b$$

4. We then apply the inverse of the first QPE circuit: this uncomputes the $|\lambda_j t\rangle$ s. We have

$$\sum_j b_j \left(\sqrt{1 - \frac{(2\pi C)^2}{(\lambda_j t)^2}} |0\rangle_{inv} + \frac{2\pi C}{\lambda_j t} |1\rangle_{inv} \right) \otimes |0\rangle_{eig} \otimes |u_j\rangle_b$$

which is

$$\begin{aligned} & \sum_j b_j \sqrt{1 - \frac{(2\pi C)^2}{(\lambda_j t)^2}} |0\rangle_{inv} \otimes |0\rangle_{eig} \otimes |u_j\rangle_b + \sum_j b_j \frac{2\pi C}{\lambda_j t} |1\rangle_{inv} \otimes |0\rangle_{eig} \otimes |u_j\rangle_b \\ &= \sum_j b_j \sqrt{1 - \frac{(2\pi C)^2}{(\lambda_j t)^2}} |0\rangle_{inv} \otimes |0\rangle_{eig} \otimes |u_j\rangle_b + \frac{2\pi C}{t} \sum_j \frac{b_j}{\lambda_j} |1\rangle_{inv} \otimes |0\rangle_{eig} \otimes |u_j\rangle_b \\ &= \sum_j b_j \sqrt{1 - \frac{(2\pi C)^2}{(\lambda_j t)^2}} |0\rangle_{inv} \otimes |0\rangle_{eig} \otimes |u_j\rangle_b + \frac{2\pi C}{t} |1\rangle_{inv} \otimes |0\rangle_{eig} \otimes \sum_j \frac{b_j}{\lambda_j} |u_j\rangle_b \end{aligned}$$

5. We then measure the register inv in the canonical basis.

In the case where the result is 1, accounting for the renormalization factor

$$\eta = \frac{t}{2\pi C \sqrt{\sum_j \frac{b_j^2}{\lambda_j^2}}}$$

we have

$$|1\rangle_{inv} \otimes |0\rangle_{eig} \otimes \sum_j \frac{\eta b_j}{\lambda_j} |u_j\rangle_b.$$

We therefore get the $|\vec{x}\rangle$ in a state corresponding to $\vec{x}$, modulo the renormalization factor η .

6. In case we are interested in η , this value can be recovered from the probability to measure 1 in Step 5.

6.3.11 This algorithm follows a *post-selection* strategy: we only know if we succeed after the measure of the register *inv*. If we get 0, we failed and we have to start over.

6.3.12 One remaining question is: what can we do with $|\vec{x}\rangle$? The coefficients are not classically available, and recovering them is costly. The HHL algorithm offers a use-case: instead of trying to recover these coefficients, one might want to compute the scalar product of $\vec{x}$ with a third vector $\vec{r}$. This can be done with the quantum co-processor without having to extract the coefficients of $\vec{x}$... See the original paper [HHL09] for details.

6.3.13 An example. A working example is

$$A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}$$

and

$$\vec{b} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

The solution $\vec{x}$ is

$$\begin{pmatrix} 9/8 \\ 3/8 \end{pmatrix}.$$

To encode the problem we only need one single qubit for $\vec{b}$, and its state is $|\vec{b}\rangle_b = |0\rangle_b$.

For the parameters, we can choose $t = 2\pi\frac{3}{8}$ et $C = \frac{1}{4}$: the calculations add up. We pick a register *eig* with two qubits, since

$$A \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{2}{3} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \text{ and then } \lambda_1 = 2/3.$$

$$A \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{4}{3} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \text{ and then } \lambda_2 = 4/3.$$

This means that $\frac{\lambda_1 t}{2\pi} = 1/4$ and $\frac{\lambda_2 t}{2\pi} = 1/2$. Therefore, with two bits one can store each of these values in binary as $0.b_1 b_2 \equiv \frac{b_1}{2} + \frac{b_2}{4}$. Since $1/4$ is 0.01 in binary and $1/2$ is 0.10 in binary, the expected register states for *eig* are $|01\rangle$ and $|10\rangle$, fitting on two qubits.

If you follow my lecture, you might be asked to do a lab-session demonstrating how the algorithm works on this contrived example.

6.4 Exercises

6.4.1 Recall [6.3.2](#): When A is not Hermitian, show how one can recover a solution for the equation $A \cdot \vec{x} = \vec{b}$ using HHL when considering instead

$$\begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}.$$

7 Algorithms for NISQ era

7.1 Variational Algorithms

7.1.1 In Shor's algorithm, we build one single circuit, once for all. The circuit depends on the size of the input (we would not use the same circuit to factor 15 and to factor 100970708303), but once the circuit is built, it is used over and over again without change, until the algorithm succeeds. HHL or Grover are similar: a circuit is carefully crafted and then used over and over.

7.1.2 Another class of algorithms are the *variational* algorithms. In such a model, the circuit is updated each time we need to use it again. The idea is to refine the circuit to get closer and closer to the solution to the problem. They can be regarded as optimization techniques: instead of getting closer and closer to an optimal ket-vector (which we cannot manipulate directly), the algorithm optimize a circuit computing the desired ket-vector.

7.1.3 Variational algorithms are particularly well-suited for the *NISQ* regime of quantum computation (*Noisy Intermediate Scale Quantum*, see 4.8.2): they usually do not require very large quantum memories, they are very flexible in term hardware, and they are arguably resilient to noise.

7.2 VQE

7.2.1 Maybe the most typical variational algorithm is *VQE*, whose initials stand for *Variational Quantum Eigensolver*. Its objective is to find the extremal eigenvectors of an Hermitian matrix, and it perfectly embodies Feynman's intuition: using a quantum physical system to compute quantum properties [Fey82]. Indeed, Hermitian matrices (or *Hamiltonian*) are typically used to encode properties of a physical systems, and the extremal eigenvectors of such operators capture important informations. For non-trivial systems, the number of dimensions of the state space quickly becomes daunting on conventional computers.

↗ 2.9.5

7.2.2 Optimization problems can be reduced to the quest for an extremal eigenvector in the following way. An optimization problem is typically under the form

Maximize / minimize $C(x)$ when x belongs to some set S

with C a *cost function* outputting real values.

In the discrete, finite case, one can always pick $S = \{0..2^n - 1\}$ (with potentially a dummy padding to match a power of two in size). The function C then simply inputs bitstrings of size n , using for instance big-endian notation (see 5.3.2).

If one builds a diagonal hermitian matrix as

↗ 2.9.4

$$H_C = \sum_x C(x) |x\rangle \langle x|,$$

we have $H_C |x\rangle = C(x) |x\rangle$. So

- Minimizing $C(x)$ consists in finding the minimal eigenvalue of H_C .
- Maximizing $C(x)$ consists in finding the maximal eigenvalue of H_C , therefore the minimal eigenvalue of $-H_C$: this is the same problem (up to a change in sign).

7.2.3 The kind of problem VQE can solve could be named **QUANTUMMINEIGEN**:

QUANTUMMINEIGEN

Input an hermitian matrix H

Output an eigenvector $|\psi\rangle$ with minimal eigenvalue

The algorithm is very simple: it is about solving an optimization problem in the state space. We aim at minimizing the function

$$F : \begin{cases} \mathcal{H}^{\otimes n} & \longrightarrow \mathbb{R} \\ |\psi\rangle & \longmapsto \langle\psi| H |\psi\rangle \end{cases} \quad (58)$$

In theory this is a regular function: one can use any procedure such as gradient descent to solve it. In practice, it is not clear how to do this efficiently.

7.2.4 Claim. Assuming that $\lambda_{\min}$ is the minimal eigenvalue of H , for all ket-vector $|\psi\rangle$ we have

$$\langle\psi| H |\psi\rangle \geq \lambda_{\min}.$$

The equality is attained when $|\psi\rangle = |\psi_{\min}\rangle$.

7.2.5 Proof of 7.2.4. According to 2.9.4, one can rewrite H as

$$H = \sum_j \lambda_j \cdot |\psi_j\rangle \langle\psi_j|,$$

where the $|\psi_j\rangle$ s form an orthonormal basis of eigenvectors. Using this basis, the vector $|\psi\rangle$ can be decomposed as

$$\sum_j \alpha_j \cdot |\psi_j\rangle.$$

Computing $\langle\psi| H |\psi\rangle$, we get:

$$\begin{aligned} \langle\psi| H |\psi\rangle &= \left(\sum_k \overline{\alpha_k} \cdot \langle\psi_k| \right) \left(\sum_j \lambda_j \cdot |\psi_j\rangle \langle\psi_j| \right) \left(\sum_l \alpha_l \cdot |\psi_l\rangle \right) \\ &= \sum_{j,k,l} \overline{\alpha_k} \alpha_l \lambda_j \langle\psi_k|\psi_j\rangle \langle\psi_j|\psi_l\rangle. \end{aligned}$$

Because the $|\psi_j\rangle$ are pairwise orthogonal, the terms in the sum are all zeros except when $j = k = l$. We then have

$$\langle\psi| H |\psi\rangle = \sum_j \overline{\alpha_j} \alpha_j \lambda_j \langle\psi_j|\psi_j\rangle \langle\psi_j|\psi_j\rangle$$

$$\begin{aligned}
&= \sum_j \bar{\alpha}_j \alpha_j \lambda_j && \text{(because every } |\psi_j\rangle \text{ is of norm 1)} \\
&= \sum_j |\alpha_j|^2 \lambda_j \\
&\geq \sum_j |\alpha_j|^2 \lambda_{\min} && \text{(because } \lambda_{\min} \text{ is the smallest of all } \lambda_j \text{'s)} \\
&= \left(\sum_j |\alpha_j|^2 \right) \lambda_{\min} \\
&= \lambda_{\min} && \text{(because } |\psi\rangle \text{ is of norm 1)}
\end{aligned}$$

This inequality becomes an equality when $|\psi\rangle$ is an eigenvector $|\psi_{\min}\rangle$ corresponding to $\lambda_{\min}$ since

$$\langle \psi_{\min} | H | \psi_{\min} \rangle = \lambda_{\min} \langle \psi_{\min} | \psi_{\min} \rangle = \lambda_{\min}.$$

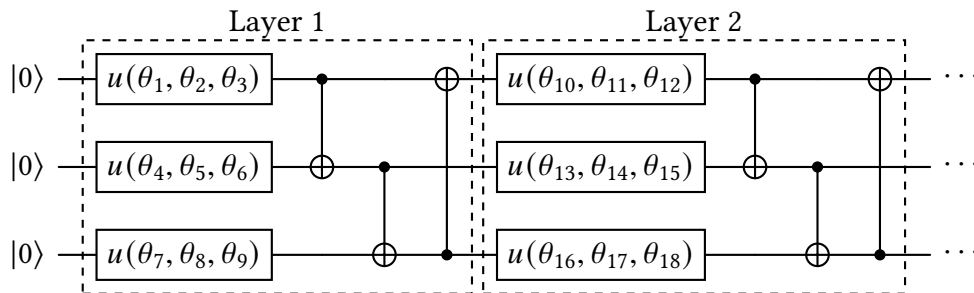
7.2.6 Sketch of the VQE algorithm. This is where we will use the quantum co-processor. Instead of directly manipulating a ket-state $|\psi\rangle$ to move it towards $|\psi_{\min}\rangle$, the idea consists in manipulating a set of (real) parameters instead, used to specify a circuit. This circuit is then used to compute a candidate $|\psi\rangle$. The procedure is as follows:

$$\vec{\theta} \in \mathbb{R}^p \xrightarrow{\text{build}} \text{some circuit} \xrightarrow{\text{evaluate}} |\psi\rangle \xrightarrow{\text{estimate}} \langle \psi | H | \psi \rangle$$

The first part generates a circuit out of p real parameters, the second part consists in evaluating the circuit to get the candidate ket-vector $|\psi\rangle$, and the third part estimates the desired scalar product. The goal of VQE is to make use of the quantum co-processor for the two last operations.

7.2.7 Structure of the circuit. For VQE, the circuit is built from an *ansatz*: a circuit typically consisting of rotations and 2-qubit gates, where the rotation are parameterized by the array of real numbers $\vec{\theta}$. There is no choice *a priori* on the structure of the circuit. We only need a circuit shape that can approximate well-enough the desired extremal eigenvector. In the context of quantum chemistry, the ansatz can for instance capitalize on the symmetry of the problem.

7.2.8 Without any knowledge on the structure of the Hermitian, we are bound to choose a generic ansatz, entangling enough to reach enough of the state space. On 1 qubit: we only need 3 angles (see 3.5.10). On more qubits, we need at least one entangling gate, typically the CNOT gate, in order to reach states that are not necessarily separable. For instance, on 3 qubits, one can consider a circuit shaped in layers, as follows. § 3.3.6



(with u a generic unitary parameterized by 3 angles.) As we increase the number of layers, the ansatz becomes more and more expressive, but also more and more expensive to compute and manipulate. Indeed, optimization techniques tend to perform poorly with a large number of parameters: this problem is known as the *Barren plateau*.

In any case, the circuit computes a candidate state parameterized by an array of θ 's. We then have a purely classical set $\mathbb{R}^p$ for some p , representing angles, from which we build a circuit that, when run on the quantum co-processor, generates the $|\psi\rangle$ we care about.

7.2.9 Estimating the scalar product Running the circuit on a quantum co-processor indeed produces a ket-vector $|\psi\rangle$, but it is hidden inside the quantum memory. Estimating $\langle\psi|H|\psi\rangle$ can be done using an estimation of the amplitudes of the various basis vectors in $|\psi\rangle$.

In this course, we focus on the case of diagonal hermitian matrices. If $|\psi\rangle$ is

$$|\psi\rangle = \sum_j \alpha_j |j\rangle$$

then

$$\langle\psi|H|\psi\rangle = \sum_j |\alpha_j|^2 C(j)$$

and this can be computed offline, on the classical device, using an estimate of the probability distribution coming from the measure of $|\psi\rangle$.

7.2.10 In details, running the VQE algorithm consists in choosing an ansatz and building a function $f : \vec{\theta} \mapsto [\text{some real}]$ as follows:

1. Using the ansatz and the parameters $\vec{\theta}$, generate a circuit.
2. Run many times (say N times) the circuit on $|00 \dots 0\rangle$, followed by a measure.
3. Each run gives a bitstring. Gathering the results, we get $M \leq N$ distinct bistrings x_k , each appearing d_k times. The value d_k/N gives then an approximation for probability of measuring x_k .
4. The output of f is built as $\sum_{k=1}^M \frac{d_k}{N} C(x_k)$.

The objective is to minimize the function f : it is a purely classical function that can be coded in any language (such as PYTHON) and this can be done using any suitable optimization library.

7.2.11 As the procedure 7.2.10 runs the function C , it might seem as if the overall algorithm is redundant: after all, running C on bitstrings looks like we are doing an exhaustive search, so why bother using a quantum computer? The answer lies in the parameter N in Item 2: at each step we are only querying N bitstrings. Said otherwise, we are in Item 2 somehow estimating $\langle\psi|H|\psi\rangle$ on a subspace of dimension $M \leq N$, and this subspace is refined at each update of the angles. In a general case, we might not explore every possible bitstrings.

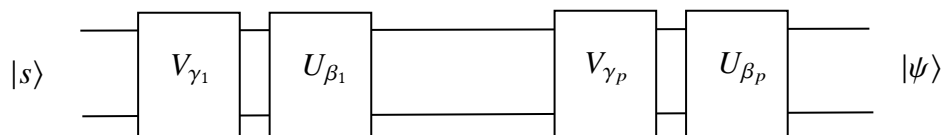
7.3 QAOA

7.3.1 Let us take a bit of a high-level view from VQE: we start from a Hermitian, but the ansatz is completely blind to the structure of this Hermitian. The algorithm only uses it at the very end to adjust the parameters. One can think that maybe making a circuit shape specific to the problem could somehow speed up the algorithm: this is the proposal of Farhi and his co-authors with QAOA [FGG14].

7.3.2 QAOA stands for *Quantum Approximate Optimization Algorithm*, and can be regarded as a specialized VQE for optimization problems. It essentially implements the simulation of an adiabatic evolution.

7.3.3 How it works. We make use of 2 hermitians, both acting on our n qubits. The first one is $B = \sum_{i=1}^n \sigma_X^i$, where σ_X^i is the action of X on qubit i , and the Hamiltonian encoding the cost function: $H_C = \sum_x C(x) |x\rangle \langle x|$. The shape of the algorithm is very close to VQE:

- Start from a state $|s\rangle = \frac{1}{\sqrt{2^n}} \sum_{x_1 \dots x_n} |x_1 \dots x_n\rangle$
 - this is just a tower of Hadamard acting on $|000\dots 00\rangle$
- Generate $|\psi\rangle$ from a circuit parameterized by two arrays $\vec{\beta}$ et $\vec{\gamma}$, each of p angles
 - $|\psi\rangle = U_{\beta_p} V_{\gamma_p} U_{\beta_{p-1}} V_{\gamma_{p-1}} \dots U_{\beta_2} V_{\gamma_2} U_{\beta_1} V_{\gamma_1} |s\rangle$
 - with
 - * $U_\lambda = e^{-i\lambda B}$
 - * $V_\lambda = e^{-i\lambda H_C}$



- Compute $\langle \psi | H_C | \psi \rangle$ as for VQE.
- This series of operations can be seen as a function that inputs two arrays $(\vec{\beta}, \vec{\gamma})$ and that outputs a real numbers. As for VQE, we can optimize this function (here, we will want to MAXIMIZE it).

Seen like that, QAOA is just an instance of VQE, with a circuit that is a bit more funky than the one we used above. The question is: why do we have a good (better) chance to get to the right vector if we perform a maximization?

7.3.4 This raises two questions:

- Is there any **proof** for any speedup compared to classical methods?
 - No
 - Some theoretical results for $p=1$ et $p=2$, with error bounds

- No speedup shown up to date
- But not proof that there are none...
- Do we at least have a guarantee on the convergence towards a solution?
 - Yes, for p "large enough"
 - To understand how it works, we need a small sidestep.

7.3.5 Sidestep: adiabatic evolution. (*Disclaimer:* I am not physicists. There are a lot of caveats in what I will say, but the intuition still stands.) A physical system is subject to an Hamiltonian H which in our case is nothing more than a hermitian matrix. This Hamiltonian might evolve along time: $H(t)$. The evolution of a system $|\psi_t\rangle$ is described by the Shrödinger equation

$$\frac{d}{dt} |\psi_t\rangle = -i \cdot H(t) |\psi_t\rangle$$

Note: if $H(t)$ is constant with value H , the solution with initial condition $|\psi_0\rangle$ is...

$$|\psi_t\rangle = e^{-itH} |\psi_0\rangle$$

(and we find back the relationship between hermitian and unitary). In any case, the *adiabatic theorem* states that

ADIABATIC THEOREM

If $H(t)$ varies slow enough, and if the system is at $t=0$ at minimal energy level (i.e. its state is a minimal eigenvector of $H(0)$), then at each t its energy level is minimal (i.e. its state is a minimal eigenvector of $H(t)$).

The “slow enough” has to do with the *spectral gap*: the distance between the two lowest eigenvalues. Note that one can play the same game with the maximal eigenvalue by instead considering $-H(t)$ (which is also Hermitian!).

7.3.6 Link with QAOA. We start with

$$|\psi_0\rangle = |s\rangle = \frac{1}{\sqrt{2^n}} \sum_{x_1 \dots x_n} |x_1 \dots x_n\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \dots \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

We want to get to the eigenvector of maximal eigenvalue for H_C .

7.3.7 Question: is $|s\rangle$ the maximal eigenvalue for somebody?

$$\rightarrow B = \sum_{i=1}^n \sigma_X^i \quad (\sigma_X^i \text{ is the action of } X \text{ on qubit } i)$$

And it has all of the required properties for the adiabatic theorem to hold.

7.3.8 From the adiabatic theorem we can deduce that the interpolation:

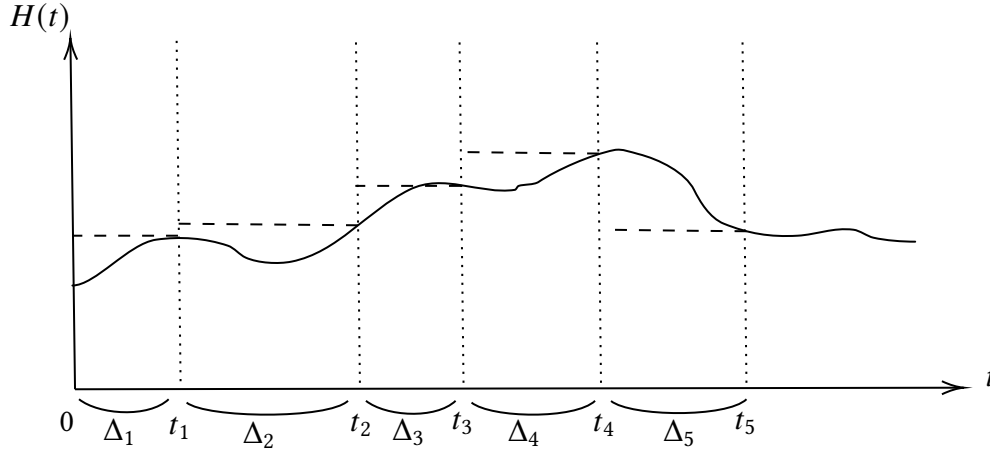
$$H(t) = \frac{t}{T} H_C + \left(1 - \frac{t}{T}\right) B$$

with a large enough T will slowly send $|s\rangle$ to the desired eigenvector

$$\rightarrow H(0) = B \text{ et } H(T) = H_C$$

Remains to understand how to do this.

7.3.9 Instead of considering $H(t)$, we shall consider a piecewise constant approximation. To not move apart from the "correct" function, we need small enough pieces.



On each timeslot Δ_i , the Hamiltonian is approximated as constant. One can then say that (according to the Shrödinger equation)

$$|\psi_{t_{i+1}}\rangle \simeq e^{-i\Delta_i H(t_i)} |\psi_{t_i}\rangle$$

and then that if we consider the slicing of $[0, T]$ into p slides,

$$|\psi_T\rangle = e^{-i\Delta_p H(t_p)} \dots e^{-i\Delta_2 H(t_2)} e^{-i\Delta_1 H(t_1)} |s\rangle$$

is an approximation of the maximal eigenvector of H_C . We almost have the form of the QAOA circuit

7.3.10 To conclude, we need the *Trotter-Suzuki* formula of 5.7 stating that

$$e^{\delta(A+B)} = e^{\delta A} e^{\delta B} + O(\delta^2)$$

whenever δ is small. So $e^{-i\Delta_p H(t_p)} = e^{-i\Delta_p(\alpha B + \beta H_C)} \simeq e^{-i\Delta_p \alpha B} e^{-i\Delta_p \beta H_C}$ since Δ_p is supposed to be small.

7.3.11 Let us summarize: for QAOA, we build

$$|\psi\rangle = U_{\beta_p} V_{\gamma_p} U_{\beta_{p-1}} V_{\gamma_{p-1}} \dots U_{\beta_2} V_{\gamma_2} U_{\beta_1} V_{\gamma_1} |s\rangle$$

with

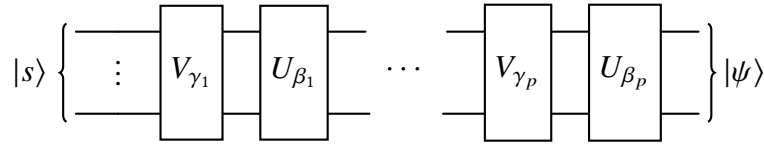
- $U_\lambda = e^{-i\lambda B}$
- $V_\lambda = e^{-i\lambda H_C}$

From the adiabatic theorem, we have

$$|\psi_T\rangle = e^{-i\Delta_p H(t_p)} \dots e^{-i\Delta_2 H(t_2)} e^{-i\Delta_1 H(t_1)} |s\rangle$$

We can then replace each $e^{-i\Delta_p H(t_p)}$ with a product $U_\alpha V_\beta$. How to choose each of these α, β , since we do not know which Δ_i and t_i are optimal? This is why we consider this as an optimization problem: a gradient descent (for instance) will figure it out for us.

7.3.12 Summary. For QAOA, we only need to build a circuit



with

- $U_\lambda = e^{-i\lambda B} \rightarrow$ this is just a tower of X -rotations (R_x -gates) with angles λ
- $V_\lambda = e^{-i\lambda H_C} \rightarrow$ well, this depends on $C(x)$!

and then we minimize a function that

- Inputs $\vec{\beta}, \vec{\gamma}$
- Realize and run the corresponding circuit many times
- Get a estimation of the probability distribution
- Compute and return (an estimate of) $\langle \psi | H_C | \psi \rangle$

Remains to know how to build $e^{-i\lambda H_C}$.

7.3.13 To build $e^{-i\lambda H_C}$, one can capitalize on the decomposition presented in 2.9.13: H_C is written as a sum of (tensors of) Pauli matrices:

$$H_C = \sum_{G_1, \dots, G_n \in \{X, Y, Z, I\}} h_{G_1, \dots, G_n} \cdot G_1 \otimes \dots \otimes G_n$$

with $h_{G_1, \dots, G_n} \in \mathbb{R}$. Note that in the very specific case of QAOA, because H_C is a diagonal matrix, so the matrices G_i 's are only among I and Z . In any case, using the Trotter-Suzuki decomposition of 5.7, we deduce that

$$e^{-i\lambda H_C} \sim \prod_{G_1, \dots, G_n \in \{X, Y, Z, I\}} e^{-i\lambda h_{G_1, \dots, G_n} \cdot G_1 \otimes \dots \otimes G_n}.$$

Realizing $e^{-i\lambda H_C}$ is then reduced to the implementation of an exponential of (tensors of) Pauli matrices, similar to what happens for Trotterization (see Sec. 5.7).

7.3.14 If H_C can necessary be written as a linear decomposition of (tensors of) Pauli matrices, in the context of QAOA the function C is in general given in a form that makes it easy to generate the decomposition: a (real) polynomials of the x_i 's, seen as the values 0 and 1, as follows.

$$C(x_1, \dots, x_n) = \sum_k h_k \cdot x_1^{d_{1,k}} \dots x_n^{d_{n,k}}.$$

The trick to go from C to H_C is to change the x_j 's with linear functions

$$X_j : \begin{array}{l} \mathcal{H}^{\otimes n} \rightarrow \mathcal{H}^{\otimes n} \\ |x_1 \dots x_n\rangle \rightarrow \begin{cases} |x_1 \dots x_n\rangle & \text{if } x_j = 1 \\ 0 & \text{else} \end{cases} \end{array}$$

and the multiplications with function composition. We then have:

$$H_C = \sum_k h_k \cdot X_1^{d_{1,k}} \cdots X_n^{d_{n,k}}$$

(where X_j^0 stands for the identity function). Now, note how one can encode each X_j as

$$X_j = (I - Z_j)/2$$

with $Z_j = I \otimes \cdots \otimes I \otimes Z \otimes I \otimes \cdots \otimes I$, the Z matrix acting on the j -th qubit. We have our decomposition:

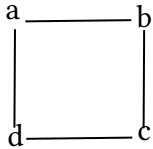
$$H_C = \sum_k h_k \cdot ((I - Z_1)/2)^{d_{1,k}} \cdots ((I - Z_n)/2)^{d_{n,k}}$$

7.4 Example: MAXCUT and QUBO problems

7.4.1 QAOA for MAXCUT. A typical *optimization problem* for QAOA is *MACUT*, defined as follows.

MAXCUT	
Input	a non-oriented graph $G = (V, E)$.
Output	A CUT: a partition of V into $V_0 \cup V_1$ (and $V_0 \cap V_1 = \emptyset$).
Constraint	Maximize the number of edges going from V_0 to V_1 .

7.4.2 Consider for instance the following graph.



- The cost of $V_0 = \{a, b\}, V_1 = \{c, d\}$ is 2
- The cost of $V_0 = \{a, c\}, V_1 = \{b, d\}$ is 4
- The cost of $V_0 = \{c\}, V_1 = \{a, b, d\}$ is 2

The maximal cost one can get is 4, and one of the maxcut is $V_0 = \{a, c\}, V_1 = \{b, d\}$

7.4.3 Cuts as bitstrings In the case of MAXCUT, the cost function works over the set of all possible cuts. A cut can be coded in a bitstring as follow. Assume $V = \{0 \dots n-1\}$: One can store in a boolean value x_i the position of the node $i \in V$:

$$i \in V_{x_i}.$$

Now, given a vector $x_0 \dots x_{n-1}$, this vector stores where each node belongs to. For instance, the max cut of 7.4.2 can be encoded as 0011 or 1100, assuming that $a = 0, b = 1, c = 2$ and $d = 3$.

7.4.4 The Hermitian of the cost function. In general, one can write a cost function

$$C(x) = \sum_{(j,k) \in E} x_j(1 - x_k) + x_k(1 - x_j).$$

This function “counts” how many edges connect V_0 and V_1 . The maximum cut is obtained with the maximization of this function. According to 7.3.14, the corresponding Hermitian is then

$$\begin{aligned} H_C &= \sum_{(j,k) \in E} \left(\frac{1}{4}(1 - Z_j)(1 + Z_k) + \frac{1}{4}(1 - Z_k)(1 + Z_j) \right) \\ &= \frac{1}{4} \sum_{(j,k) \in E} (1 - Z_j + Z_k - Z_j Z_k + 1 - Z_k + Z_j - Z_k Z_j) \\ &= \frac{1}{4} \sum_{(j,k) \in E} (2 - 2Z_j Z_k) \\ &= \frac{1}{2} \sum_{(j,k) \in E} (1 - Z_j Z_k) \end{aligned}$$

All the terms in the sum commute: the unitary $e^{i\lambda H_C}$ is then exactly

$$e^{i\lambda H_C} = \prod_{(j,k) \in E} e^{i\lambda(1 - Z_j Z_k)} = (\text{some phase}) \cdot \prod_{(j,k) \in E} e^{-i\lambda Z_j Z_k}.$$

For the corresponding circuit, see 5.7.12.

7.4.5 QUBO The cost function for MAXCUT has a very peculiar form: if $x_1, \dots, x_n$ is regarded as a column 0/1 vector $\vec{x}$, it can be seen at the bilinear form

$$\vec{x}^T \cdot Q \cdot \vec{x} = \sum_{i,j} Q_{i,j} x_i x_j$$

where Q is an $n \times n$ symmetric matrix (or, equivalently, a triangular matrix). This sum has monomials:

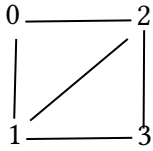
- of degree 1 (because $x_i x_i = x_i$), corresponding to the diagonal of Q
- of degree 2, for the non-diagonal elements.

An optimization problem for which the cost function can be written in this form is called a *QUBO (Quadratic Unconstrained Binary Optimization)* problem.

Many optimization problems fit within this class. It is therefore one of the main target for several quantum platforms based on adiabatic computation, such as DWAVE or Pasqal and other companies based on neutral atoms.

7.5 Exercises

7.5.1 MAXCUT Consider the following graph.



What are (or “is”) the maximum cuts here? What bitstrings do they correspond to? What is the cost function, and the corresponding Hermitian H_C ?

A Cosine-Sine Decomposition

The statement of Theorem 4.3.13 is as follows:

A.1 Statement

Let U be any $n + 1$ -qubit unitary. One can decompose U as

$$\begin{pmatrix} B_1 & 0 \\ 0 & B_2 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}$$

with A, B, C and S n -qubit unitaries, with C and S n -qubit diagonals such that $S^2 + C^2 = Id$.

A.2 Proof

A.2.1 Write U blockwise with n -qubit-sized blocks:

$$U = \begin{pmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{pmatrix}$$

Let $B_1 C A_1 = U_{11}$ be a diagonalization of U_{11} : the matrices A_1 and B_1 are unitary, and D is diagonal with real entries. Without loss of generality, we can assume the entries are non-negative and in decreasing order (with the largest on the left). Because the columns of the matrix U_{11} are pieces of columns of the unitary U , their norms are less than 1: so are the eigenvalues. The matrix C can then be represented blockwise as

$$C = \begin{pmatrix} Id & 0 & 0 \\ 0 & D & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

with D diagonal with positive entries that are neither 0 nor 1.

A.2.2 Now, using QL and QR decompositions, choose B'_2 and A'_2 so to make $B'_2 U_{21} A'_1$ lower and $B'_1 U_{12} A'_2$ upper triangular. Without loss of generality, we can assume that the former has non-negative real numbers on the diagonal, while the later has non-positive ones. Define the matrix E as

$$E = \begin{pmatrix} B'_1 & 0 \\ 0 & B'_2 \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{pmatrix} \begin{pmatrix} A'_1 & 0 \\ 0 & A'_2 \end{pmatrix} = \begin{pmatrix} C & B'_1 U_{12} A'_2 \\ B'_2 U_{21} A'_1 & B'_2 U_{22} A'_2 \end{pmatrix}. \quad (59)$$

It is unitary and can then be written as blocks as follows:

$$E = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & R_1 & Q_1 & Q_2 \\ 0 & D & 0 & 0 & R_2 & Q_3 \\ 0 & 0 & 0 & 0 & 0 & R_3 \\ \hline L_1 & 0 & 0 & N_1 & N_2 & N_3 \\ P_1 & L_2 & 0 & N_4 & N_5 & N_6 \\ P_3 & P_4 & L_3 & N_7 & N_8 & N_9 \end{array} \right).$$

Unitarity imposes constraints from which we can explicit each block

A.2.3 Collapse of 1st block-row and 1st block-column . Each row in

$$Id \quad 0 \quad 0 \mid R_1 \quad Q_1 \quad Q_2$$

and each column in

$$\begin{array}{c} Id \\ 0 \\ 0 \\ \hline L_1 \\ P_1 \\ P_3 \end{array}$$

is of norm 1: this forces $R_1, Q_1, Q_2, L_1, P_1, P_3$ to be zero-matrices since one of the entries is necessarily 1 (from the Id block).

A.2.4 Collapse of the last block-row. Consider the 3rd block of columns and the last block of rows:

$$\begin{array}{ccc|ccc} 0 & & & & & \\ 0 & & & & & \\ 0 & & & & & \\ \hline 0 & & & & & \\ 0 & & & & & \\ P_3 & P_4 & L_3 & N_7 & N_8 & N_9 \end{array}$$

The matrix L_3 is lower-triangular as follows:

$$L_3 = \begin{pmatrix} \ddots & \vdots & \vdots & \vdots \\ \cdots & l_6 & 0 & 0 \\ \cdots & l_5 & l_4 & 0 \\ \cdots & l_3 & l_2 & l_1 \end{pmatrix}.$$

Consider the column of E ending with l_1 : all of its other entries are 0s. Since it is of norm 1, l_1 is 1. The last row of E is also of norm 1: because one of its entry is 1, all other entries are 0: the matrix L_3 is

$$L_3 = \begin{pmatrix} \ddots & \vdots & \vdots & \vdots \\ \cdots & l_6 & 0 & 0 \\ \cdots & l_5 & l_4 & 0 \\ \cdots & 0 & 0 & 1 \end{pmatrix},$$

and the last lines of P_3, P_4, N_7, N_8 and N_9 are all 0s.

A similar argument can now be used on the column of E containing l_4 : we derive that l_4 is 1, and that the second-to-last line of E contains zero everywhere except at the position l_4 :

$$L_3 = \begin{pmatrix} \ddots & \vdots & \vdots & \vdots \\ \cdots & l_6 & 0 & 0 \\ \cdots & 0 & 1 & 0 \\ \cdots & 0 & 0 & 1 \end{pmatrix},$$

and the second-to-last lines of P_3, P_4, N_7, N_8 and N_9 are all 0s. Working through all of the rows of L_3 , we end up with

$$L_3 = Id, P_3 = 0, P_4 = 0, N_7 = 0, N_8 = 0, N_9 = 0.$$

Using a symmetric argument, we can derive that

$$R_3 = -Id, Q_3 = 0, Q_2 = 0, N_3 = 0, N_6 = 0, N_9 = 0$$

(For the $-Id$, remember that the matrix has non-positive coefficients on the diagonal).

A.2.5 So far, we simplified E into

$$E = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & 0 & 0 & 0 \\ 0 & D & 0 & 0 & R_2 & 0 \\ 0 & 0 & 0 & 0 & 0 & -Id \\ \hline 0 & 0 & 0 & N_1 & N_2 & 0 \\ 0 & L_2 & 0 & N_4 & N_5 & 0 \\ 0 & 0 & Id & 0 & 0 & 0 \end{array} \right).$$

A.2.6 Instantiating R_2 . The next step is to instantiate R_2 and L_2 . For this, consider the second block of rows:

$$0 \quad D \quad 0 \quad | \quad 0 \quad R_2 \quad 0.$$

It is of the form

$$\begin{array}{cccc|cccc} 0 & \cdots & 0 & \ddots & \vdots & \vdots & \vdots & 0 & \cdots & 0 & \ddots & \vdots & \vdots & \vdots & 0 & \cdots & 0 \\ \vdots & & \vdots & \cdots & c_3 & 0 & 0 & \vdots & & \vdots & \vdots & \cdots & r_3 & z_3 & z_2 & \vdots & & \vdots \\ \vdots & & \vdots & \cdots & 0 & c_2 & 0 & \vdots & & \vdots & \vdots & \cdots & 0 & r_2 & z_1 & \vdots & & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & 0 & c_1 & 0 & \cdots & 0 & \cdots & 0 & 0 & r_1 & 0 & \cdots & 0 \end{array} \quad (60)$$

Because E is unitary, the last row in Eq. (60) is of norm 1:

$$c_1^2 + r_1^2 = 1.$$

Since $c_1 \neq 1$, we deduce $r_1 \neq 0$. Since the last and second-to-last are orthogonal, we have $r_1 z_1 = 0$, so $z_1 = 0$. The same argument on the last and third-to-last gives $z_2 = 0$. Proceeding similarly for all the remaining rows, we conclude that the last column of R_2 is zero, except for its last element r_1 . Eq. (60) can then be rewritten

$$\begin{array}{cccc|cccc} 0 & \cdots & 0 & \ddots & \vdots & \vdots & \vdots & 0 & \cdots & 0 & \ddots & \vdots & \vdots & \vdots & 0 & \cdots & 0 \\ \vdots & & \vdots & \cdots & c_3 & 0 & 0 & \vdots & & \vdots & \vdots & \cdots & r_3 & z_3 & 0 & \vdots & & \vdots \\ \vdots & & \vdots & \cdots & 0 & c_2 & 0 & \vdots & & \vdots & \vdots & \cdots & 0 & r_2 & 0 & \vdots & & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & 0 & c_1 & 0 & \cdots & 0 & \cdots & 0 & 0 & r_1 & 0 & \cdots & 0 \end{array} \quad (61)$$

Using the same argument, focusing on the norm of the second-to-last row and the fact that $c_2 \neq 1$, we deduce that $r_2 \neq 0$, and then that all elements in R_2 above r_2 are null. We can proceed upwards, clearing all elements off diagonal: we deduce that R_2 is diagonal with non-zero entries, and that

$$D^2 + R_2^2 = Id. \quad (62)$$

A.2.7 Instantiating L_2 . Using a similar reasoning but with column in place of rows, we can deduce that L_2 is diagonal with non-zero entries, and that

$$D^2 + L_2^2 = Id. \quad (63)$$

Since all of the diagonal elements of R_2 are non-positive and ones of L_2 are non-negative, from Eq. (62) and (63) we derive that

$$R_2 = -L_2.$$

A.2.8 Let us define the diagonal matrix L_2 with T : we simplified E into

$$E = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & 0 & 0 & 0 \\ 0 & D & 0 & 0 & -T & 0 \\ 0 & 0 & 0 & 0 & 0 & -Id \\ \hline 0 & 0 & 0 & N_1 & N_2 & 0 \\ 0 & T & 0 & N_4 & N_5 & 0 \\ 0 & 0 & Id & 0 & 0 & 0 \end{array} \right).$$

A.2.9 Collapsing N_2 and N_4 . The next step is to show how N_2 and N_4 are zero-matrices. To show $N_2 = 0$, we consider the orthogonality of rows in

$$0 \quad D \quad 0 \quad | \quad 0 \quad -T \quad 0$$

with rows in

$$0 \quad 0 \quad 0 \quad | \quad N_1 \quad N_2 \quad 0$$

Computing the scalar product of row number i in the latter with row number j in the former yields

$$t_i n_{j,i} = 0,$$

where $n_{j,i}$ is the j, i -th element in N_2 and t_i the i th element on the diagonal of T . Since none of these diagonal elements are zero, $t_i \neq 0$, so $n_{j,i} = 0$. And this is true for all i, j . So $N_2 = 0$.

We can follow the same argument with columns instead and derive that $N_4 = 0$.

A.2.10 We then simplified E into

$$E = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & 0 & 0 & 0 \\ 0 & D & 0 & 0 & -T & 0 \\ 0 & 0 & 0 & 0 & 0 & -Id \\ \hline 0 & 0 & 0 & N_1 & 0 & 0 \\ 0 & T & 0 & 0 & N_5 & 0 \\ 0 & 0 & Id & 0 & 0 & 0 \end{array} \right).$$

A.2.11 Instantiating N_5 . We can now show how N_5 is diagonal. First, consider the scalar product of one the i th row in the block of rows

$$0 \quad D \quad 0 \quad | \quad 0 \quad -T \quad 0$$

and the i th row in the block of rows

$$0 \quad T \quad 0 \quad | \quad 0 \quad N_5 \quad 0. \tag{64}$$

We get $c_i t_i - t_i n_{i,i} = 0$: we deduce that $n_{i,i} = c_i$. The i th row in the block of rows of Eq. 64 has a norm of the form

$$t_i^2 + c_i^2 + \sum_{j \neq i} |n_{i,j}|^2,$$

supposed to be equal to 1. But since $t_i^2 + c_i^2 = 1$, we deduce that all of the terms $n_{i,j}$ with $i \neq j$ are zero: The matrix N_5 is diagonal, it is in fact equal to D .

A.2.12 We then simplified E into

$$E = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & 0 & 0 & 0 \\ 0 & D & 0 & 0 & -T & 0 \\ 0 & 0 & 0 & 0 & 0 & -Id \\ \hline 0 & 0 & 0 & N_1 & 0 & 0 \\ 0 & T & 0 & 0 & D & 0 \\ 0 & 0 & Id & 0 & 0 & 0 \end{array} \right).$$

A.2.13 Instantiating N_1 . Remains to study N_1 . For this, consider the fact that $E^*E = EE^* = Id$. Blockwise, this means $N_1N_1^* = N_1^*N_1 = Id$.

So if instead of A'_2 we had chosen

$$A''_2 = A'_2 \begin{pmatrix} -N_1^* & 0 & 0 \\ 0 & Id & 0 \\ 0 & 0 & Id \end{pmatrix}$$

Then we would have gotten instead some other E'

$$E' = \left(\begin{array}{ccc|ccc} Id & 0 & 0 & 0 & 0 & 0 \\ 0 & D & 0 & 0 & -T & 0 \\ 0 & 0 & 0 & 0 & 0 & -Id \\ \hline 0 & 0 & 0 & I & 0 & 0 \\ 0 & T & 0 & 0 & C & 0 \\ 0 & 0 & Id & 0 & 0 & 0 \end{array} \right),$$

of the required form.

Some Standard Elementary Quantum Gates

As discussed in 2.6.8 and 3.3.3, we use here a *lexicographic ordering on basis vectors*.

From 3.5.1

$$X \triangleq \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z \triangleq \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y \triangleq \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

From 2.8.6

$$\text{HAD} \triangleq \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

From 3.5.2, 3.5.4 and 3.5.9:

$$S \triangleq \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T \triangleq \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad \text{PH}(\theta) \triangleq R_\theta \triangleq \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}.$$

From 3.5.7

$$R_G(\theta) \triangleq e^{-i\frac{\theta}{2} \cdot G} \triangleq \cos\left(\frac{\theta}{2}\right) \cdot \text{Id} - i \sin\left(\frac{\theta}{2}\right) \cdot G,$$

for $G \in \{X, Y, Z\}$. Matrix-wise, they can be written as

$$\begin{aligned} R_X(\theta) &\triangleq \begin{pmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \\ R_Y(\theta) &\triangleq \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \\ R_Z(\theta) &\triangleq \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \end{aligned}$$

From 3.6.7, 3.6.10 and 3.4.12

$$\text{CNOT} \triangleq \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad \text{C-Z} \triangleq \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \quad \text{SWAP} \triangleq \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

From 3.6.8, the Toffoli gate:

$$\text{C-C-X} \triangleq \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

References

- [BK05] Sergey Bravyi and Alexei Kitaev. Universal quantum computation with ideal clifford gates and noisy ancillas. *Physical Review A*, 71:022316, Feb 2005.
- [Cle22] Andrew N. Cleland. An introduction to the surface code. *SciPost Physics Lecture Notes*, 49, 2022. Part of the Quantum Information Machines Session 113 of the Les Houches School, July 2019.
- [DKPP09a] Vincent Danos, Elham Kashefi, Prakash Panangaden, and Simon Perdrix. Extended measurement calculus. In *Semantic Techniques in Quantum Computation*, page 235. Cambridge University Press, 2009.
- [DKPP09b] Vincent Danos, Elham Kashefi, Prakash Panangaden, and Simon Perdrix. *Semantic Techniques in Quantum Computation*, chapter Extended measurement calculus, pages 235–310. Cambridge University Press Cambridge, 2009.
- [Fey82] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(7-8):467–488, 1982.
- [FGG14] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. Technical Report MIT-CTP/4610, MIT, 2014.
- [HHL09] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103:150502, 2009.
- [Kit97] Alexei Kitaev. Quantum computations: Algorithms and error correction. *Russian Mathematical Surveys*, 52(6):1191, 1997.
- [KLM07] Phillip Kaye, Raymond Laflamme, and Michele Mosca. *An Introduction to Quantum Computing*. Oxford University Press, 2007.
- [KMN⁺07] Pieter Kok, W. J. Munro, Kae Nemoto, T. C. Ralph, Jonathan P. Dowling, and G. J. Milburn. Linear optical quantum computing with photonic qubits. *Reviews of Modern Physics*, 79:135–174, Jan 2007.
- [Kni02] E. Knill. Quantum gates using linear optics and postselection. *Phys. Rev. A*, 66:052306, Nov 2002.
- [Mer07] N. David Mermin. *Quantum Computer Science - An Introduction*. Cambridge University Press, 2007.
- [NC02] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2002.
- [RS16] Neil J. Ross and Peter Selinger. Optimal ancilla-free Clifford+T approximation of Z-rotations. *Quantum Information and Computation*, 16(11&12):901–953, 2016.
- [Shi03] Yaoyun Shi. Both Toffoli and controlled-NOT need little help to do universal quantum computing. *Quantum Information and Computation*, 3(1):84–92, 2003.

- [Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

Index

- absolute value, 5
- absolutely converging, 5
- adder, 93
- adiabatic theorem, 131
- amplitude, 6
- Amplitude Amplification, 96
- ancillas, 37
- ansatz, 86, 128
- approximate universality, 53
- array, 8
- auxiliary qubits, 37
- Barren plateau, 129
- base 2, 91
- Basel problem, 5
- basis, 9
- basis elements, 9
- basis state, 23
- Bell basis, 26, 48
- Bell state, 26
- Bernstein-Vazirani, 115
- big-endian, 91
- big-O, 52
- bilinear map, 12
- bitstrings, 91
- blackbox, 112
- Bloch sphere, 24
- canonical basis element, 7
- canonical representative element, 24
- carry, 94
- causality, 30
- circuit synthesis, 32
- classical problem, 85
- Clifford, 54
- CNOT, 35
- coefficient, 7, 13
- coincidence, 80
- complex numbers, 5
- complex plane, 6
- complexity, 52
- compute, 89
- conjugate, 5
- conjugate transpose, 10
- control qubit, 34
- controlled operation, 34
- converging, 5
- copy, 89
- copying, 46
- correction, 63, 83
- Cosine-Sine Decomposition, 57
- cost function, 126
- creation operator, 78
- decoherence, 68
- deferred measurement, 41
- dense coding, 48
- destructive measure, 38
- detectors, 74
- dimension, 7
- discard, 42
- dual, 10
- dual rail, 79
- eigenvalue, 16
- eigenvector, 16
- electromagnetic wave, 72
- encoding, 91
- entangled, 26
- EPR pair, 26
- errors, 69
- extremal, 16
- family of quantum circuits, 85
- field, 4
- functionals, 10
- garbage qubits, 89
- gate-set, 53
- geometric series, 5
- global phase, 23, 72
- graph state, 62
- Grover, 96
- H, 32
- Had, 32
- Hadamard, 14, 32
- half-adder, 94
- Hamiltonian, 16, 126

heralded, 80, 81
Hermitian, 16
HHL, 119
Hilbert space, 9
Hong-Ou-Mandel effect, 79

imaginary, 5
indistinguishable, 77
instance, 85
internal operations, 27
inverse, 31

joint quantum system, 25

ket, 8
ket-notation, 9
Kronecker product, 11, 25

least significant bit, 91
lexicographic order, 11
linear map, 13
linear optical components, 74
linear optics, 71
little-endian, 92
LOQC, 74
LSQ, 69

MACUT, 134
magic state, 60
magic states, 54
majority function, 88
mapping, 70, 71
maximal, 16
MBQC, 54, 63
measurement, 27, 37
Measurement-based Computation, 54
minimal, 16
mode, 73
modes, 74
multiplexor, 57

negative controls, 35
NISQ, 69, 126
no-cloning theorem, 46
noise, 69
norm, 5

operator, 13
optical circuit, 74

optimization problem, 134
oracle, 87, 96
oracle-based, 112
orthogonal, 9
orthogonality, 23
orthonormal basis, 9

parametrization, 33
particle, 71
pattern, 64, 65
Pauli, 17, 31
phase, 6, 16, 24, 71
photons, 71
pointer, 21
polarization, 72
polynomial reduction, 116
post-selection, 80, 124

QAOA, 130
QFT, 99, 101
QPE, 102
quantum algorithm, 85
quantum bit, 21, 22
quantum circuit, 28
Quantum Fourier Transform, 99, 101
quantum gates, 27
quantum memory, 21
Quantum Phase Estimation, 102
quantum registers, 21
qubit, 22
QUBO, 135

radial representation, 6
real number, 4
reasonable, 53
reference, 21
register, 21
repeat-until-success, 62
representative element, 23
ripple-carry adder, 94
rotations gates, 32
routing, 70, 71

S, 31
scalar product, 8
separable, 25, 26
series, 5
SHIFT basis, 25

signals, [65](#)
Solovay-Kitaev algorithm, [32](#)
sources, [74](#)
space complexity, [52](#)
spectral gap, [131](#)
subgraph isomorphism problem, [70](#)
surface code, [60](#)
swap, [30](#)
symmetric tensor, [77](#)
synthesis, [56](#)

T, [32](#)
teleportation, [46](#)
tensor, [11](#)
time complexity, [52](#)
Toffoli, [36](#)
topology, [69](#)
tradeoffs, [60](#)
Trotter-Suzuki, [105](#), [132](#)

uncompute, [89](#)
unequivocally distinguished, [22](#)
unitary, [15](#), [27](#)
universal, [76](#)
universality, [53](#)

variational, [126](#)
variational algorithms, [86](#)
vector space, [7](#)
vectors, [7](#)
VQE, [126](#)

wave, [71](#)

X, [31](#)

Y, [31](#)

Z, [31](#)
zero, [4](#)